Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015



Association for Standardisation of
Automation and Measuring Systems

ASAM MCD-2 MC (ASAP2 / A2L)

Data Model for ECU Measurement and Calibration

Version 1.7.0

Date: 2015-02-26

Base Standard

© by ASAM e.V., 2015

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Disclaimer

This document is the copyrighted property of ASAM e.V.
Any use is limited to the scope described in the license terms. The license terms can be viewed at www.asam.net/license

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Table of Contents

Foreword 9

1 Introduction 10

1.1 Overview 10

1.2 Motivation 10

1.3 Scope 10

1.4 History 11

2 Relations to Other Standards 12

2.1 Backward Compatibility to Earlier Releases 12

2.1.1 Incompatibility of Keyword FORMULA 12

2.1.2 Restriction for Brackets 13

2.1.3 Win32 APIs for Seed&Key and Checksum Calculation 13

2.1.4 Optional numbers for MATRIX_DIM 13

2.2 References to Other Standards 13

2.2.1 ASAM MCD-2 MC within the ASAM AE Overall Context 13

2.2.2 ASAM MCD-1 14

2.2.3 ASAM MCD-2 14

2.2.3.1 ASAM MCD-2 D ODX 15

2.2.3.2 ASAM MCD-2 NET 15

2.2.4 ASAM MCD-3 15

2.2.5 More ASAM Standards with an ASAM MCD-2 MC relation 15

3 Division of the description data 16

4 Format of the description file 18

4.1 Encoding of the A2L file 18

4.1.1 Unicode Transformation Format 18

4.1.2 Byte-Order Mark 18

4.2 Hierarchic division of the keywords 19

4.3 Name Spaces 28

4.3.1 Functions 28

4.3.2 Groups 28

4.3.3 Variants 28

4.3.4 Measure and adjustable objects 28

4.3.5 Transformer 28

4.3.6 Conversion methods 28

4.3.7 Conversion tables 28

4.3.8 Memory segments 28

4.3.9 Record layouts 29

4.3.10 Type definitions 29

4.3.11 Structure components 29

4.4 Predefined data types 29

4.5 Mapping of predefined data Types to ASAM data types 32

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.6 Comments 32

4.7 Alphabetical list of keywords 33

4.7.1 General 33
4.7.2 A2ML 34
4.7.3 A2ML_VERSION 35
4.7.4 ADDR_EPK 36
4.7.5 ADDRESS_TYPE 37
4.7.6 ALIGNMENT_BYTE 38
4.7.7 ALIGNMENT_FLOAT32_IEEE 39
4.7.8 ALIGNMENT_FLOAT64_IEEE 40
4.7.9 ALIGNMENT_INT64 41
4.7.10 ALIGNMENT_LONG 42
4.7.11 ALIGNMENT_WORD 43
4.7.12 ANNOTATION 44
4.7.13 ANNOTATION_LABEL 45
4.7.14 ANNOTATION_ORIGIN 46
4.7.15 ANNOTATION_TEXT 47
4.7.16 ARRAY_SIZE 48
4.7.17 ASAP2_VERSION 49
4.7.19 AXIS_DESCR 50
4.7.20 AXIS_PTS 54
4.7.21 AXIS_PTS_REF 58
4.7.22 AXIS_PTS_X / _Y / _Z / _4 / _5 60
4.7.23 AXIS_RESCALE_X 61
4.7.24 BIT_MASK 63
4.7.25 BIT_OPERATION 64
4.7.26 BLOB 65
4.7.27 BYTE_ORDER 67
4.7.28 CALIBRATION_ACCESS 69
4.7.29 CALIBRATION_HANDLE 70
4.7.30 CALIBRATION_HANDLE_TEXT 71
4.7.31 CALIBRATION_METHOD 72
4.7.32 CHARACTERISTIC 73
4.7.33 COEFFS 79
4.7.34 COEFFS_LINEAR 80
4.7.36 COMPARISON_QUANTITY 81
4.7.37 COMPU_METHOD 82
4.7.38 COMPU_TAB 86
4.7.39 COMPU_TAB_REF 88
4.7.40 COMPU_VTAB 89
4.7.41 COMPU_VTAB_RANGE 91
4.7.42 CONSISTENT_EXCHANGE 93
4.7.43 CONVERSION 94
4.7.44 CPU_TYPE 95
4.7.45 CURVE_AXIS_REF 96
4.7.46 CUSTOMER 98
4.7.47 CUSTOMER_NO 99
4.7.48 DATA_SIZE 100
4.7.49 DEF_CHARACTERISTIC 101
4.7.50 DEFAULT_VALUE 102
4.7.51 DEFAULT_VALUE_NUMERIC 103
4.7.52 DEPENDENT_CHARACTERISTIC 104
4.7.53 DEPOSIT 106

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.54 DISCRETE 107
4.7.55 DISPLAY_IDENTIFIER 108
4.7.56 DIST_OP_X / _Y / _Z / _4 / _5 109
4.7.57 ECU 110
4.7.58 ECU_ADDRESS 111
4.7.59 ECU_ADDRESS_EXTENSION 112
4.7.60 ECU_CALIBRATION_OFFSET 113
4.7.61 ENCODING 114
4.7.62 EPK 115
4.7.63 ERROR_MASK 116
4.7.64 EXTENDED_LIMITS 117
4.7.65 FIX_AXIS_PAR 118
4.7.66 FIX_AXIS_PAR_DIST 119
4.7.67 FIX_AXIS_PAR_LIST 120
4.7.68 FIX_NO_AXIS_PTS_X / _Y / _Z / _4 / _5 121
4.7.69 FNC_VALUES 122
4.7.70 FORMAT 124
4.7.71 FORMULA 125
4.7.72 FORMULA_INV 128
4.7.73 FRAME 129
4.7.74 FRAME_MEASUREMENT 130
4.7.75 FUNCTION 131
4.7.76 FUNCTION_LIST 133
4.7.77 FUNCTION_VERSION 134
4.7.78 GROUP 135
4.7.79 GUARD_RAILS 139
4.7.80 HEADER 140
4.7.81 IDENTIFICATION 141
4.7.82 IF_DATA (Example) 142
4.7.83 IN_MEASUREMENT 143
4.7.84 INPUT_QUANTITY 144
4.7.85 INSTANCE 145
4.7.86 LAYOUT 148
4.7.87 LEFT_SHIFT 149
4.7.88 LIMITS 150
4.7.89 LOC_MEASUREMENT 151
4.7.90 MAP_LIST 152
4.7.91 MATRIX_DIM 153
4.7.92 MAX_GRAD 154
4.7.93 MAX_REFRESH 155
4.7.94 MEASUREMENT 156
4.7.95 MEMORY_LAYOUT 160
4.7.96 MEMORY_SEGMENT 162
4.7.97 MOD_COMMON 166
4.7.98 MOD_PAR 168
4.7.100 MODEL_LINK 170
4.7.101 MODULE 171
4.7.102 MONOTONY 173
4.7.103 NO_AXIS_PTS_X / _Y / _Z / _4 / _5 174
4.7.104 NO_OF_INTERFACES 175
4.7.105 NO_RESCALE_X 176
4.7.106 NUMBER 177
4.7.107 OFFSET_X / _Y / _Z / _4 / _5 178
4.7.108 OUT_MEASUREMENT 179

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.109 OVERWRITE ... 180
4.7.110 PHONE_NO ... 183
4.7.111 PHYS_UNIT ... 184
4.7.112 PROJECT ... 185
4.7.113 PROJECT_NO ... 186
4.7.114 READ_ONLY ... 187
4.7.115 READ_WRITE ... 188
4.7.116 RECORD_LAYOUT ... 189
4.7.117 REF_CHARACTERISTIC ... 195
4.7.118 REF_GROUP ... 196
4.7.119 REF_MEASUREMENT ... 197
4.7.120 REF_MEMORY_SEGMENT /DEPRECATED ... 198
4.7.121 REF_UNIT ... 199
4.7.122 RESERVED ... 200
4.7.123 RIGHT_SHIFT ... 201
4.7.124 RIP_ADDR_W / _X / _Y / _Z / _4 / _5 ... 202
4.7.125 ROOT ... 204
4.7.126 SHIFT_OP_X / _Y / _Z / _4 / _5 ... 205
4.7.127 SIGN_EXTEND ... 206
4.7.128 SI_EXPONENTS ... 207
4.7.129 SRC_ADDR_X / _Y / _Z / _4 / _5 ... 208
4.7.130 STATIC_ADDRESS_OFFSETS ... 209
4.7.131 STATIC_RECORD_LAYOUT ... 210
4.7.132 STATUS_STRING_REF ... 212
4.7.133 STEP_SIZE ... 213
4.7.134 STRUCTURE_COMPONENT ... 214
4.7.135 SUB_FUNCTION ... 216
4.7.136 SUB_GROUP ... 217
4.7.137 SUPPLIER ... 218
4.7.138 SYMBOL_LINK ... 219
4.7.139 SYSTEM_CONSTANT ... 220
4.7.140 TRANSFORMER ... 221
4.7.141 TRANSFORMER_IN_OBJECTS ... 225
4.7.142 TRANSFORMER_OUT_OBJECTS ... 226
4.7.143 TYPEDEF_AXIS ... 227
4.7.144 TYPEDEF_BLOB ... 229
4.7.145 TYPEDEF_CHARACTERISTIC ... 230
4.7.146 TYPEDEF_MEASUREMENT ... 232
4.7.147 TYPEDEF_STRUCTURE ... 234
4.7.148 UNIT ... 237
4.7.149 UNIT_CONVERSION ... 239
4.7.150 USER ... 240
4.7.151 USER_RIGHTS ... 241
4.7.152 VAR_ADDRESS ... 243
4.7.153 VAR_CHARACTERISTIC ... 244
4.7.154 VAR_CRITERION ... 245
4.7.155 VAR_FORBIDDEN_COMB ... 246
4.7.156 VAR_MEASUREMENT ... 247
4.7.157 VAR_NAMING ... 248
4.7.158 VAR_SELECTION_CHARACTERISTIC ... 249
4.7.159 VAR_SEPARATOR ... 250
4.7.160 VARIANT_CODING ... 251
4.7.161 VERSION ... 253
4.7.162 VIRTUAL ... 254

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.163 VIRTUAL_CHARACTERISTIC 255

5 Include mechanism 257

6 ASAM MCD-2 MC Metalanguage 258

6.1 General 258

6.2 Format of the ASAM MCD-2 MC metalanguage 259

6.3 Designing AML-file 262

7 Terms and Definitions 263

8 Symbols and Abbreviated Terms 264

9 Bibliography 265

Appendix: A. IEEE-Floating-Point-Format 267

A.1. 32-Bit Format 267

A.2. 64-Bit Format 267

Appendix: B. Using Reference Curves as Normalization Axes for Maps 268

B.1. Background 268

B.1.1. General 268
B.1.2. Overall technique 268
B.1.3. Determining the map indices 268
B.1.4. Determining the map normalized value 268

B.2. Example 269

Appendix: C. Codes for scaling units (CSE) 271

Appendix: D. Transformer 273

D.1. Basics 273

D.2. Transformer DLL API definition and rules 275

D.2.1. General API description 275
D.2.2. Detailed C transformer API definition 276
D.2.2.1. Typedefs 276
D.2.2.2. IsTransformationPossible DLL Function 279
D.2.2.3. GetTransformerDescription DLL Function 280
D.2.2.4. GetDllDescription DLL Function 281
D.2.2.5. ExecuteTransformation DLL Function 282

Appendix: E. ASAP2 Container 283

E.1. Container 283

E.2. Common 283

E.3. Category 284

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

E.4. ABLOCK 285
E.5. FILES 285
E.6. AREF 285
E.7. Example for Container Catalog File 286
Index of Keywords and Enum Values 291
Figure Directory 294
Table Directory 295

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Foreword

ASAM MCD-2 MC describes a data model for ECU Measurement and Calibration via a non XML format. Such data resides inside the memory of the ECU. The format definition makes the data accessible through application systems for tuning real-time testing. The description contains information about data types, dimensions, record layouts and memory locations of ECU variables. Further information describes how the variable values shall be converted into human-readable quantities and displayed in an MC-system.
In addition to the data description, the standard allows to describe the device interface between the MC-system and the ECU for read- and write access. As a result, the ASAM MCD-2 MC description contains all information in one place that is needed to access, modify, interpret and display ECU-internal variables.

The format definition describes the following content:

- A2L description file keywords
- AML format description
- AML template
- A2L and AML example

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

1 Introduction

1.1 Overview

ASAM MCD-2 MC defines a description format for internal ECU variables used for measurement and calibration purposes.
Calibration means the adaption of characteristics (scalars, curves and maps) within the functional code of ECUs to achieve and optimize an appropriate system behavior. These calibration operations are either performed manually by a calibration engineer or are executed by external client applications, such as optimization programs or test bed automation systems.
This adaption requires a WRITE-access to the ECU to set a new value or a READ-access to retrieve the current one.
Whether this calibration already produces the intended effect or not will normally be checked by inspection of other ECU variables by MEASUREMENT access. Therefore, the host tool configures so-called measurement tasks, transmits them to the ECU and henceforth takes the values which are automatically sent by the ECU.
The description is typically used by measurement & calibration systems (MC-systems) for tuning scalar constants, curves and maps of the ECU software and to record the system's response via measurement variables during real-time testing. The description contains information about data types, dimensions, record layouts and memory locations of ECU variables. Further information describes how the variable values shall be converted into human-readable quantities and displayed in an MC-system. Furthermore, it describes the HW interface of the ECU for device driver configuration of the application system. The standard is also known as "ASAP2".

1.2 Motivation

An essential part of ECU software development is the calibration of control strategy's parameters. This means the adaption of scalars, curves and maps to achieve an optimized and appropriate system behavior. Internal variables need to be read from the ECU to evaluate the effectiveness of the calibrated software. Such operations are carried out by tools which need a detailed description of the calibration parameters and internal variables. They furthermore need to have a description of the device interface to the ECU for read and write access. This description is typically produced by function developers, software engineers, tool & instrumentation experts, and is used by calibration engineers.
The standard allows the connection of software development tools, calibration tools and ECU calibration interfaces with a neutral description format (A2L). Tools that support the description format are able to exchange and process the included information. Hence, no vendor-specific or technology-specific dependencies are required between tools of an ASAM-compliant calibration tool-chain.

1.3 Scope

The ASAM MCD-2 MC standard (market name ASAP2 / A2L) was developed to take into consideration the needs of all groups involved in the calibration process. The standard defines a description format that describes the calibration parameters (called CHARACTERISTIC) and internal variables (called MEASUREMENT) of ECU software. The description includes elementary information like addresses, data types, dimensions,

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

identifiers and much more. To convert the ECU internal characteristic and measurement implementation values into physical values, ASAM MCD-2 MC describes computation methods for their conversion between both representations. Calibration engineers can work with the ECU data in a familiar format without having to understand ECU-internal data formats. Software engineers can provide this data to them or even get the description files automatically generated from code generators. An include mechanism ensures that description files can originate from different sources.
The ECU normally stores the measurement and calibration quantities internally in an implementation optimized format. This format is very often a fixed-point format. Outside the ECU physical models are used. The ASAM MCD-2 MC standard describes by so-called record layouts how data are stored inside the ECU and which computation methods are needed to transform the ECU internal data representation into the physical one and vice versa.
The ASAM MCD-2 MC standard also allows to describe and configure the ECU interfaces or vendor specific extensions by a meta description language (AML). For ASAM standardized ECU interfaces, such as CCP and XCP the content of these AML parts are also standardized. But there are also a lot of vendor specific instantiations in the market which use this mechanism.
Measurement and calibration tools are normally only used during development phase of ECUs. They allow a direct, address-oriented write- and read-access but also a synchronous, continuous measurement access to ECU internal variables.
MC tools also offer features for flashing of new software versions comprising of new code and/or new parameter sets using the relevant ASAM MCD-1 interfaces. For calibration and flashing purposes the ASAM MCD-2 MC standard describes the memory segment configuration to rebuild code and data externally.
Address-oriented information of ASAM MCD-2 MC files may become obsolete with a new ECU software version running a new compiler/linker run because variables may be relocated in the memory. Therefore, code generator tools but also other utilities very often generate or update ASAM MCD-2 MC files.
The ASAM MCD-2 MC standard is widely used in the automotive industry and supported by every major calibration tool on the market.
It is used by many code generators, calibration and diagnostics tools, rapid control prototyping tools, data loggers, measurement systems, automation systems, etc.
The standard is aligned with other ASAM standards like ASAM MCD-1 XCP/CCP, ASAM CDF and ASAM MCD-3 MC/ASAP3.

1.4 History

First versions of the standard were developed already before foundation of ASAM e.V. in 1998. These versions were named ASAP-2. They have been part of a 3-layer base architecture, which is described in chapter 2.2.
ASAP-2 was renamed to ASAM MCD-2 MC. MCD stands for Measurement, Calibration and Diagnostics.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

2 Relations to Other Standards

2.1 Backward Compatibility to Earlier Releases

The version ASAM MCD-2 MC V1.7.0 is in general downward compatible to the former version ASAM MCD-2 MC V1.6.1. This means that the keywords and keyword combinations used to describe ECU software are in the same way supported as in the former version.
The ASAM MCD-2 MC reuses the same notation format of the former ASAP2 standards to ensure a downward compatibility. This is necessary as a wide set of existing ASAP2 tools is in the market and a switch to a different language format (e.g. XML) leads to high efforts to modify these tools.
Because of the maintenance some details are no longer downward compatible. This chapter lists the critical ones.

2.1.1 Incompatibility of Keyword FORMULA

The former definition of the keyword was not compatible to ANSI-C notation. The ANSI-C compatibility is important to convert formula descriptions easily between different systems engineering system (compiler) and software description systems.
In detail these are the logical operators:
Since ASAM MCD-2 MC V1.6.0 (ANSI-C) the meaning of some operators differs from ASAM MCD-2 MC V1.5.

Table 1 formula operator compatibility

Operator	ASAM MCD-2 MC V1.5.1	ASAM MCD-2 MC V1.6.0 (ANSI-C)
&	logical AND	bitwise AND
|	logical OR	bitwise OR
XOR	exclusive OR	not supported
~	logical NOT	bitwise NOT
^	Power	bitwise exclusive OR
&&	not supported	logical AND
||	not supported	logical OR
!	not supported	logical NOT
ln(x)	supported, but not specified	not supported
log(x)	supported, but not specified	natural logarithm
log10(x)	not supported	decimal logarithm

Since ASAM MCD-2 MC V1.6.0 (ANSI-C) the notation of some operators differs from ASAM MCD-2 MC V1.5.

Table 2 formula operator notiation

ASAM MCD-2 MC V1.5.1	ASAM MCD-2 MC V1.6.0 (ANSI-C)
arcsin(x)	asin(x)
arcos(x)	acos(x)
arctan(x)	atan(x)

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Note: If the keyword ASAP2_VERSION is missing (former ASAM MCD-2 MC version) or states a version smaller than V1.6.0 tools shall use the formula interpretation of ASAM MCD-2 MC V1.5.1.

2.1.2 Restriction for Brackets

Since ASAM MCD-2 MC V1.6.0 always brackets of the form '/begin' '/end' are requested. Curly brackets '{' '}' are no longer supported.

2.1.3 Win32 APIs for Seed&Key and Checksum Calculation

The definition of Win32 APIs for Seed&Key and checksum calculation which was formerly part of the ASAM MCD-2 MC V1.5.1 specification is no longer part of this specifiation. These definitions are now found at ASAM Seed&Key [7].

2.1.4 Optional numbers for MATRIX_DIM

ASAM MCD-2 MC V1.7 defines the dimensions as optional numbers. This allows to add only the dimensions used.
In earlier versions (<= V1.6.1) always 3 dimensions are given. To describe objects with less than 3 dimensions ASAM MCD-2 MC V1.6.1 defined a rule that not used dimensions are stated as ‘1’.
This leads to inconsistency in case of usage of MATRIX_DIM for 1 or two dimensions. E.g. MATRIX_DIM 4 2 1 describes in V1.7 a 3-dimensional object (z-dimension = 1) and in V1.6.1 a 2-dimensional object.
To avoid conflicts MATRIX_DIM must be handled version dependent.

2.2 References to Other Standards

2.2.1 ASAM MCD-2 MC within the ASAM AE Overall Context

The MCD standards of ASAM Automotive Electronics are structured in a 3 layer system as follows:

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

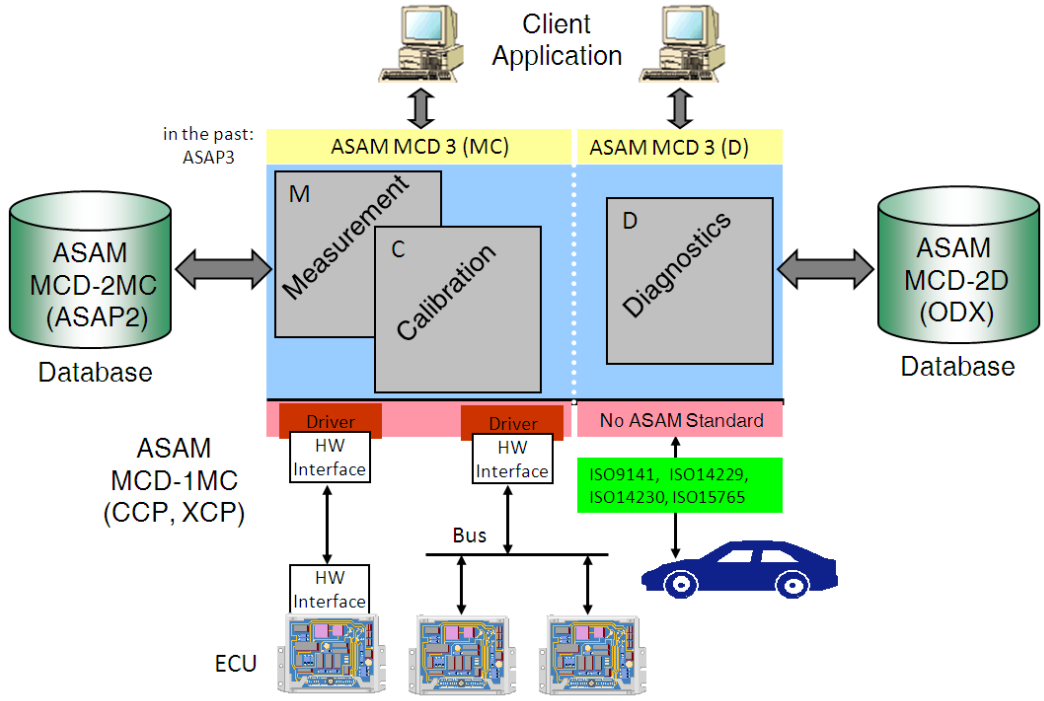


Figure 1 Structure of the MCD standards of ASAM AE

2.2.2 ASAM MCD-1

This summarizing term today denotes a set of standards to define the data interface primarily of ECUs. As shown in Figure 1 the ASAM defined interfaces are used only for measurement and calibration tasks.
Diagnostic protocols and interfaces are not standardized by ASAM but by ISO (ISO 9141 [8], 14229 [9], 14230 [10], 15765 [11] or the PDU-API in ISO/DIS 22900-2 [13]).
The most import standard in this sector, implemented in many ECUs worldwide, is the so-called CAN Calibration Prototocol. All market relevant calibration tools support this protocol as a base feature. But also specific measurement equipment, such as data loggers are able to connect to ECUs directly via the CAN bus.
Based on several years of experience with this CAN bus limited protocol ASAM developed the XCP standard, which in contrast to CCP is defined independently from the specific transport layer. Today several layers are available, such as CAN, USB, Ethernet, Flexray and SxI.
Very important for the success of XCP was the functional extension. As CCP was restricted to measurement and calibration, XCP also added a standardized option to write values back to the ECU in a synchronous manner (stimulation).
CCP as well as XCP allow to flash ECUs during development phase. This flashing access will normally be eliminated when the ECU is given to production and therefore is not available during service phase. Here the corresponding diagnostics services are applied.

2.2.3 ASAM MCD-2

The different ASAM MCD-2 formats build the data basis of an MCD system. ASAM MCD-2 MC describes the necessary data for an MC oriented access, ASAM MCD-2 D ODX [6] for a service-oriented diagnostics access and ASAM MCD-2 NET [3] for an access via the different bus systems.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

2.2.3.1 ASAM MCD-2 D ODX

ASAM MCD-2 D ODX in its versions since 2.0 is a unique, open XML exchange format for diagnostics data. The seamless data exchange between different partners along the process chain (suppliers, OEMs or service partners) is a very important process improvement.
Diagnostic tools like service testers or more development oriented tools can be parameterized via this format. The ODX standard defines an object-oriented data model, which is described in UML (Unified Modeling language). Inheritance and associations help to avoid data redundancies.
In contrast to the ASAM MCD-2 MC standard, ODX data describe the parameters and access information for a diagnostic service oriented ECU access.
The ODX standard is also available as ISO 22901-1 [12].

2.2.3.2 ASAM MCD-2 NET

ASAM MCD-2 NET is also an XML format. It is capable to describe the entire vehicle communication network. All relevant bus technologies, such as CAN, MOST, LIN or Flexray are supported. Information exchange between tools of different suppliers is a standard ASAM MCD-2 NET use case. Partial information can be completed step by step along the development progress.

2.2.4 ASAM MCD-3

ASAM MCD-3 specifies an object-oriented programming resp. remote-control interface to an MCD-server system.
This standard comprises of a base standard which is coded as a technology independent UML model with corresponding interface implementations for (D)COM, Java and C++.
The functional columns M (measurement), C (calibration) and D(diagnostics) can be applied independently of each other, but also in combined manner as M, MD, MC or entire MCD systems. Common parts, such as project or hardware setup, are defined as single source.
There is a clear market trend that more and more combined and integrated MCD systems will become available.
The diagnostics part of ASAM MCD-3 [4] is available as 22900-3 [14].

It is important to mention the well-established remote-control interface ASAP-3 MC [5]. This standard was developed in the early nineties for RS232 and TCP/IP based communication between test bed automation systems and calibration tools. It is still in usage in many applications until today.

2.2.5 More ASAM Standards with an ASAM MCD-2 MC relation

Beside the MCD standards additional standards either have been developed from scratch or have been modified and adapted based on pre-developments on MSR side.
The ASAM CDF [1] standard specifies an XML format to store calibration data, their level of maturity and other development process related data.

The ASAM MDF [2] standard allows to store measurement data in a very efficient binary format.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

3 Division of the description data

The definition of the ASAM MCD-2 MC interface and hence the specification of the ASAM MCD-2 MC data base is aimed at defining a database independently of a computer or an operating system in such a way that a transparent and manufacturer-independent standard is established. As exchange format for such ECU descriptions *.a2l files are used.
From the calibration point of view the database in accordance with the ASAM MCD-2 MC interface contains the complete description of all control unit relevant data in a project. A project consists of project specific header, which is typically created by the project manager, data and one or more control unit specific descriptions. These control unit descriptions (= description of an ECU) include all conversion formulas and explanations about the applicable (adjustable) and measurable (non-adjustable) quantities and present a format description of the interface specific parameters. The measurement and calibration system needs only to evaluate the quantities (and their conversion etc.), but not the interface specific parameters. The latter are only passed on to the structures of the driver. To make sure that these structures are correctly filled the MCD must know the parameter type. The type is communicated with the ASAM MCD-2 MC metalanguage. As exchange format for such ECU IF Data descriptions *.aml files are used.
A project may include the control unit descriptions of various control units from different suppliers. The descriptions differ in terms of content, but use a common information storage methodology to allow for a global management of the project components. An INCLUDE mechanism allows to summarize the various control unit descriptions of various projects (Single-Source-Concept).
The ASAM MCD-2 MC database thus consists of a number of different subcomponents structured in accordance with the following diagram. The MODULE keyword denotes an independent ECU or device.

```
/begin PROJECT
  /begin HEADER
    /* Project description */
  /end HEADER

  /begin MODULE          /* Specific Device description*/
    /begin MOD_PAR
           /* Control unit management data */
    /end   MOD_PAR

    /begin MOD_COMMON
           /* Module-wide (ECU specific) definitions */
    /end   MOD_COMMON

    /begin CHARACTERISTIC
           /* Adjustable objects */
    /end CHARACTERISTIC

    /begin CHARACTERISTIC
    /end CHARACTERISTIC

    /begin AXIS_PTS
           /* Axis points objects */
    /end AXIS_PTS

    /begin AXIS_PTS
    /end AXIS_PTS
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
        ...
        /begin MEASUREMENT
               /* Measurement objects */
        /end MEASUREMENT

        /begin MEASUREMENT
        /end MEASUREMENT
        ...
        /begin COMPU_METHOD
               /* Conversion method */
        /end COMPU_METHOD

        /begin COMPU_METHOD
               .
        /end COMPU_METHOD
        /begin COMPU_TAB
               /* Conversion tables */
        /end COMPU_TAB

        /begin COMPU_TAB
        /end COMPU_TAB
        ...
        /begin FUNCTION
               /* Function allocations */
        /end FUNCTION
        /begin FUNCTION
        /end FUNCTION
        ...
        /begin GROUP
               /* Groups */
        /end GROUP

        /begin GROUP
        /end GROUP
        ...
        /begin RECORD_LAYOUT
               /* Record layouts of adjustable objects */
        /end RECORD_LAYOUT

        /begin RECORD_LAYOUT
        /end RECORD_LAYOUT
      /end MODULE
      ...                    /* further device descriptions can follow */
   /end PROJECT              /* END OF PROJECT */
```

The following rules apply for a valid a2l file:
The ASAP2_VERSION keyword is mandatory and expected before the keyword PROJECT. The file must contain exactly one PROJECT. The PROJECT must contain at least one MODULE.
The keywords defined in the ASAM MCD-2 MC database are described in the following chapter.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4 Format of the description file

4.1 Encoding of the A2L file

ASAM MCD-2 MC files are used in different language areas. To support the different character sets used in the different language areas it is necessary to add to the ASAM MCD-2 MC file the information about the used character set.
The ASCII and ISO-8859-x character sets, defined for several language areas are not sufficient. The ASAM MCD-2 MC files needs to be exchangeable world wide.

4.1.1 Unicode Transformation Format

World wide exchange is supported by "Unicode Transformation Format" (UTF). Currently there are 3 relevant versions of UTF available: UTF -8, UTF-16, UTF-32.
UTF-8 is compact and supports nearly every possible character world wide. Therefore UTF-8 is the preferred encoding for ASAM MCD-2 MC files.

Note: Tools shall support at least UTF-8.

4.1.2 Byte-Order Mark

The encoding that is used for the ASAM MCD-2 MC file is defined in a Byte-Order Mark (BOM). The BOM is a byte sequence at the beginning of the ASAM MCD-2 MC file.

Currently defined BOM sequences:

Table 3 byte-order mark coding

Bytes	Encoding Form
00 00 FE FF	UTF-32, big-endian
FF FE 00 00	UTF-32, little-endian
FE FF	UTF-16, big-endian
FF FE	UTF-16, little-endian
EF BB BF	UTF-8

If no encoding can be detected, ISO-8859-1 (Latin-1) encoding is used.

Note: For data type "ident" the restrictions listed in chapter 3.2 "Predefined data types" are valid.

Note: For user defined tags and enum values in AML the restrictions defined in chapter 5.2 "Format of the ASAM MCD-2 MC Meta Language" are valid.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.2 Hierarchic division of the keywords

Table 4 Hierarchic division of the keywords

Keyword	Multiple	Meaning
ASAP2_VERSION		ASAM MCD-2 MC version identification
A2ML_VERSION		Version number of ASAM MCD-2 MC Meta Language
PROJECT		Project description
HEADER		Project header description
PROJECT_NO		Project number
VERSION		Project version number
MODULE	X	Description of the ECU
A2ML		ASAM MCD-2 MC Meta-Language (interface-specific description data)
AXIS_PTS	X	Axis points distribution
ANNOTATION	X	Set of notes
ANNOTATION_LABEL		Title of annotation
ANNOTATION_ORIGIN		Creator of annotation
ANNOTATION_TEXT		Text of annotation
BYTE_ORDER		Byte order of axis points
CALIBRATION_ACCESS		Access for calibration
DEPOSIT		Absolute or difference axis points
DISPLAY_IDENTIFIER		Optional display name
ECU_ADDRESS_EXTENSION		Address extension of the ECU address
EXTENDED_LIMITS		Extended range of values
FORMAT		Display format of axis points
FUNCTION_LIST		Function orientation
GUARD_RAILS		Indicates the use of guardrails
IF_DATA	X	Interface-specific description data
MAX_REFRESH		Maximum refresh rate
MODEL_LINK		Parameter name in the software model
MONOTONY		Monotony with respect to this axis
PHYS_UNIT		Physical unit of the axis points
READ_ONLY		'Read Only' attribute
REF_MEMORY_SEGMENT		Reference to memory segment
STEP_SIZE		Delta value
SYMBOL_LINK		Reference to symbol of linker map file
BLOB	X	Binary blob object
ADDRESS_TYPE		Address type
ANNOTATION	X	Description
ANNOTATION_LABEL		Title of annotation
ANNOTATION_ORIGIN		Creator of annotation

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Keyword	Multiple	Meaning
ANNOTATION_TEXT		Text of annotation
CALIBRATION_ACCESS		Access for calibration
DISPLAY_IDENTIFIER		Optional display name
ECU_ADDRESS _EXTENSION		Address extension of the ECU address
IF_DATA	X	Interface-specific description data
MAX_REFRESH		Maximum refresh rate
MODEL_LINK		Parameter name in the software model
SYMBOL_LINK		Reference to symbol of linker map file
CHARACTERISTIC	X	Adjustable objects
ANNOTATION	X	Description
ANNOTATION_LABEL		Title of annotation
ANNOTATION_ORIGIN		Creator of annotation
ANNOTATION_TEXT		Text of annotation
AXIS_DESCR	X	Axis description
ANNOTATION	X	Set of notes
ANNOTATION_LABEL		Title of annotation
ANNOTATION_ORIGIN		Creator of annotation
ANNOTATION_TEXT		Text of annotation
AXIS_PTS_REF		Reference to axis point distribution
BYTE_ORDER		Byte order of axis points
CURVE_AXIS_REF		Used to normalize or scale an axis
DEPOSIT		Absolute or difference axis points
EXTENDED_LIMITS		Extended limits, e.g. hard limits
FIX_AXIS_PAR		Fixed axis parameters
FIX_AXIS_PAR_DIST		Fixed axis parameters (variant)
FIX_AXIS_PAR_LIST		Fixed axis values
FORMAT		Display format of axis points
MAX_GRAD		Maximum gradient with respect to this axis
MONOTONY		Monotony with respect to this axis
PHYS_UNIT		Physical unit of the axis points
READ_ONLY		'Read Only' attribute
STEP_SIZE		Delta value
BIT_MASK		Bit mask
BYTE_ORDER		Byte order
CALIBRATION_ACCESS		Access for calibration
COMPARISON_QUANTITY		Comparison quantity
DEPENDENT_CHARACTERISTIC		References to characteristics
DISCRETE		Attribute for discrete object values

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Keyword	Multiple	Meaning
DISPLAY_IDENTIFIER		Optional display name
ECU_ADDRESS_EXTENSION		Address extension of the ECU address
ENCODING		Encoding for string interpretation
EXTENDED_LIMITS		Extended limits, e.g. hard limits
FORMAT		Display format of values
FUNCTION_LIST		Function orientation
GUARD_RAILS		Indicates the use of guardrails
IF_DATA	X	Interface-specific description data
MAP_LIST		For cuboids: comprising maps
MATRIX_DIM		Dimensions of multi-dimensional arrays
MAX_REFRESH		Maximum refresh rate
MODEL_LINK		Parameter name in the software model
NUMBER		Number of ASCII characters or fixed values
PHYS_UNIT		Physical unit of the characteristic values
READ_ONLY		'Read Only' attribute
REF_MEMORY_SEGMENT		Reference to memory segment
STEP_SIZE		Delta value
SYMBOL_LINK		Reference to symbol of linker map file
VIRTUAL_CHARACTERISTIC		Mark for being virtual
COMPU_METHOD	X	Conversion method
COEFFS		Coefficients for fractional rational function
COEFFS_LINEAR		Coefficients for linear function
COMPU_TAB_REF		Reference to conversion table
FORMULA		Conversion formula
FORMULA_INV		Inverse conversion formula
REF_UNIT		Reference to a measurement unit
STATUS_STRING_REF		Reference to an additional conversion table with status strings
COMPU_TAB	X	Conversion table
DEFAULT_VALUE		Default output string
DEFAULT_VALUE_NUMERIC		Default value
COMPU_VTAB	X	Verbal conversion table
DEFAULT_VALUE		Default output string
COMPU_VTAB_RANGE	X	Description of range based verbal conversion tables
DEFAULT_VALUE		Default output string
FRAME	X	Frame
FRAME_MEASUREMENT		Frame measurement objects
IF_DATA	X	Interface-specific description data
FUNCTION	X	Function description

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Keyword	Multiple	Meaning
ANNOTATION	X	Set of notes
ANNOTATION_LABEL		Title of annotation
ANNOTATION_ORIGIN		Creator of annotation
ANNOTATION_TEXT		Text of annotation
DEF_CHARACTERISTIC		Defined adjustable objects
FUNCTION_VERSION		Version of the function
IF_DATA	X	Interface-specific description data
IN_MEASUREMENT		Input quantity
LOC_MEASUREMENT		Local quantity
OUT_MEASUREMENT		Output quantity
REF_CHARACTERISTIC		Referenced adjustable objects
SUB_FUNCTION		Sub function of respective function
GROUP	X	Declaration of groups
ANNOTATION	X	Set of notes
ANNOTATION_LABEL		Title of annotation
ANNOTATION_ORIGIN		Creator of annotation
ANNOTATION_TEXT		Text of annotation
FUNCTION_LIST		Function list
IF_DATA	X	Interface-specific description data
REF_CHARACTERISTIC		Reference to characteristic objects
REF_MEASUREMENT		Reference to measurement objects
ROOT		Flag for root node
SUB_GROUP		Sub group
IF_DATA	X	Interface-specific description data
INSTANCE	X	Instance of a measure or adjustable object
ANNOTATION	X	Description
ANNOTATION_LABEL		Title of annotation
ANNOTATION_ORIGIN		Creator of annotation
ANNOTATION_TEXT		Text of annotation
CALIBRATION_ACCESS		Access for calibration
DISPLAY_IDENTIFIER		Optional display name
ECU_ADDRESS_EXTENSION		Address extension of the ECU address
IF_DATA	X	Interface-specific description data
LAYOUT		Layout of multi-dimensional arrays
MATRIX_DIM		Dimensions of multi-dimensional arrays
MAX_REFRESH		Maximum refresh rate
MODEL_LINK		Parameter name in the software model
OVERWRITE	X	Overwrite properties of typedef
CONVERSION		Overwrite conversion method
EXTENDED_LIMITS		Overwrite extended physical limits

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Keyword	Multiple	Meaning
FORMAT		Overwrite format
INPUT_QUANTITY		Overwrite input quantity
LIMITS		Overwrite physical limits
MONOTONY		Overwrite monotony
PHYS_UNIT		Overwrite physical unit
READ_WRITE		Marks the instance as writable
SYMBOL_LINK		Reference to symbol of linker map file
MEASUREMENT	X	Measurement object
ADDRESS_TYPE		Address type
ANNOTATION	X	Set of notes
ANNOTATION_LABEL		Title of annotation
ANNOTATION_ORIGIN		Creator of annotation
ANNOTATION_TEXT		Text of annotation
ARRAY_SIZE		Array size of measurement objects
BIT_MASK		Bit mask to decode single-bit values
BIT_OPERATION		Bit operation
LEFT_SHIFT		Number of bit positions to shift left
RIGHT_SHIFT		Number of bit positions to shift right
SIGN_EXTEND		Sign extension for measurement data
BYTE_ORDER		Byte order of measurement object
DISCRETE		Attribute for discrete object values
DISPLAY_IDENTIFIER		Optional display name
ECU_ADDRESS		Address
ECU_ADDRESS_EXTENSION		Address extension of the ECU address
ERROR_MASK		Mask error bits
FORMAT		Display format of measurement object
FUNCTION_LIST		Function orientation
IF_DATA	X	Interface-specific description data
LAYOUT		Layout of multi-dimensional arrays
MATRIX_DIM		Dimensions of multi-dimensional arrays
MAX_REFRESH		Refresh rate in the control unit
MODEL_LINK		Parameter name in the software model
PHYS_UNIT		Physical unit of the measurement values
READ_WRITE		'Writeable'
REF_MEMORY_SEGMENT		Reference to memory segment
SYMBOL_LINK		Reference to symbol of linker map file
VIRTUAL		Virtual measurement
MOD_COMMON		Module-wide (ECU specific) valid definitions
ALIGNMENT_BYTE		Alignment border for byte values
ALIGNMENT_FLOAT32_IEEE		Alignment border for float32 values

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Keyword	Multiple	Meaning
ALIGNMENT_FLOAT64_IEEE		Alignment border for float64 values
ALIGNMENT_INT64		Alignment border for int64 values
ALIGNMENT_LONG		Alignment border for long values
ALIGNMENT_WORD		Alignment border for word values
BYTE_ORDER		Byte order
DATA_SIZE		Data size in bits
DEPOSIT		Standard deposit mode for axis
MOD_PAR		Control unit management data
ADDR_EPK	X	Address of EPROM identifier
CALIBRATION_METHOD	X	Access method
CALIBRATION_HANDLE	X	Handle for calibration method
CALIBRATION_HANDLE _TEXT		Additional Text for caibration method
CPU_TYPE		CPU
CUSTOMER		Firm or customer
CUSTOMER_NO		Customer number
ECU		Control unit
ECU_CALIBRATION_OFFSET		Address offset
EPK		EPROM identifier
MEMORY_LAYOUT	X	Memory layout
IF_DATA	X	Interface-specific description data
MEMORY_SEGMENT	X	Memory segment
IF_DATA	X	Interface-specific description data
NO_OF_INTERFACES		Number of interfaces
PHONE_NO		Phone number of calibration engineer responsible
SUPPLIER		Manufacturer or supplier
SYSTEM_CONSTANT	X	System-defined constants
USER		User
VERSION		Module-specific version identifier
RECORD_LAYOUT	X	Description of the record layout
ALIGNMENT_BYTE		Alignment border for byte values
ALIGNMENT_FLOAT32_IEEE		Alignment border for float32 values
ALIGNMENT_FLOAT64_IEEE		Alignment border for float64 values
ALIGNMENT_INT64		Alignment border for int64 values
ALIGNMENT_LONG		Alignment border for long values
ALIGNMENT_WORD		Alignment border for word values
AXIS_PTS_X / _Y / _Z / _4 / _5		Axis points
AXIS_RESCALE_X		Rescaling axis points
DIST_OP_X / _Y / _Z / _4 / _5		Parameter 'distance' for fixed characteristics

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Keyword	Multiple	Meaning
FIX_NO_AXIS_PTS_X / _Y / _Z / _4 / _5		Fixed number of axis points
FNC_VALUES		Table values
IDENTIFICATION		Identification
NO_AXIS_PTS_X / _Y / _Z / _4 / _5		Number of X axis points
NO_RESCALE_X		Number of rescale pairs for axis
OFFSET_X / _Y / _Z / _4 / _5		Parameter 'offset' for fixed characteristics
RESERVED	X	Parameter is skipped (not interpreted)
RIP_ADDR_W		Table value: Address 'result of interpolation'
RIP_ADDR_X / _Y / _Z / _4 / _5		Address 'result of interpolation'
SHIFT_OP_X / _Y / _Z / _4 / _5		Parameter 'shift' for fixed characteristics
SRC_ADDR_X / _Y / _Z / _4 / _5		Address of input quantity
STATIC_ADDRESS_OFFSETS		Static addresses for non-compact data used, FNC Values handled dynamically
STATIC_RECORD_LAYOUT		Static addresses for non-compact data used, FNC Values handled statically
TRANSFORMER	X	Transformation of ECU calibration BLOBs
TRANSFORMER_IN_ OBJECTS		Transformer Input Object List
TRANSFORMER_OUT_ OBJECTS		Transformer Output Object List
TYPEDEF_AXIS	X	Type definition for axis points
BYTE_ORDER		Byte order of axis points
DEPOSIT		Absolute or difference axis points
EXTENDED_LIMITS		Extended range of values
FORMAT		Display format of axis points
MONOTONY		Monotony with respect to this axis
PHYS_UNIT		Physical unit of the axis points
STEP_SIZE		Delta value
TYPEDEF_BLOB	X	Type definition for blob objects
ADDRESS_TYPE		Address type
TYPEDEF_CHARACTERISTIC	X	Type definition for an adjustable object
AXIS_DESCR	X	Axis description
ANNOTATION	X	Set of notes
ANNOTATION_LABEL		Title of annotation
ANNOTATION_ORIGIN		Creator of annotation
ANNOTATION_TEXT		Text of annotation
AXIS_PTS_REF		Reference to axis point distribution
BYTE_ORDER		Byte order of axis points
CURVE_AXIS_REF		Used to normalize or scale an axis

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Keyword	Multiple	Meaning
DEPOSIT		Absolute or difference axis points
ENCODING		Encoding for string interpretation
EXTENDED_LIMITS		Extended limits, e.g. hard limits
FIX_AXIS_PAR		Fixed axis parameters
FIX_AXIS_PAR_DIST		Fixed axis parameters (variant)
FIX_AXIS_PAR_LIST		Fixed axis values
FORMAT		Display format of axis points
MAX_GRAD		Maximum gradient with respect to this axis
MONOTONY		Monotony with respect to this axis
PHYS_UNIT		Physical unit of the axis points
READ_ONLY		'Read Only' attribute
STEP_SIZE		Delta value
BIT_MASK		Bit mask
BYTE_ORDER		Byte order
DISCRETE		Attribute for discrete object values
EXTENDED_LIMITS		Extended limits, e.g. hard limits
FORMAT		Display format of values
MATRIX_DIM		Dimensions of multi-dimensional arrays
NUMBER		Number of ASCII characters or fixed values
PHYS_UNIT		Physical unit of the characteristic values
STEP_SIZE		Delta value
TYPEDEF_MEASUREMENT	X	Type definition for measurement object
ADDRESS_TYPE		Address Type
BIT_MASK		Bit mask to decode single-bit values
BIT_OPERATION		Bit operation
LEFT_SHIFT		Number of bit positions to shift left
RIGHT_SHIFT		Number of bit positions to shift right
SIGN_EXTEND		Sign extension for measurement data
BYTE_ORDER		Byte order of measurement object
DISCRETE		Attribute for discrete object values
ERROR_MASK		Mask error bits
FORMAT		Display format of measurement object
LAYOUT		Layout of multi-dimensional arrays
MATRIX_DIM		Dimensions of multi-dimensional arrays
PHYS_UNIT		Physical unit of the measurement values
TYPEDEF_STRUCTURE	X	Structure definition
ADDRESS_TYPE		Address type
CONSISTENT_EXCHANGE		Ensures consistent access to the structure

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Keyword	Multiple	Meaning
STRUCTURE_COMPONENT	X	Definition of a structure component
LAYOUT		Layout of multi-dimensional arrays
MATRIX_DIM		Dimensions of multi-dimensional arrays
UNIT	X	Measurement unit
REF_UNIT		Reference to another unit
SI_EXPONENTS		Exponential of base dimensions
UNIT_CONVERSION		Specifies relationship between two units
USER_RIGHTS	X	Groups with constitute access rights
READ_ONLY		Read only
REF_GROUP	X	List of referenced groups
VARIANT_CODING		Variant coding
VAR_CHARACTERISTIC	X	Definition of variant coded adjustable objects
VAR_ADDRESS		Adjustable objects address list (start address of variants)
VAR_CRITERION	X	Definition of variant criterion
VAR_MEASUREMENT		Measurement object which indicates criterion value
VAR_SELECTION_ CHARACTERISTIC		Characteristic object which modifies criterion value
VAR_FORBIDDEN_COMB	X	Forbidden combinations of different variants
VAR_NAMING		Naming of variant coded adjustable objects
VAR_SEPERATOR		Separator of adjustable objects names

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.3 Name Spaces

In an ASAP2 file, there are different object types which can be referenced via their object names. To be able to resolve these references it is necessary to have unique identifiers. The unambiguousness of identifiers is not required in the whole ASAP2 file, but within special name spaces. This chapter defines the name spaces for different keywords.

4.3.1 Functions

Within one MODULE, each FUNCTION identifier must be unique. It is not allowed to have two functions with the same identifier.

4.3.2 Groups

Within one MODULE, each GROUP identifier must be unique. It is not allowed to have two groups with the same identifier.

4.3.3 Variants

Within one VARIANT_CRITERION, each variant identifier must be unique. It is not allowed to have two variants with the same identifier.

4.3.4 Measure and adjustable objects

Within one MODULE, each measure and adjustable object identifier must be unique. It is not allowed to have two objects with the same identifier. Relevant keywords for measure and adjustable objects are MEASUREMENT, AXIS_PTS, CHARACTERISTIC, BLOB and INSTANCE.
Furthermore, the full identifiers of structure components belong to this name space. They are built by adding the component name(s) to the instance name separated by dot(s). See keyword INSTANCE. So it is also not allowed to have e.g. a measure object and a structure component with same identifier (see also Note: at Keyword INSTANCE).

4.3.5 Transformer

Within one MODULE, each TRANSFORMER identifier must be unique. It is not allowed to have two transformers with the same identifier.

4.3.6 Conversion methods

Within one MODULE, each COMPU_METHOD identifier must be unique. It is not allowed to have two conversion methods with the same identifier.

4.3.7 Conversion tables

Within one MODULE, each conversion table identifier must be unique. Relevant keywords for conversion tables are COMPU_TAB, COMPU_VTAB and COMPU_VTAB_RANGE.
It is not allowed to have two conversion tables with the same identifier.

4.3.8 Memory segments

Within one MODULE, each MEMORY_SEGMENT identifier must be unique. It is not allowed to have two memory segments with the same identifier.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.3.9 Record layouts

Within one MODULE, each RECORD_LAYOUT identifier must be unique. It is not allowed to have two record layouts with the same identifier.

4.3.10 Type definitions

Within one MODULE, each type definition identifier must be unique. Relevant keywords for type definitions are TYPEDEF_MEASUREMENT, TYPEDEF_AXIS, TYPEDEF_CHARACTERISTIC, TYPEDEF_STRUCTURE and TYPEDEF_BLOB.
It is not allowed to have two type definitions with the same identifier.

4.3.11 Structure components

Within one structure definition, each STRUCTURE_COMPONENT identifier must be unique. It is not allowed to have two components with the same identifier.

4.4 Predefined data types

Table 5 Predefined data types

Pre-defined data type		**description**
ident	typedef char [MAX_IDENT + 1] ident	String with *MAX_IDENT (at present = 1024)* alphanumerical characters including points and brackets, interpreted as hierarchical concatenation of partial strings separated by points. Every partial string may not exceed *MAX_PARTIAL_IDENT (at present = 128)* characters, including the length of an optional array index (numeric or as a symbolic string) in brackets at the end of the partial string. One string without a point in between is also possible, in this case MAX_IDENT = MAX_PARTIAL_IDENT. The number of partial strings within ident is not limited. The character chain must correspond with the identifier laws defined in programming language C. Identifiers can represent instances of array elements or instances of elements of complex C types or nested combinations of these. An instance of the element of a struct type would be represented by the concatenation of the instance name, a point and the element name. An instance of an array element would be represented by an instance name followed by a pair of brackets which contain either a numeric value or a symbolic string which is defined as an enumerator of an ENUM definition of the C program. Identifiers are random names which may contain characters A through Z, a through z, underscore (_), numerals 0 through 9, points (‘.’) and brackets (‘[‘,’]’) . However, the following limitations apply: the first character must be a letter or an underscore, brackets must occur in pairs at the end of a partial string and must contain a number or an alphabetic string (description of the index of an array element). Note: Identifiers consisting of partial identifiers separated by points (concatenation of instance name and element name) may be presented by

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Pre-defined data type		description
		the MCD system in a hierarchical manner (show instance name first, then allow access to an element of the instance). This allows existing MCD systems to restrict the display length of the identifier to MAX_PARTIAL_IDENT. Note: Identifiers generally must not match to the following defined ASAM MCD-2 MC keywords and enum values. All keyword and enum values are listed in the Index of Keywords and Enum Values. Note: A lower case “x” and a upper case “X” can be used as unique identifiers. In other words, variables are case sensitive so that x and X are different identifiers.
string	typedef char [MAX_STRING + 1] string	ANSI C compliant ‘C type’ string with maximum MAX_STRING (at present = 255) characters**.** Begin and end of the string are indicated by a double inverted comma. The following escape sequences are allowed: \´ inverted comma \” quotation mark \\ backslash \n new line \r carriage return \t horizontal tab Additionally, for compatibility with ASAP2 V1.2 and prior, the following is allowed: “” quotation mark Examples: "hello \"world\" how are you ?" "hello ""world"" how are you ?" MCD systems may ignore the carriage return sequence and/or apply wrapping or scrolling of strings when displayed. Please note that new line can only be inserted with the shown escape sequence.
float	8-byte floating point number (IEEE format)	The character for a decimal point is fixed as a dot “.”. A comma “,” is not allowed for use as a decimal point. Alternatively the exponential representation can be used, e.g. 12E-2. Numbers without dot are accepted as well.
int	2-byte signed integer	The notation of hexadecimal values is fixed, e.g. 0xE0, 0xFF, etc.
uint	2-byte unsigned integer	
uint64	8- byte unsigned integer	
long	4-byte signed integer	
ulong	4-byte unsigned integer	

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Pre-defined data type		description
datatype	typedef enum datatype { UBYTE, SBYTE, UWORD, SWORD, ULONG, SLONG, A_UINT64, A_INT64, FLOAT32_IEEE, FLOAT64_IEEE }	Enumeration for description of the basic data types in the ECU program (format of FLOAT32/64_IEEE: see Appendix: A). Note: If ECU values of type integer 64 are converted in physical values depending on the computation formula a higher precision in the physical area is necessary. The currently used float 64 format supports less precision than int 64. Therefore the precision of the physical representation is reduced to the precision of float 64. This is relevant for ASAM MCD-3 standards [4] where data transfer is defined as physical. This is additionally relevant for all tools working on PCs / Operating systems that do not support higher precision than float 64. Here the representation is rounded for physical and maybe also for internal representation.
datasize	typedef enum datasize { BYTE, WORD, LONG }	Enumeration for description of the word lengths in the ECU program
addrtype	typedef enum addrtype { PBYTE PWORD PLONG PLONGLONG DIRECT }	Enumeration for description of the addressing: The object address is interpreted as a 1 byte pointer to the object data. The object address is interpreted as a 2 byte pointer to the object data. The object address is interpreted as a 4 byte pointer to the object data. The object address is interpreted as an 8 byte pointer to the object data. The object address is interpreted as object data.
byteorder	typedef enum byteorder { LITTLE_ENDIAN, BIG_ENDIAN, MSB_LAST, MSB_FIRST, MSB_FIRST_MSW_LAST, MSB_LAST_MSW_FIRST }	Enumeration for description of the byte order in the control unit program. Note: Use of LITTLE_ENDIAN and BIG_ENDIAN defined with keyword BYTE_ORDER leads to mistakes because it is in contradiction to general use of terms „little endian“ and „big endian“. The keywords LITTLE_ENDIAN and BIG_ENDIAN should no longer be used, they should be replaced by MSB_LAST and MSB_FIRST which are equivalent (definition of MSB_LAST and MSB_FIRST: see keyword BYTE_ORDER).
indexorder	typedef enum indexorder { INDEX_INCR, INDEX_DECR }	Enumeration for description of the axis point sequence in the memory. Increasing index with increasing address decreasing index with increasing address

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.5 Mapping of predefined data Types to ASAM data types

The following table shows, how the ASAM data types are mapped to predefined data types.

Table 6 Map ASAM data types to predefined data types

ASAM data type	predefined data type
A_INT16	int
A_UINT16	uint
A_INT32	long
A_UINT32	ulong
A_FLOAT64	float

4.6 Comments

Single line and multi line comments may be added everywhere in an aml and a2l file.

Single line comments start with the character string "//" and end at the end of the same line. Multi line comments start with the character string "/*" and end with the character string "*/". Nested multi line comments are not allowed.

Example for a single line comment:

```
// This is a single line comment
```

Example for a multi line comment

```
/*
This is a
multi line comment
*/
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7 Alphabetical list of keywords

4.7.1 General

Some individual elements of the database are delimited by ‘/begin’ and ‘/end’ keywords. The delimiters are applied to those elements that contain an optional part, to prevent ambiguous expressions. The delimiters following defined with the ASAM MCD-2 MC keywords are mandatory, i.e. the delimiters have to be used if defined and mustn’t be used if not defined.
Optional keywords are shown with help of square brackets, which include an arrow followed by the keyword. If the keyword can be used multiple times this is shown with help of asterisk after the closing bracket, e.g. [-> keyword]*.
The description of the parameters of the keyword you find at the keyword description itself. The parameter description consists of the type and the identifier. If the parameters of a keyword can be used multiple times they are defined inside parenthesis followed by asterisk or plus, i.e. (parameter_type parameter_identifier)* or (parameter_type parameter_identifier)$^{+}$.
Multiple times means for asterix that an expression can appear 0 till n times, for plus 1 till n times.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.2 A2ML

Prototype:

```
/begin A2ML          FormatSpecification
/end A2ML
```

Parameters:

FormatSpecification	AML code for description of interface specific description data.

Description:

This keyword identifies the format description of the interface specific description data.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.3 A2ML_VERSION

Prototype:

```
A2ML_VERSION        uint          VersionNo
                    Uint          UpgradeNo
```

Parameters:

uint	VersionNo	Version number of AML part
uint	UpgradeNo	Upgrade number of AML part

Description:

The reason for this keyword is, to declare what kind of BLOBs should be generated from the AML parts. Since ASAP2 version 1.3.1 a specification for the storage layout of the BLOBs exist. The keyword is optional. When the keyword is omitted, or the version number is below 1.3.1 then the old BLOB format is used. When the A2ML version number is 1.3.1, then the new format must be generated.

The A2ML version can be expressed by two numerals:

- VersionNo
- UpgradeNo

where ‘VersionNo’ represents the main version number and ‘UpgradeNo’ the upgrade number (fractional part of version number).
This keyword will not be interpreted semantically anymore.

Example:

```
A2ML_VERSION              1
                          31    /* Version 1.3.1 */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.4 ADDR_EPK

Prototype:

```
ADDR_EPK            ulong        Address
```

Parameters:

ulong	Address	Address of the EPROM identifier

Description:

Address of the EPROM identifier

Example:

```
ADDR_EPK                0x145678
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.5 ADDRESS_TYPE

Prototype:

```
ADDRESS_TYPE        addrtype      AddressType
```

Parameters:

addrtype	AddressType	Type of the address (See in Table 5 enum addrtype)

Description:

Dependent on the ECU addressing a different address width is necessary.

Note: If an adjustable or measurable object is defined with indirect addressing (ADDRESS_TYPE is not DIRECT) and if the used interface does not support indirect addressing, it is the responsibility of the MC-System to dereference the object's address before accessing the data or configuring measurement lists.

Example:

```
ADDRESS_TYPE              PWORD
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.6 ALIGNMENT_BYTE

Prototype:

```
ALIGNMENT_BYTE      uint          AlignmentBorder
```

Parameters:

uint	AlignmentBorder	describes the border at which the value is aligned to, i.e. its memory address must be dividable by the value AlignmentBorder.

Description:

In complex objects (maps and axis) the alignment of a value may not coincide with the bitwidth of a value. This keyword is used to define the alignment in the case of bytes.

Example:

```
ALIGNMENT_BYTE          4       /* bytes have a 4-byte alignment */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.7 ALIGNMENT_FLOAT32_IEEE

Prototype:

```
ALIGNMENT_FLOAT32_IEEE        uint            AlignmentBorder
```

Parameters:

uint	AlignmentBorder	describes the border at which the value is aligned to, i.e. its memory address must be dividable by the value AlignmentBorder.

Description:

In complex objects (maps and axis) the alignment of a value may not coincide with the bitwidth of a value. This keyword is used to define the alignment in the case of 32bit floats.

Example:

```
ALIGNMENT_FLOAT32_IEEE   4       /* 32bit floats have a 4-byte alignment */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.8 ALIGNMENT_FLOAT64_IEEE

Prototype:

```
ALIGNMENT_FLOAT64_IEEE        uint            AlignmentBorder
```

Parameters:

uint	AlignmentBorder	describes the border at which the value is aligned to, i.e. its memory address must be dividable by the value AlignmentBorder.

Description:

In complex objects (maps and axis) the alignment of a value may not coincide with the bitwidth of a value. This keyword is used to define the alignment in the case of 64bit floats.

Example:

```
ALIGNMENT_ FLOAT64_IEEE  8      /* 64bit floats have a 8-byte alignment */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.9 ALIGNMENT_INT64

Prototype:

```
ALIGNMENT_INT64     uint          AlignmentBorder
```

Parameters:

uint	AlignmentBorder	describes the border at which the value is aligned to, i.e. its memory address must be dividable by the value AlignmentBorder.

Description:

In complex objects (maps and axis) the alignment of a value may not coincide with the bitwidth of a value. This keyword is used to define the alignment in the case of int64.

Example:

```
ALIGNMENT_INT64          4      /* int64 have a 4-byte alignment */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.10 ALIGNMENT_LONG

Prototype:

```
ALIGNMENT_LONG      uint          AlignmentBorder
```

Parameters:

uint	AlignmentBorder	describes the border at which the value is aligned to, i.e. its memory address must be dividable by the value AlignmentBorder.

Description:

In complex objects (maps and axis) the alignment of a value may not coincide with the bitwidth of a value. This keyword is used to define the alignment in the case of longs.

Example:

```
ALIGNMENT_LONG          8       /* longs have a 8-byte alignment */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.11 ALIGNMENT_WORD

Prototype:

```
ALIGNMENT_WORD      uint          AlignmentBorder
```

Parameters:

uint	AlignmentBorder	describes the border at which the value is aligned to, i.e. its memory address must be dividable by the value AlignmentBorder.

Description:

In complex objects (maps and axis) the alignment of a value may not coincide with the bitwidth of a value. This keyword is used to define the alignment in the case of words. The alignment is 2 if the parameter is missing.

Example:

```
ALIGNMENT_WORD          4      /* words have a 4-byte alignment */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.12 ANNOTATION

Prototype:

```
/begin ANNOTATION
                       [-> ANNOTATION_LABEL]
                       [-> ANNOTATION_ORIGIN]
                       [-> ANNOTATION_TEXT]
/end ANNOTATION
```

Parameters:

none

Optional Parameters:

-> ANNOTATION_LABEL	label or title of the annotation
-> ANNOTATION_ORIGIN	creator or creating system of the annotation
-> ANNOTATION_TEXT	text of the annotation, voluminous description text

Description:

One ANNOTATION may represent a voluminous description. Its purpose is to be e.g. an application note which explains the function of an identifier for the calibration engineer.

Example:

```
/begin CHARACTERISTIC  annotation.example1
....
   /begin ANNOTATION
      ANNOTATION_LABEL     "Luftsprungabhängigkeit"
      ANNOTATION_ORIGIN    "Graf Zeppelin"
      /begin ANNOTATION_TEXT
         "Die luftklasseabhängigen Zeitkonstanten t_hinz\r\n"
         "& t_kunz können mit Hilfe von Luftsprüngen ermittelt werden.\r\n"
         "Die Taupunktendezeiten in großen Flughöhen sind stark "
         "schwankend."
      /end ANNOTATION_TEXT
   /end ANNOTATION

   /begin ANNOTATION
      ANNOTATION_LABEL     "Taupunktendezeiten"
      /begin ANNOTATION_TEXT
         "Flughöhe         Taupunktendezeit\r\n"
         " 13000ft         20 sec\r\n"
         " 25000ft         40 sec\r\n"
         " 35000ft         12 sec"
      /end ANNOTATION_TEXT
   /end ANNOTATION
....
/end CHARACTERISTIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.13 ANNOTATION_LABEL

Prototype:

```
ANNOTATION_LABEL    string       label
```

Parameters:

string	label	label or title of the annotation

Description:

Assign a title to an annotation. Useful as a definition can contain more than one annotation.
Recommendation: The ANNOTATION_LABEL shall describe the use-case of the ANNOTATION, e.g. „Calibration Note“.

Example:

```
ANNOTATION_LABEL          "Calibration Note"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.14 ANNOTATION_ORIGIN

Prototype:

```
ANNOTATION_ORIGIN   string     origin
```

Parameters:

string	origin	creator or creating system of the annotation

Description:

To identify who or which system has created an annotation.

Example:

```
ANNOTATION_ORIGIN       "from the calibration planning department"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.15 ANNOTATION_TEXT

Prototype:

```
/begin ANNOTATION_TEXT
                    (string    annotation_text)*
/end ANNOTATION_TEXT
```

Parameters:

string	annotation_text

Description:

One ANNOTATION_TEXT may represent a multi-line ASCII description text (voluminous description). Its purpose is to be an application note which explains the function of an identifier for the calibration engineer.

Example:

```
/begin CHARACTERISTIC  annotation.example2 ...
....
  /begin ANNOTATION
    ANNOTATION_LABEL  "Calibration Note"
    /begin ANNOTATION_TEXT
      "The very nice ASAM MCD-2 MC Specification."
      "Text.\r\n"
      "In case of a quotation mark "
      "use \" or "" to mark it."
    /end ANNOTATION_TEXT
  /end ANNOTATION
....
/end CHARACTERISTIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.16 ARRAY_SIZE

Prototype:

```
ARRAY_SIZE          uint          Number
```

Parameters:

uint	Number	Number of measurement values included in respective measurement object (maximum value of 'Number': 32767).

Description:

This keyword marks a measurement object as an array of <Number> measurement values.

Note: The use of this keyword should be replaced by MATRIX_DIM.

Example:

```
/begin MEASUREMENT  N                 /* name */
                    "Engine speed"    /* long identifier */
                    UWORD             /* datatype */
                    R_SPEED_3         /* conversion */
                    1                 /* resolution */
                    0                 /* accuracy */
                    120.0             /* lower limit */
                    8400.0            /* upper limit */
  ARRAY_SIZE        8                 /* array of 8 values */
  BIT_MASK          0x0FFF
  BYTE_ORDER        MSB_FIRST
  /begin FUNCTION_LIST
                    ID_ADJUSTM
                    FL_ADJUSTM
  /end FUNCTION_LIST
  /begin IF_DATA
                    ISO
                    SND
                    0x10
                    0x00
                    0x05
                    0x08
                    RCV
                    4
                    long
  /end IF_DATA
/end MEASUREMENT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.17 ASAP2_VERSION

Prototype:

```
ASAP2_VERSION       uint          VersionNo
                    uint          UpgradeNo
```

Parameters:

uint	VersionNo	Version number of ASAM MCD-2 MC standard
uint	UpgradeNo	Upgrade number of ASAM MCD-2 MC standard

Description:

The ASAM MCD-2 MC version can be expressed by two numerals:
- VersionNo
- UpgradeNo

where ‘VersionNo’ represents the main version number and ‘UpgradeNo’ the upgrade number (fractional part of version number). The upgrade number will be incremented if additional functionality is implemented to ASAM MCD-2 MC standard which has no effect on existing applications (compatible modifications). The version number will be incremented in case if incompatible modifications.

The ASAP2_VERSION keyword is mandatory and expected before the keyword PROJECT.

Example:

```
ASAP2_VERSION             1
                          70    /* Version 1.7.0 */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.19 AXIS_DESCR

Prototype:

```
/begin AXIS_DESCR   enum       Attribute
                    ident      InputQuantity
                    ident      Conversion
                    uint       MaxAxisPoints
                    float      LowerLimit
                    float      UpperLimit
                    [-> ANNOTATION]*
                    [-> AXIS_PTS_REF]
                    [-> BYTE_ORDER]
                    [-> CURVE_AXIS_REF]
                    [-> DEPOSIT]
                    [-> EXTENDED_LIMITS]
                    [-> FIX_AXIS_PAR]
                    [-> FIX_AXIS_PAR_DIST]
                    [-> FIX_AXIS_PAR_LIST]
                    [-> FORMAT]
                    [-> MAX_GRAD]
                    [-> MONOTONY]
                    [-> PHYS_UNIT]
                    [-> READ_ONLY]
                    [-> STEP_SIZE]
/end AXIS_DESCR
```

Parameters:

enum Attribute — Description of the axis points:

CURVE_AXIS	Curve axis. This axis type uses a separate CURVE CHARACTERISTIC to rescale the axis. The referenced CURVE is used to lookup an axis index, and the index value is used by the controller to determine the operating point in the CURVE or MAP. See Appendix: B for more details.
COM_AXIS	Group axis points or description of the axis points for deposit. For this variant of the axis points the axis point values are separated from the table values of the curve or map in the emulation memory and must be described by a special AXIS_PTS data record. The reference to this record occurs with the keyword 'AXIS_PTS_REF'.
FIX_AXIS	This is a curve or a map with virtual axis points that are not deposited at EPROM. The axis points can be calculated from parameters defined with keywords FIX_AXIS_PAR, FIX_AXIS_PAR_DIST and FIX_AXIS_PAR_LIST. The axis points can‘t be modified.
RES_AXIS	Rescale axis. For this variant of the axis points the axis point values are separated

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

from the table values of the curve or map in the emulation memory and must be described by a special AXIS_PTS data record. The reference to this record occurs with the keyword 'AXIS_PTS_REF'.

STD_AXIS Standard axis

ident InputQuantity
Reference to the data record for description of the input quantity (see MEASUREMENT). If there is no input quantity assigned, parameter 'InputQuantity' should be set to "NO_INPUT_QUANTITY" (measurement and calibration systems must be capable to treat this case).

Note: If the referenced input quantity is an element of an array or a component of a structure, the identifier to be used for the reference shall be the name built according to rules described at INSTANCE.

ident Conversion
Reference to the relevant record of the description of the conversion method (see COMPU_METHOD). If there is no conversion method, as in the case of CURVE_AXIS, the parameter 'Conversion' should be set to "NO_COMPU_METHOD" (measurement and calibration systems must be able to handle this case).

uint MaxAxisPoints
Maximum number of axis points

Note: The measurement and calibration system can change the dimensions of a characteristic (increase or decrease the number of axis points). The number of axis points may not be increased at random as the address range reserved for each characteristic in the ECU program by the measurement and calibration system cannot be changed.

float LowerLimit
Plausible range of axis point values, lower limit

float UpperLimit
Plausible range of axis point values, upper limit

Note: Depending on the type of conversion, the limit values are interpreted as physical or internal values.
For conversions of type COMPU_VTAB and COMPU_VTAB_RANGE, the limit values are interpreted as internal values. For all other conversion types, the limit values are interpreted as physical values.

Optional parameters:

-> ANNOTATION
Set of notes (represented as multi-line ASCII description texts) which are related. Can serve e.g. as application note. When a COM_AXIS is referenced it is sufficient to place the ANNOTATION with its AXIS_PTS in order to avoid redundant information.

-> AXIS_PTS_REF
Reference to the AXIS_PTS record for description of the axis points distribution.

-> BYTE_ORDER
Where the standard value does not apply this parameter can be used to specify the byte order (Intel format, Motorola format) of the axis point value.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

-> CURVE_AXIS_REF	When the axis type is CURVE_AXIS, this keyword must be used to specify the CURVE CHARACTERISTIC that is used to normalize or scale this axis.
-> DEPOSIT	The axis points of a characteristic can be deposited in two different ways: a) The individual axis point values are deposited as absolute values. b) The individual axis point are stored as differences. Each axis point value is determined from the adjacent axis point (predecessor). Where the standard value does not apply this parameter can be used to specify the axis point deposit.
-> EXTENDED_LIMITS	This keyword can be used to specify an extended range of values. In the measurement and calibration system, for example, when leaving the standard range of values (lower limit...upper limit) a warning could be generated (extended limits enabled only for "power user").
-> FIX_AXIS_PAR	For curves or maps, the axis points distribution is not stored in memory but it is computed on the basis of the offset (initial value) and a difference. For the record layouts used today, these parameters must be included in the description file. The specification occurs with keyword 'FIX_AXIS_PAR'.
-> FIX_AXIS_PAR_DIST	Similar to FIX_AXIS_PAR but with a different computing method
-> FIX_AXIS_PAR_LIST	The original values of the axis are directly contained in the file. The assigned COMPU_METHOD is applied to achieve the actual display values from the values with this keyword
-> FORMAT	With deviation from the display format specified with keyword COMPU_METHOD referenced by parameter <Conversion> a special display format can be specified to be used to display the axis points.
-> MAX_GRAD	This keyword can be used to specify a maximum permissible gradient for the adjustable object with respect to this axis (MaxGrad= max (abs(($W_{i,k}$-$W_{i-1,k}$)/(X_i-X_{i-1})))).
-> MONOTONY	This keyword can be used to specify a monotonous behavior for the adjustable object with respect to this axis.
-> PHYS_UNIT	With this keyword a physical unit can be specified for the axis points if no conversion rule is referenced (NO_COMPU_METHOD). Note: If a conversion rule is referenced the additional usage of PHYS_UNIT overrules the unit specified at the referenced conversion rule.
-> READ_ONLY	This keyword can be used to indicate that the axis points of adjustable object cannot be changed (but can be read only). Note: This optional keyword used at CHARACTERISTIC record indicates the adjustable object to be read only at all (table values and axis points).

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

-> STEP_SIZE This keyword can be used to define a delta value which is added to or subtracted from the current value when using up/down keys while calibrating.

Description:

Axis description within an adjustable object

Note: With the 'input quantity' parameter a reference is made to a measurement object (MEASUREMENT). The MEASUREMENT keyword also specifies the 'conversion', 'lower limit' and 'upper limit' parameters.
It is expected that both conversions are equivalent, i.e. they must lead to the same result. The ‘upper limit’ and ‘lower limit’ parameters may be different.

Note: The keywords FIX_AXIS_PAR, FIX_AXIS_PAR_DIST, DEPOSIT and FIX_AXIS_PAR_LIST are mutually exclusive, i.e. at most one of these keywords is allowed to be used at the same AXIS_DESCR record.

Note: For the axis types COM_AXIS, RES_AXIS and CURVE_AXIS some attributes are defined twice: both at the AXIS_DESCR record and at the referenced AXIS_PTS resp. CHARACTERISTIC record. These redundant attributes are *InputQuantity*, *Conversion*, *MaxAxisPoints*, *LowerLimit*, *UpperLimit* and some optional parameters (e.g.: PHYS_UNIT). To support existing use cases where one common axis is used with different input quantities (e.g. multiple cylinders) it is recommended to ignore the redundant attributes defined at AXIS_PTS and use the values of the AXIS_DESCR record instead. Exeptions are *MaxAxisPoints* and MONOTONY which are used from AXIS_PTS.

Note: For the axis type COM_AXIS, it is necessary to have the same dimension for the axis object and for the referencing curve / map object. Theoretically it is possible to define the dimension both at the axis and at the curve resp. map – e.g. the corresponding record layouts can both contain the NO_AXIS_PTS_X component. It is recommended not to do so, but if the dimension is defined twice and if it is not equal, then application systems shall always use the dimension of the AXIS_PTS object.

Example:

```
/begin  AXIS_DESCR  STD_AXIS     /* Standard axis points */
                    N            /* Reference to input quantity */
                    CONV_N       /* Conversion */
                    14           /* Max.number of axis points*/
                    0.0          /* Lower limit */
                    5800.0       /* Upper limit*/
        MAX_GRAD    20.0         /* Axis: maximum gradient*/
/end AXIS_DESCR
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.20 AXIS_PTS

Prototype:

```
/begin AXIS_PTS     ident         Name
                    string     LongIdentifier
                    ulong         Address
                    ident         InputQuantity
                    ident         Deposit
                    float         MaxDiff
                    ident         Conversion
                    uint          MaxAxisPoints
                    float         LowerLimit
                    float         UpperLimit
                    [-> ANNOTATION]*
                    [-> BYTE_ORDER]
                    [-> CALIBRATION_ACCESS]
                    [-> DEPOSIT]
                    [-> DISPLAY_IDENTIFIER]
                    [-> ECU_ADDRESS_EXTENSION]
                    [-> EXTENDED_LIMITS]
                    [-> FORMAT]
                    [-> FUNCTION_LIST]
                    [-> GUARD_RAILS]
                    [-> IF_DATA]*
                    [-> MAX_REFRESH]
                    [-> MODEL_LINK]
                    [-> MONOTONY]
                    [-> PHYS_UNIT]
                    [-> READ_ONLY]
                    [-> REF_MEMORY_SEGMENT]
                    [-> STEP_SIZE]
                    [-> SYMBOL_LINK]
/end AXIS_PTS
```

Parameters:

ident	Name	unique identifier in the ECU program Note: The name has to be unique within all measure and adjustable objects of the MODULE, i.e. there must not be another AXIS_PTS, MEASUREMENT, CHARACTERISTIC, BLOB or INSTANCE object with an equal identifier in the same MODULE. Furthermore it is not allowed to have a structure component with equal name in the MODULE. (Rules for building the full identifiers of structure components: see at INSTANCE).
string	LongIdentifier	comment, description
ulong	Address	address of the adjustable object in the emulation memory
ident	InputQuantity	reference to the data record for description of the input quantity (see MEASUREMENT). If there is no input quantity assigned, parameter 'InputQuantity' should be set

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

to "NO_INPUT_QUANTITY" (measurement and calibration systems must be capable to treat this case).

Note: If the referenced input quantity is an element of an array or a component of a structure, the identifier to be used for the reference shall be the name built according to rules described at INSTANCE.

ident	Deposit	reference to the relevant data record for description of the record layout (see RECORD_LAYOUT)
float	MaxDiff	Maximum difference of physical value recommended for parameter change within one calibration step. If the difference in change exceeds this value, control algorithms might fail. The value 0 describes that there is no limit.
ident	Conversion	Reference to the relevant record of the description of the conversion method (see COMPU_METHOD). If there is no conversion method, as in the case of CURVE_AXIS, the parameter 'Conversion' should be set to "NO_COMPU_METHOD" (measurement and calibration systems must be able to handle this case).
uint	MaxAxisPoints	maximum number of axis points
float	LowerLimit	plausible range of axis point values, lower limit
float	UpperLimit	plausible range of axis point values, upper limit

Note: Depending on the type of conversion, the limit values are interpreted as physical or internal values.
For conversions of type COMPU_VTAB and COMPU_VTAB_RANGE, the limit values are interpreted as internal values. For all other conversion types, the limit values are interpreted as physical values.

Note: AXIS_PTS and AXIS_DESCR define the same parameters. Which parameters are dominate is described at AXIS_DESCR.

Optional parameters:

-> ANNOTATION	Set of notes (represented as multi-line ASCII description texts) which are related. Can serve e.g. as application note.
-> BYTE_ORDER	Where the standard value does not apply, this parameter can be used to specify the byte order (Intel format, Motorola format) of the axis points.
-> CALIBRATION_ACCESS	This keyword specifies the access of the axis points for calibration. It replaces the READ_ONLY attribute.
-> DEPOSIT	The axis points of a characteristic can be deposited in one of the following two modes: a) the individual axis points are deposited as absolute values; b) the individual axis points are deposited as differences. Each axis point is determined from the adjacent point (predecessor). Where the standard value does not

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

apply, this parameter can be used to specify the deposit of axis points.

-> DISPLAY_IDENTIFIER Can be used as a display name (alternative to the ‘name’ attribute).

-> ECU_ADDRESS_EXTENSION This keyword is an additional address information. For instance it can be used, to distinguish different address spaces of an ECU (multi-micro controller devices).

-> EXTENDED_LIMITS This keyword can be used to specify an extended range of values. In the measurement and calibration system, for example, when leaving the standard range of values (lower limit...upper limit) a warning could be generated (extended limits enabled only for "power user").

-> FORMAT With deviation from the display format specified in the COMPU_METHOD record referenced by parameter <Conversion> a special display format can be specified to be used to display the axis points.

-> FUNCTION_LIST This keyword can be used to specify a list of 'functions' to which the axis points distribution is allocated (function orientation).

-> GUARD_RAILS This keyword is used to indicate that an AXIS_PTS uses guard rails. The Measurement and Calibration System does not allow the user to edit the outermost axis breakpoints (see GUARD_RAILS).

-> IF_DATA Data record to describe interface specific data of the axis points. The parameters associated with this keyword have to be described in the ASAM MCD-2 MC metalanguage.

-> MAX_REFRESH Maximum refresh rate of this (adaptive) axis in the control unit. The existence of the keyword implies that the values of the axis are changed by the control unit (adaptive characteristics).

-> MODEL_LINK name used in the software model.

-> MONOTONY This keyword can be used to specify a monotonous behavior for the adjustable object with respect to this axis.

-> PHYS_UNIT With this keyword a physical unit can be specified for the axis points if no conversion rule is referenced (NO_COMPU_METHOD).

Note: If a conversion rule is referenced the additional usage of PHYS_UNIT overrules the unit specified at the referenced conversion rule.

-> READ_ONLY This keyword can be used to indicate that the axis points of axis points distribution cannot be changed (but can be read only).

Note: This optional keyword used at CHARACTERISTIC record indicates the adjustable object to be read only at all (table values and axis pts).

-> REF_MEMORY_SEGMENT Reference to the memory segment.

-> STEP_SIZE This keyword can be used to define a delta value which is added to or subtracted from the current value when using up/down keys while calibrating.

-> SYMBOL_LINK Reference to symbol name within a linker map file.

Description:

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Specification of parameters for the handling of an axis points distribution.

Example:

```
/begin AXIS_PTS        STV_N       /* name */
                       "axis points distribution speed"
                                   /* long identifier */
                       0x9876      /* address */
                       N           /* input quantity */
                       DAMOS_SST   /* deposit */
                       100.0       /* maxdiff */
                       R_SPEED     /* conversion */
                       21          /* maximum number of axis points */
                       0.0         /* lower limit */
                       5800.0      /* upper limit */
  GUARD_RAILS                     /* uses guard rails*/
  REF_MEMORY_SEGMENT   Data3
  /begin FUNCTION_LIST
                       ID_ADJUSTM
                       FL_ADJUSTM
                       SPEED_LIM
  /end FUNCTION_LIST
  /begin IF_DATA       DIM
                       EXTERNAL
                       DIRECT
  /end IF_DATA
  CALIBRATION_ACCESS   CALIBRATION
/end AXIS_PTS
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.21 AXIS_PTS_REF

Prototype:

```
AXIS_PTS_REF        ident        AxisPoints
```

Parameters:

ident AxisPoints — Name of the AXIS_PTS data record which describes the common axis points distribution (group axis points and record layout: see AXIS_PTS).

Note: If the referenced axis is an element of an array or a component of a structure, the identifier to be used for the reference shall be the name built according to rules described at INSTANCE.

Note: If (and only if) the axis description is utilized inside a structure definition and the common axis referenced by this keyword is a component of the same structure, then it is also allowed to refer to the axis by using the THIS keyword followed by a dot and the component name of the axis instead of using a concrete instance name.

Description:

If the addresses of the axis point values are separated from the table values in the emulation memory and must be described by a special AXIS_PTS data record, the data record is referenced by means of the keyword AXIS_PTS_REF.

Example:

```
/* Group characteristic curve with reference to axis points distribution GRP_N */
/begin CHARACTERISTIC    TORQUE                  /* name */
                         "Torque limitation"     /* long identifier */
                         CURVE                   /* type*/
                         0x1432                  /* address */
                         DAMOS_GKL               /* deposit */
                         0.2                     /* maxdiff */
                         R_TORQUE                /* conversion */
                         0.0                     /* lower limit */
                         43.0                    /* upper limit */
   /begin IF_DATA        DIM
                         EXTERNAL
                         INDIRECT
   /end IF_DATA
   /begin AXIS_DESCR     /* description of X-axis points */
                         COM_AXIS                /* common axis points */
                         N                       /* input quantity */
                         CONV_N                  /* conversion */
                         14                      /* max. no. of axis p.*/
                         0.0                     /* lower limit */
                         5800.0                  /* upper limit */
      AXIS_PTS_REF       GRP_N
   /end AXIS_DESCR
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
/end CHARACTERISTIC

/* Axis points distribution data record */
/begin AXIS_PTS          GRP_N                      /* name */
                         "Group axis points speed" /* long identifier */
                         0x1032                     /* address */
                         N                          /* input quantity */
                         DAMOS_GST                  /* deposit */
                         50.0                       /* maxdiff */
                         CONV_N                     /* conversion */
                         11                         /* max. no. of axis points */
                         0.0                        /* lower limit */
                         5800.0                     /* upper limit */
   /begin IF_DATA        DIM
                         EXTERNAL
                         INDIRECT
   /end IF_DATA
/end AXIS_PTS
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.22 AXIS_PTS_X / _Y / _Z / _4 / _5

Prototype:

```
AXIS_PTS_X / _Y / _Z / _4 / _5
                    uint       Position
                    datatype   Datatype
                    indexorder IndexIncr
                    addrtype   Addressing
```

Parameters:

uint	Position	Position of the axis point values in the deposit structure (description of sequence of elements in the data record).
datatype	Datatype	Data type of the axis point values
indexorder	IndexIncr	Decreasing or increasing index with increasing addresses
addrtype	Addressing	Addressing of the table values (see enum addrtype).

Description:

Description of the X, Y, Z, Z4 or Z5 axis points in the memory (see keyword RECORD_LAYOUT).

Note: If the Alternate option is used with FNC_VALUES, the position parameter determines the order of values and axis points.

Example:

```
AXIS_PTS_X              3
                        ULONG
                        INDEX_INCR
                        DIRECT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.23 AXIS_RESCALE_X

Prototype:

```
AXIS_RESCALE_X
                    Uint        Position
                    Datatype    Datatype
                    Uint        MaxNumberOfRescalePairs
                    Indexorder  IndexIncr
                    Addrtype    Adressing
```

Parameters:

uint	Position	position of the rescale axis point value pairs in the deposit structure (description of sequence of elements in the data record).
datatype	Datatype	Data type of the rescale axis point values
uint	MaxNumberOfRescalePairs	maximum number of rescaling axis point pairs (see NO_RESCALE_X)
indexorder	IndexIncr	Decreasing or increasing index with increasing addresses
addrtype	Adressing	Addressing of the table values (see enum addrtype).

Description:

Description of rescaling the axis values of an adjustable object. A rescale axis consists mainly of a number of rescaling axis points pairs ($axis_i$, $virtual_i$) which describe a rescale mapping between the axis points and a virtual axis that is used for the access of the table function values deposited in the control unit. Between two pairs the mapping is linear. Both, the axis points and the virtual axis points must be in ascending order. Consider, for example, the three rescale pairs (0x00, 0x00), (0x64, 0xC0) and (0xD8, 0xFF). Then all axis points between 0x00 and 0x64 are mapped linear to the virtual axis [0x00, 0xC0], and all axis points between 0x64 and 0xD8 are mapped linear to the virtual axis [0xC0, 0xFF]:

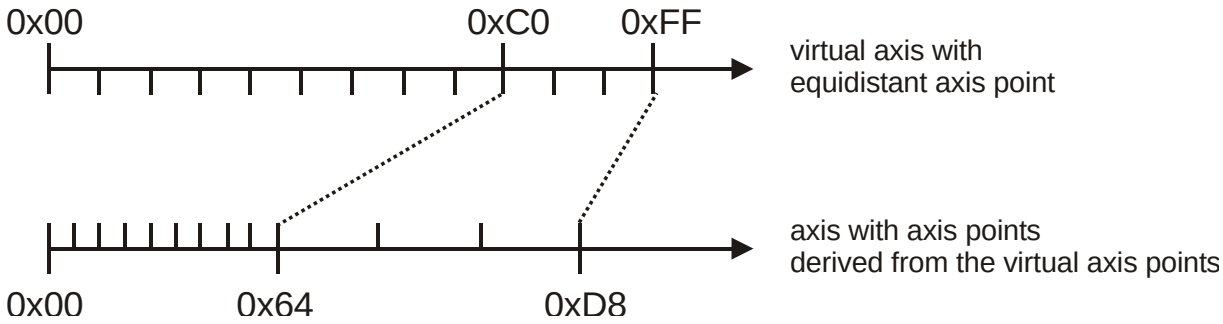


Figure 2 AXIS_RESCALE_X

Accordingly, to each axis point there is a virtual axis point. The virtual axis points are distributed equidistantly on the virtual axis including the axis limits, e.g. the virtual axis points can be derived from the size of the virtual axis and the number of axis points. According to the rescale mapping the axis point can be computed from the virtual axis points. The following algorithm can be applied, where D is the length of the (equidistant) intervals on virtual axis:

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

$$D = \frac{\text{last virtual axis point} - \text{first virtual axis point} + 1}{no_axis_pts - 1} \qquad k = 1$$

FOR i = 1 TO $(no_rescale_x - 1)$

FOR $k * d + virtual_1 < virtual_{i+1}$
/* repeat for the number of points in the interval on the virtual axis */

$k = k + 1$

$$X_k = axis_i + ((k-1)D - virtual_i)\frac{axis_{i+1} - axis_i}{virtual_{i+1} - virtual_i}$$

$X_1 = axis_1$
$X_{no_axis_pts} = axis_{no_rescale_x}$

It is recommended that D is a power of 2, i.e. if the size of the virtual axis is 256, the number of axis points should be $no_axis_pts = 2^n + 1$ = {3, 5, 9, 17, 33}.

The following example makes clear how the evaluation of the formula can be used to derive the actual axis points. We have no_of_rescale_pairs = 3 and $virtual_1$ = 0x00 = 0, $virtual_2$ = 0xC0 = 192, $virtual_3$ = 0xFF = 255, $axis_1$ = 0x00 = 0, $axis_2$= 0x64 = 100, $axis_3$ = 0xD8 = 216. Assume no_axis_pts = 9, and therefore D = 32. The first of the two executions of the inner loop (j-loop) is on $virtual_2 - virtual_1$/ D = 192/32 = 6 iterations. For each iteration $(axis_2 - axis_1)/(virtual_2 - virtual_1)$ = 100/192, and therefore

X_2 = 0 + 32 * 100/192 = 16,666,
X_3 = 0 + 64 * 100/192 = 33,333,
X_4 = 0 + 96 * 100/192 = 50,
X_5 = 0 + 128 * 100/192 =66,666,
X_6 = 0 + 160 * 100/192 = 83,333.

For the second execution there are $virtual_3 - virtual_2$ / D = 2 iterations with $(axis_3 - axis_2)/(virtual_3 - virtual_2)$ = 116/64. Consequently

X_7 = 100 + (192 – 192) * 116/64 = 100 and
X_8 = 100 + (224 – 192) * 116/64 = 158.

Also $X_1 = axis_1$ = 0 and $X_9 = axis_3$ = 216.

Example:

```
AXIS_RESCALE_X          3
                        UBYTE
                        5
                        INDEX_INCR
                        DIRECT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.24 BIT_MASK

Prototype:

```
BIT_MASK            uint64      Mask
```

Parameters:

uint64	Mask	mask to mask out single bits

Description:

The BIT_MASK keyword can be used to mask out single bits of the value to be processed. The least significant bit in BIT_MASK determines how far the masked value is shifted to the right.

Example:

BIT_MASK	0x00000FFF	
Value to be masked	BIT_MASK	Result
10110110	0x1 = 1 (bin)	0 (bin)
10110110	0x2 = 10 (bin)	1 (bin)
10110110	0x6 = 110 (bin)	11 (bin)
10110110	0xC = 1100 (bin)	01 (bin)
10111010	0xC = 1100 (bin)	10 (bin)
10111110	0xC = 1100 (bin)	11 (bin)
10111110	0xA = 1010 (bin)	101 (bin)

Note: The newly added comments about the least significant bit and the inserted samples are valid only while no keyword BIT_OPERATION is used. If the keyword BIT_OPERATION is used then its defined parameters dominate those parameters of the BIT_MASK keyword.

If it is required to use BIT_MASK without a shift operation, then use BIT_OPERATION with a right or left shift of zero, as shown in the following example.

Example:

```
BIT_MASK 0x40
/begin BIT_OPERATION
          LEFT_SHIFT    0
/end BIT_OPERATION
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.25 BIT_OPERATION

Prototype:

```
/begin BIT_OPERATION
                    [-> LEFT_SHIFT]
                    [-> RIGHT_SHIFT]
                    [-> SIGN_EXTEND]
/end BIT_OPERATION
```

Parameters:

none

Optional parameters

-> LEFT_SHIFT	Number of positions to left shift data, zeros will be shifted in from the right.
-> RIGHT_SHIFT	Number of positions to right shift data, zeros will be shifted in from the left.
-> SIGN_EXTEND	Gives a sign extension of sign bit for measurement data.

Description:

The BIT_OPERATION keyword can be used to perform operation on the masked out value.
First BIT_MASK will be applied on measurement data, then LEFT_SHIFT / RIGHT_SHIFT is performed and last the SIGN_EXTEND is carried out.

SIGN_EXTEND means that the sign bit (masked data's leftmost bit) will be copied to all bit positions to the left of the sign bit. This results in a new datatype with the same signed value as the masked data.

Example:

```
/begin BIT_OPERATION
           RIGHT_SHIFT   4                  /*4 positions*/
           SIGN_EXTEND
/end BIT_OPERATION
```

Table 7 BIT_OPERATION

Explanation	Data	Comment
Data after mask operation	0000000000100000	
Data after shift operation	0000000000000010	shifted right 4 positions
Data after sign extend	1111111111111110	

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.26 BLOB

Prototype:

```
/begin BLOB         ident      Name
                    string     LongIdentifier
                    ulong      Address
                    ulong      Size
                    [-> ADDRESS_TYPE]
                    [-> ANNOTATION]*
                    [-> CALIBRATION_ACCESS]
                    [-> DISPLAY_IDENTIFIER]
                    [-> ECU_ADDRESS_EXTENSION
                    [-> IF_DATA]*
                    [-> MAX_REFRESH]
                    [-> MODEL_LINK]
                    [-> SYMBOL_LINK]
/end BLOB
```

Parameters:

ident	Name	Identifier in the ECU program. The name has to be unique within all measure and adjustable objects of the MODULE, i.e. there must not be another AXIS_PTS, MEASUREMENT, CHARACTERISTIC, BLOB or INSTANCE object with an equal identifier in the same MODULE. Furthermore it is not allowed to have a structure component with equal name in the MODULE. (Rules for building the full identifiers of structure components: see at INSTANCE).
string	LongIdentifier	Comment, description for the blob object.
ulong	Address	Address of blob in the emulation memory.
ulong	Size	Blob size, number of bytes

Optional parameters

-> ADDRESS_TYPE	Addressing of the BLOB (see in Table 5 enum addrtype) Note: If the keyword ADDRESS_TYPE is missing DIRECT addressing is used.
-> ANNOTATION	Set of related notes.
-> CALIBRATION_ACCESS	Specifies the access rights for the blob object.
-> DISPLAY_IDENTIFIER	Specifies a display name (alternative to the 'name' attribute).
-> ECU_ADDRESS_EXTENSION	Additional address information to be used e.g. to distinguish different address spaces of an ECU (multi-microcontroller devices).
-> IF_DATA	Description of interface specific data of the blob. The parameters associated with this keyword have to be described in the ASAM MCD-2 MC meta language.
-> MAX_REFRESH	Maximum refresh rate of this BLOB in the control unit. The existence of this keyword implies that

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

	the blob content is changed by the control unit (adaptive characteristics).
-> MODEL_LINK	name used in the software model.
-> SYMBOL_LINK	Reference to a symbol name within a linker map file.

Description:

Specification of a binary blob object which has no further semantic interpretation in the MCD system. It is just an array of bytes without a conversion method or special record layout.

Example:

```
/begin BLOB
  FIM_Blob                              // blob name
  "binary blob for FIM matrix"          // description
  0x1004080                             // start address
  1024                                  // number of bytes in blob
/end BLOB
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.27 BYTE_ORDER

Prototype:

```
BYTE_ORDER          byteorder  ByteOrder
```

Parameters:

byteorder	ByteOrder	Byte order of the relevant quantity in the ECU program Possible ByteOrder: MSB_FIRST MSB_LAST MSB_FIRST_MSW_LAST MSB_LAST_MSW_FIRST

Note: Use of LITTLE_ENDIAN and BIG_ENDIAN defined with keyword BYTE_ORDER in version 1.0 leads to mistakes because it is in contradiction to general use of terms „little endian“ and „big endian“. Since version 1.2 the keywords LITTLE_ENDIAN and BIG_ENDIAN are permissible but should not longer be used. They should be replaced by MSB_LAST and MSB_FIRST which are equivalent:

MSB_LAST corresponds to the Intel format (equivalent former keyword is BIG_ENDIAN).

MSB_FIRST corresponds to the Motorola format (equivalent former keyword is LITTLE_ENDIAN).

Description:

Where the standard value does not apply this parameter can be used to specify the byte order (Intel format, Motorola format).

Example:

```
BYTE_ORDER              MSB_LAST
```

Table 8 Byte order (2 Byte values)- memory data deposition

Byte Order	Increasing address -->	
	n	n+1
MSB_FIRST	$Byte_1$	$Byte_0$
MSB_LAST	$Byte_0$	$Byte_1$

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Table 9 Byte order (4 Byte values) - memory data deposition

Byte Order	Increasing address -->			
	n	n+1	n + 2	n + 3
MSB_FIRST_MSW_LAST	$Byte_1$	$Byte_0$	$Byte_3$	$Byte_2$
MSB_LAST_MSW_FIRST	$Byte_2$	$Byte_3$	$Byte_0$	$Byte_1$
MSB_FIRST	$Byte_3$	$Byte_2$	$Byte_1$	$Byte_0$
MSB_LAST	$Byte_0$	$Byte_1$	$Byte_2$	$Byte_3$

Table 10 Byte order (8 Byte values) - memory data deposition

Byte Order	Increasing address -->							
	n	n+1	n + 2	n + 3	n+4	n+5	n + 6	n + 7
MSB_FIRST_MSW_LAST	$Byte_1$	$Byte_0$	$Byte_3$	$Byte_2$	$Byte_5$	$Byte_4$	$Byte_7$	$Byte_6$
MSB_LAST_MSW_FIRST	$Byte_6$	$Byte_7$	$Byte_4$	$Byte_5$	$Byte_2$	$Byte_3$	$Byte_0$	$Byte_1$
MSB_FIRST	$Byte_7$	$Byte_6$	$Byte_5$	$Byte_4$	$Byte_3$	$Byte_2$	$Byte_1$	$Byte_0$
MSB_LAST	$Byte_0$	$Byte_1$	$Byte_2$	$Byte_3$	$Byte_4$	$Byte_5$	$Byte_6$	$Byte_7$

Note:

$Byte_n$ == Most Significant Byte

$Byte_0$ == Least Significant Byte

MSB_FIRST == Motorola / LITTLE_ENDIAN

MSB_LAST == Intel / BIG_ENDIAN

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.28 CALIBRATION_ACCESS

Prototype

```
CALIBRATION_ACCESS  enum        Type
```

Parameters

enum Type

Possible Types:

CALIBRATION Characteristic or axis points with calibration allowed.

NO_CALIBRATION This keyword can be used to indicate that the axis points cannot be changed (but can be read only).

Note: This optional keyword used at CHARACTERISTIC record indicates the adjustable object to be read only at all (table values and axis pts).

NOT_IN_MCD_SYSTEM Internal adjustable objects which are not readable or writeable by the MCD-System. If there are references between AXIS_PTS and CHARACTERISTICS with one of them gets this value of CALIBRATION_ACCESS, the other one must get this value too.

OFFLINE_CALIBRATION Variables which can be flashed but not emulated ore calibrated, e.g. values representing safety relevant property while driving.

Description

This keyword specifies the access of an adjustable object (CHARACTERISTIC, AXIS_PTS or BLOB) for calibration. It substitutes the READ_ONLY attribute since ASAM-MCD-2 MC V1.4

Example

```
CALIBRATION_ACCESS      CALIBRATION
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.29 CALIBRATION_HANDLE

Prototype:

```
/begin CALIBRATION_HANDLE
                    (long      Handle)*
                    [-> CALIBRATION_HANDLE_TEXT]
/end CALIBRATION_HANDLE
```

Parameters:

long	Handle	Handle for the calibration method

Optional parameters

-> CALIBRATION_HANDLE_TEXT	Additional text for a calibration handle.

Description:

Definition of the calibration method specific. The interpretation of this data depends on the calibration method used.

Example:

```
/begin CALIBRATION_HANDLE
                        0x10000 /* start address of pointer table */
                        0x200   /* length of pointer table */
                        0x4     /* size of one pointer table entry */
                        0x30000 /* start address of flash section */
                        0x20000 /* length of flash section */
                        CALIBRATION_HANDLE_TEXT "Nmot"
/end CALIBRATION_HANDLE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.30 CALIBRATION_HANDLE_TEXT

Prototype:

```
CALIBRATION_HANDLE_TEXT
                    string     text
```

Parameters:

string	text	text string

Description:

Additional text for a calibration handle.

Example:

```
CALIBRATION_HANDLE_TEXT   "Torque"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.31 CALIBRATION_METHOD

Prototype:

```
/begin CALIBRATION_METHOD
                    string     Method
                    ulong      Version
                    [-> CALIBRATION_HANDLE]
/end CALIBRATION_METHOD
```

Parameters:

string	Method	the string identifies the calibration method to be used. A convention regarding the meaning of the calibration methods. The following strings are already in use: 'InCircuit', 'SERAM', 'DSERAP', 'BSERAP'
ulong	Version	Version number of the method used

Optional Parameters:

-> CALIBRATION_HANDLE	Contains the (method specific) arguments for the calibration method. The arguments themselves and their meaning are dependent of the calibration method.

Description:

This keyword is used to indicate the different methods of access that are implemented in the ECU and that can be used regardless of the actual interface of the ECU.

Example:

```
/begin CALIBRATION_METHOD
    „InCircuit"
    2
    /begin CALIBRATION_HANDLE
         0x10000          /* start address of pointer table */
         0x200            /* length of pointer table */
         0x4              /* size of one pointer table entry */
         0x10000          /* start address of flash section */
         0x10000          /* length of flash section */
    /end CALIBRATION_HANDLE
/end CALIBRATION_METHOD
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.32 CHARACTERISTIC

Prototype:

```
/begin CHARACTERISTIC ident    Name
                      string   LongIdentifier
                      enum     Type
                      ulong    Address
                      ident    Deposit
                      float    MaxDiff
                      ident    Conversion
                      float    LowerLimit
                      float    UpperLimit
                      [-> ANNOTATION]*
                      [-> AXIS_DESCR]*
                      [-> BIT_MASK]
                      [-> BYTE_ORDER]
                      [-> CALIBRATION_ACCESS]
                      [-> COMPARISON_QUANTITY]
                      [-> DEPENDENT_CHARACTERISTIC]
                      [-> DISCRETE]
                      [-> DISPLAY_IDENTIFIER]
                      [-> ECU_ADDRESS_EXTENSION]
                      [-> ENCODING]
                      [-> EXTENDED_LIMITS]
                      [-> FORMAT]
                      [-> FUNCTION_LIST]
                      [-> GUARD_RAILS]
                      [-> IF_DATA]*
                      [-> MAP_LIST]
                      [-> MATRIX_DIM]
                      [-> MAX_REFRESH]
                      [-> MODEL_LINK]
                      [-> NUMBER]
                      [-> PHYS_UNIT]
                      [-> READ_ONLY]
                      [-> REF_MEMORY_SEGMENT]
                      [-> STEP_SIZE]
                      [-> SYMBOL_LINK]
                      [-> VIRTUAL_CHARACTERISTIC]
/end CHARACTERISTIC
```

Parameters:

ident Name — unique identifier in the ECU program

Note: The name has to be unique within all measure and adjustable objects of the MODULE, i.e. there must not be another AXIS_PTS, MEASUREMENT, CHARACTERISTIC, BLOB or INSTANCE object with an equal identifier in the same MODULE. Furthermore it is not allowed to have a structure component with equal name in the MODULE. (Rules for building the full

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

identifiers of structure components: see at INSTANCE).

string	LongIdentifier	comment, description
enum	Type	possible Types: ASCII (string) CURVE (1-dimensional array with axes) MAP (2-dimensional array with axes) CUBOID (3-dimensional array with axes) CUBE_4 (4-dimensional array with axes) CUBE_5 (5-dimensional array with axes) VAL_BLK (array without axes) VALUE (scalar)
ulong	Address	address of the adjustable object in the emulation memory
ident	Deposit	reference to the corresponding data record for description of the record layout (see RECORD_LAYOUT)
float	Maxdiff	Maximum difference of physical value recommended for parameter change within one calibration step. If the difference in change exceeds this value, control algorithms might fail. The value 0 describes that there is no limit.
ident	Conversion	Reference to the relevant record of the description of the conversion method (see COMPU_METHOD). If there is no conversion method, as in the case of CURVE_AXIS, the parameter 'Conversion' should be set to "NO_COMPU_METHOD (measurement and calibration systems must be able to handle this case).
float	LowerLimit	plausible range of table values, lower limit
float	UpperLimit	plausible range of table values, upper limit

Note: Depending on the type of conversion, the limit values are interpreted as physical or internal values.
For conversions of type COMPU_VTAB and COMPU_VTAB_RANGE, the limit values are interpreted as internal values. For all other conversion types, the limit values are interpreted as physical values.

Optional parameters

-> ANNOTATION	Set of notes (represented as multi-line ASCII description texts) which are related. Can serve e.g. as application note.
-> AXIS_DESCR	This keyword is used to specify the parameters for the axis description (with characteristic curves and maps). The first parameter block describes the X-axis, the second parameter block the Y-axis, the third parameter block the Z-axis (CUBOID), the fourth parameter block the Z4-axis (CUBE_4), the fifth parameter block the Z5-axis (CUBE_5). Exception: For MAP_LIST only one AXIS_DESCR is accepted that describes the Z-axis. (The X- and Y-axes are described at the MAPs referenced by MAP_LIST)

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

-> BIT_MASK This parameter can be used to specify a bit mask for the handling of single bits.

-> BYTE_ORDER Where the standard value does not apply this parameter can be used to specify the byte order (Intel format, Motorola format) if the standard value is not to be used.

-> CALIBRATION_ACCESS This keyword specifies the access of the characteristic for calibration. Use it instead of the READ_ONLY Attribute.

-> COMPARISON_QUANTITY This keyword references a valid MEASUREMENT in the ASAM MCD-2 MC file.

Semantic Interpretation (for a CURVE, a CHARACTERISTIC with only one AXIS_DESC) : The conventional workpoint for a -CURVE has only one input quantity (assigned to AXIS_DESCR) and moves on the CURVE. The 'free-moving' workpoint in an xy diagram of a CURVE is described by two quantities (the conventional input quantity with the AXIS_DESC, the x-axis, and an additional comparison quantity described as an optional attribute directly with the CURVE, the y-axis).The 'free-moving' workpoint does not move on the CURVE, but on the xy-diagram in which the CURVE is located. The crossing of the free-moving workpoint and the CURVE would describe an EVENT. Such display is required by calibration engineers of automatic transmission control (EVENT=gear shift). When this keyword with a CURVE is present, the workpoint display of the MCD system shall apply the INPUT_QUANTITY and the COMPARISON_QUANTITY in the xy-diagram.

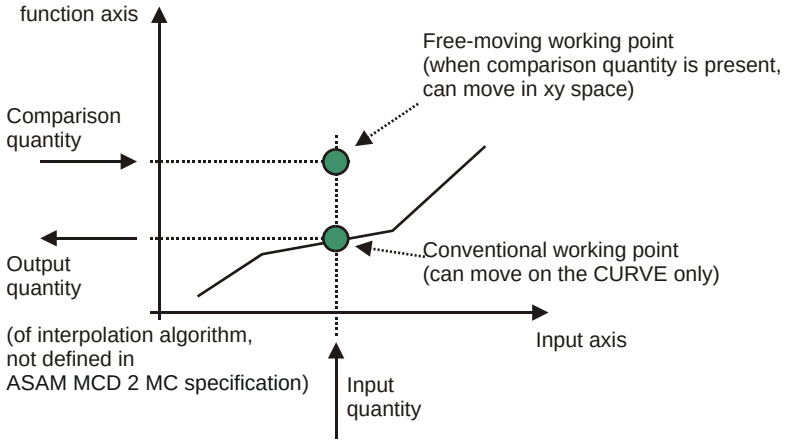


Figure 3 Comparison quantity

-> DEPENDENT_CHARACTERISTIC Describes the formula and references to characteristics, upon which this characteristic depends on.

Note: The dependence graph described by the dependence relation must be acyclic. This must be ensured by the producer of the ASAM MCD-2 MC file. This keyword is only valid for characteristics of type VALUE

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

-> DISCRETE	This keyword indicates that the characteristic values are discrete values which should not be interpolated – e.g. in graphic display windows or further calculations. This flag can be used e.g. for integer objects describing states. If the keyword is not specified the values are interpreted as continuous values which can be interpolated.
-> DISPLAY_IDENTIFIER	Can be used as a display name (alternative to the 'name' attribute).
-> ECU_ADDRESS_EXTENSION	This keyword is an additional address information. For instance it can be used, to distinguish different address spaces of an ECU (multi-microcontroller devices).
-> ENCODING	This keyword specifies the type of encoding used when the CHARACTERISTIC is of type ASCII. If no ENCODING is specified, the content is interpreted as string of 8 Bit characters.
-> EXTENDED_LIMITS	This keyword can be used to specify an extended range of values. In the measurement and calibration system, for example, when leaving the standard range of values (lower limit...upper limit) a warning could be generated (extended limits enabled only for "power user").
-> FORMAT	With deviation from the display format specified in the COMPU_METHOD record referenced by parameter <Conversion> a special display format can be specified to be used to display the table values.
-> FUNCTION_LIST	This keyword can be used to specify a list of 'functions' to which the relevant adjustable object is allocated (function orientation).
-> GUARD_RAILS	This keyword is used to indicate that an adjustable CURVE or MAP uses guard rails. The Measurement and Calibration System does not allow the user to edit the outermost values of the adjustable object (see GUARD_RAILS).
-> IF_DATA	Data record to describe interface specific data of the characteristic. The parameters associated with this keyword have to be described in the ASAM MCD-2 MC metalanguage.
-> MAP_LIST	For the adjustable object type CUBOID which are `sliced', this keyword specifies the MAPs which comprise the cuboids. Note: The MAPs referenced by MAP_LIST may have different number of axis points. Note: MAP_LIST is supported only for CUBOID, not for CUBE_4 or CUBE_5.
-> MATRIX_DIM	Shows the size and dimension of a multi-dimensional characteristic (e.g. VAL_BLK). If the MATRIX_DIM keyword is used, then the option NUMBER is not needed. However, if the keywords NUMBER and MATRIX_DIM are both used, the resulting value in NUMBER must be the same as the product of all dimensions of MATRIX_DIM. If the keyword is missing the array has only one dimension with the size given at NUMBER.
-> MAX_REFRESH	Maximum refresh rate of this (adaptive) characteristic in the control unit. The existence of the keyword implies that

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

the value of the characteristic is changed by the control unit (adaptive characteristics).

-> MODEL_LINK Name used in the software model.

-> NUMBER For adjustable objects of type ASCII, this keyword specifies the number of bytes reserved for the string.

-> PHYS_UNIT With this keyword a physical unit can be specified for the characteristic object if no conversion rule is referenced (NO_COMPU_METHOD).

Note: If a conversion rule is referenced the additional usage of PHYS_UNIT overrules the unit specified at the referenced conversion rule.

-> READ_ONLY This keyword can be used to indicate that the adjustable object cannot be changed (but can be read only). This keyword indicates the adjustable object to be read only at all (table values and axis points). The optional keyword used at AXIS_DESCR record indicates the related axis points to be read only

-> REF_MEMORY_SEGMENT Reference to the memory segment.

-> STEP_SIZE This keyword can be used to define a delta value which is added to or subtracted from the current value when using up/down keys while calibrating.

-> SYMBOL_LINK Reference to symbol name within a linker map file.

-> VIRTUAL_CHARACTERISTIC Marks a characteristic as being virtual, i.e. not existing in the memory of the control unit. The address can therefore be ignored for virtual characteristic. Initial value of the virtual characteristic depends on the values of other characteristic.

Note: The corresponding graph (in analogy to the dependence graph) must also be acyclic and each sink of the graph must be a non virtual characteristic. This must be ensured by the producer of the ASAM MCD-2 MC file. This keyword is only valid for characteristics of type VALUE.

Description:

Specification of the parameters for the processing of an adjustable object.

Example:

```
/begin CHARACTERISTIC    PUMKF     /* name */
                         "Pump characteristic map"
                                   /* long identifier */
                         MAP       /* type */
                         0x7140    /* address */
                         DAMOS_KF  /* deposit */
                         100.0     /* maxdiff */
                         R_VOLTAGE/* conversion */
                         0.0       /* lower limit */
                         5000.0    /* upper limit */
        MAX_REFRESH      3 15      /* 15 msec */
    /begin DEPENDENT_CHARACTERISTIC
                         "sin(X1)"
                         ALPHA
    /end DEPENDENT_CHARACTERISTIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
        /begin VIRTUAL_CHARACTERISTIC
                           „sqrt(X1)“
                           B_AREA
        /end VIRTUAL_CHARACTERISTIC
        REF_MEMORY_SEGMENT Data1
        /begin FUNCTION_LIST
                           NL_ADJUSTMENT
                           FL_ADJUSTMENT
                           SPEED_LIM
        /end FUNCTION_LIST
        /begin IF_DATA
                           DIM
                           EXTERNAL
                           INDIRECT
        /end IF_DATA
        /begin AXIS_DESCR  /* description of X-axis points */
                           STD_AXIS /* standard axis points */
                           N        /* reference to input quantity */
                           CON_N    /* conversion */
                           13       /* maximum number of axis points*/
                           0.0      /* lower limit */
                           5800.0   /* upper limit */
            MAX_GRAD       20.0     /* X-axis: maximum gradient */
        /end AXIS_DESCR
        /begin AXIS_DESCR  /* description of Y-axis points */
                           STD_AXIS /* standard axis points */
                           AMOUNT   /* reference to input quantity */
                           CON_ME   /* conversion */
                           17       /* maximum number of axis points*/
                           0.0      /* lower limit */
                           43.0     /* upper limit */
        /end AXIS_DESCR
    /end CHARACTERISTIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.33 COEFFS

Prototype:

```
COEFFS              float      a  b  c  d  e  f
```

Parameters:

float a, b, c, d, e, f: coefficients for the specified formula: f(x) = (axx + bx + c) / (dxx + ex + f)

Description:

Specification of coefficients for the formula f(x) = (axx + bx + c) / (dxx + ex + f). This term describes the conversion from physical values to control unit internal values:

INT = f(PHYS);

Note: For these coefficients restrictions have to be defined because this general equation cannot always be inverted.

Example:

```
COEFFS   0 4 8 0 0 5
/* Control unit internal values of revolutions (INT) is calculated  from */
/* physical values (PHYS: unit of PHYS is [rpm]) as follows: */
/* INT = (4/5) * PHYS/[rpm] + (8/5)              */
/* inverted:    PHYS/[rpm] = 1.25 * INT - 2.0    */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.34 COEFFS_LINEAR

Prototype:

```
COEFFS_LINEAR       float       a   b
```

Parameters:

float a, b: coefficients for the specified formula: f(x) = ax + b

Description:

Specification of coefficients for the formula f(x) = ax + b. This term describes the conversion from control unit internal values to physical values:

PHYS = f(INT);

Example:

```
COEFFS_LINEAR   1.25   -2.0
/* The physical value (PHYS) with unit is calculated from the      */
/* control unit's internal value of revolutions (INT) as follows:  */
/*       PHYS = 1.25 * INT - 2.0          */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.36 COMPARISON_QUANTITY

Prototype:

```
COMPARISON_QUANTITY ident       Name
```

Parameters:

ident	Name	Unique identifier in the program (Reference to a valid MEASUREMENT)

Description:

This keyword references a valid MEASUREMENT in the ASAM MCD-2 MC file.

Semantic Interpretation (for a CURVE, a CHARACTERISTIC with only one AXIS_DESC): The conventional work point for a -CURVE has only one input quantity (assigned to AXIS_DESCR) and moves on the CURVE. The 'free-moving' work point in an xy diagram of a CURVE is described by two quantities (the conventional input quantity with the AXIS_DESC, the x-axis, and an additional comparison quantity described as an optional attribute directly with the CURVE, the y-axis).The 'free-moving' work point does not move on the CURVE, but on the xy-diagram in which the CURVE is located. The crossing of the free-moving work point and the CURVE would describe an EVENT. Such display is required by calibration engineers of automatic transmission control (EVENT=gear shift). When this keyword with a CURVE is present, the work point display of the MCD system shall apply the INPUT_QUANTITY and the COMPARISON_QUANTITY in the xy-diagram.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.37 COMPU_METHOD

Prototype:

```
/begin COMPU_METHOD ident      Name
                    string     LongIdentifier
                    enum       ConversionType
                    string     Format
                    string     Unit
                    [-> COEFFS]
                    [-> COEFFS_LINEAR]
                    [-> COMPU_TAB_REF]
                    [-> FORMULA]
                    [-> REF_UNIT]
                    [-> STATUS_STRING_REF]
/end COMPU_METHOD
```

Parameters:

ident Name — unique identifier in the program for the conversion method

Note: The name of the conversion method has to be unique within all conversion methods of the ASAM MCD-2 MC MODULE, i.e. there must not be another COMPU_METHOD object with the same identifier in the MODULE.

string LongIdentifier — comment, description

enum ConversionType — possible Types:

- IDENTICAL — no conversion of the internal source value
  The following equation is applied:
  PHYS = INT
- FORM — conversion based on the formula specified by the FORMULA keyword.
- LINEAR — linear function of the following type:
  f(x)=ax + b
  for which:
  PHYS=f(INT)
  The coefficients a and b have to be specified by the COEFFS_LINEAR keyword.
- RAT_FUNC — fractional rational function of the following type
  f(x)=(axx + bx + c)/(dxx + ex + f)
  for which:
  INT = f(PHYS)
  Coefficients a, b, c, d, e, f have to be specified by the COEFFS keyword.

  Note: For linear functions, use the ConversionType LINEAR, for ident functions the ConversionType IDENT. For non linear functions it must be possible to invert the formula within the limits of the AXIS_PTS, CHARACTERISTIC or

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

MEASUREMENT where it is used. Otherwise use the ConversionType FORM.

TAB_INTP table with interpolation
TAB_NOINTP table without interpolation
TAB_VERB verbal conversion table

string Format — display format in %[<length>].<layout>
<length> is an optional unsigned integer value, which indicates the overall length;
<layout> is a mandatory unsigned integer value, which indicates the decimal places;
The format string must always contain at least "%", "." and <layout>.

string Unit — physical unit

Optional parameters:

-> COEFFS — This keyword is used to specify coefficients a, b, c, d, e, f for the fractional rational function of the following type:
f(x)=(axx + bx + c) / (dxx + ex + f)

-> COEFFS_LINEAR — This keyword is used to specify the coefficients a and b of the linear function of the following type:
f(x)=ax + b

-> COMPU_TAB_REF — This keyword is used to specify a conversion table (reference to COMPU_TAB, COMPU_VTAB or COMPU_VTAB_RANGE data record).

-> FORMULA — Formula to be used for the conversion

-> REF_UNIT — This keyword is used to reference a measurement unit (e.g. an object of type UNIT). The string parameter Unit is a redundant information because the record referenced by REF_UNIT contain it too. Just for the purpose of compatibility with previous versions of ASAM MCD-2 MC the parameter REF_UNIT is optional.

-> STATUS_STRING_REF — This keyword is used to split up the value range of ECU internal values into a numerical and a verbal part. The verbal part can be used to visualize status information (e.g. “Sensor not connected”).

Description:

Specification of a conversion method

Examples:

```
/begin COMPU_METHOD      TMPCON1        /* name */
                         "conversion method for engine temperature"
                         TAB_NOINTP     /* convers_type */
                         "%4.2"         /* display format */
                         "°C"           /* physical unit */
   COMPU_TAB_REF         MOTEMP1
/end COMPU_METHOD

/begin COMPU_METHOD      CM_IDENTITY    /* name */
                         "conversion method identity (no formula)"
                         IDENTICAL/* convers_type */
                         "%4.0"    /* display format */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
                        ""        /* physical unit */
/end COMPU_METHOD

/begin COMPU_METHOD     CM_LINFUNC    /* name */
                        "conversion method for linear function"
                        LINEAR    /* convers_type */
                        "%4.0"    /* display format */
                        "rpm"     /* physical unit */
                        COEFFS_LINEAR  2.0 5.0
/end COMPU_METHOD

/begin COMPU_METHOD     TMPCON2       /* name */
                        "conversion method for air temperature"
                        FORM          /* convers_type */
                        "%4.2"        /* display format */
                        "°C"          /* physical unit */
  /begin FORMULA        "3*X1/100 + 22.7"
  /end FORMULA
/end COMPU_METHOD

/begin COMPU_METHOD     CM_DiagStatus /* name */
                        ""            /*convers_type */
                        TAB_VERB      /*convers_type */
                        "%0.0"        /* display format */
                        ""            /* physical unit */
  COMPU_TAB_REF         CT_DiagStatus
/end COMPU_METHOD

/begin COMPU_METHOD     CM_RPM        /* name */
                        "conversion method for engine rpm"
                        TAB_INTP      /*convers_type */
                        "%7.1"        /* display format */
                        "rpm "        /* physical unit */
  COMPU_TAB_REF         CT_RPM
/end COMPU_METHOD

/begin COMPU_METHOD     CM_NM         /* name */
                        " conversion method for air temperature "
                        TAB_INTP      /* convers_type */
                        "%7.1"        /* display format */
                        "nm "         /* physical unit */
  COMPU_TAB_REF         CT_NM
/end COMPU_METHOD

/begin COMPU_METHOD     FIXED_UW_03
                        "Conversion method for FIXED_UW_03"
                        RAT_FUNC      /* convers_type */
                        "%8.3"        /* display format */
                        "NO_PHYSICAL_QTY"
                        COEFFS 0 8 0 0 0 1
/end COMPU_METHOD

/begin COMPU_METHOD     BYTE
                        "Conversion method for BYTE"
                        RAT_FUNC      /* convers_type */
                        "%3.0"        /* display format */
                        "NO_PHYSICAL_QTY"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
                        COEFFS 0 1 0 0 0 1
/end COMPU_METHOD

/begin COMPU_METHOD     SHORTINT
                        "Conversion method for SHORTINT"
                        RAT_FUNC
                        "%4.0"
                        "NO_PHYSICAL_QTY"
                        COEFFS 0 1 0 0 0 1
/end COMPU_METHOD

/begin COMPU_METHOD     WORD
                        "Conversion method for WORD"
                        RAT_FUNC
                        "%5.0"
                        "NO_PHYSICAL_QTY"
                        COEFFS 0 1 0 0 0 1
/end COMPU_METHOD

/begin COMPU_METHOD     INTEGER
                        "Conversion method for INTEGER"
                        RAT_FUNC
                        "%6.0"
                        "NO_PHYSICAL_QTY"
                        COEFFS 0 1 0 0 0 1
/end COMPU_METHOD

/begin COMPU_METHOD     LONGWORD
                        "Conversion method for LONGWORD"
                        RAT_FUNC
                        "%10.0"
                        "NO_PHYSICAL_QTY"
                        COEFFS 0 1 0 0 0 1
/end COMPU_METHOD

/begin COMPU_METHOD     LONGINT
                        "Conversion method for LONGINT"
                        RAT_FUNC
                        "%11.0"
                        "NO_PHYSICAL_QTY"
                        COEFFS 0 1 0 0 0 1
/end COMPU_METHOD
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.38 COMPU_TAB

Prototype:

```
/begin COMPU_TAB    ident      Name
                    string     LongIdentifier
                    enum       ConversionType
                    uint       NumberValuePairs
                    (float     InVal
                      float    OutVal)*
                    [-> DEFAULT_VALUE]
                    [-> DEFAULT_VALUE_NUMERIC]
/end COMPU_TAB
```

Parameters:

ident	Name	unique identifier in the program for the conversion table Note: The name of the conversion table has to be unique within all conversion tables of the ASAM MCD-2 MC MODULE, i.e. there must not be another COMPU_TAB, COMPU_VTAB or COMPU_VTAB_RANGE object with the the same identifier in the MODULE.
string	LongIdentifier	comment, description
enum	ConversionType	possible Types: TAB_INTP table with interpolation TAB_NOINTP table without interpolation Note: This parameter is redundant information because the record defined with COMPU_METHOD also contains it.
uint	NumberValuePairs	number of successive value pairs for this conversion table
float	InVal	axis point
float	OutVal	axis value

Optional parameters

-> DEFAULT_VALUE	string used as OutVal for display when the ECU value is out of any declared range. This string is not selectable for calibration (when writing to the ECU). This parameter cannot be used in combination with DEFAULT_VALUE_NUMERIC. For COMPU_TAB it is recommended to use DEFAULT_VALUE_NUMERIC rather than DEFAULT_VALUE.
-> DEFAULT_VALUE_NUMERIC	Float value used as OutVal for display when the ECU value is out of any declared range. This value is not selectable for calibration (when writing to the ECU). The value is handled like a physical value (transferred via ASAM MCD-3 [4]). This parameter must not be used in combination with DEFAULT_VALUE.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Description:

Conversion table for conversions that cannot be represented as a function.

Example:

```
/begin COMPU_TAB          TT                  /* name */
                          "conversion table for oil temperatures"
                          TAB_NOINTP          /* convers_type */
                          7                   /* number_value_pairs */
                          1  4.3  2  4.7  3  5.8  4  14.2  5  16.8
                          6  17.2  7  19.4   /* value pairs */
                          DEFAULT_VALUE_NUMERIC 99.0
/end COMPU_TAB
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.39 COMPU_TAB_REF

Prototype:

```
COMPU_TAB_REF       ident       ConversionTable
```

Parameters:

ident	ConversionTable	reference to the data record which contains the conversion table (see COMPU_TAB).

Description:

Reference to the data record which contains the conversion table (see keyword COMPU_TAB).

Note: COMPU_TAB_REF may only refer to objects of type COMPU_TAB, COMPU_VTAB or COMPU_VTAB_RANGE.

Example:

```
COMPU_TAB_REF                TEMP_TAB    /*TEMP_TAB: conversion table*/
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.40 COMPU_VTAB

Prototype:

```
/begin COMPU_VTAB   ident      Name
                    string     LongIdentifier
                    enum       ConversionType
                    uint       NumberValuePairs
                    ( float    InVal
                      string   OutVal)*
                    [-> DEFAULT_VALUE]
/end COMPU_VTAB
```

Parameters:

ident	Name	unique identifier in the program for the verbal conversion table Note: The name of the conversion table has to be unique within all conversion tables of the ASAM MCD-2 MC MODULE, i.e. there must not be another COMPU_TAB, COMPU_VTAB or COMPU_VTAB_RANGE object with the same identifier in the MODULE.
string	LongIdentifier	comment, description
enum	ConversionType	at present only the following types are possible: TAB_VERB verbal conversion table Note: This parameter is a redundant information because the record defined with COMPU_METHOD also contains it.
uint	NumberValuePairs	number of successive value pairs for this conversion table
float	InVal	internal value Note: Datatype “float” is used for the input value of COMPU_TAB and COMPU_VTAB. Since the accepted use case of COMPU_VTAB input values are integers, all float values are rounded to the nearest integer by following the formula (0.5 <= x_raw < 1.5 lead to x = 1). Only float input values used with COMPU_VTAB_RANGE remain float values without truncation If you want to use float as internal values (without truncation), you should use COMPU_VTAB_RANGE.
string	OutVal	description (meaning) of the corresponding byte value

Optional parameters

-> DEFAULT_VALUE:	string used as OutVal for display when the ECU value is out of any declared range. This string shall not be selectable for calibration (when writing to the ECU).

Description:

Conversion table for the visualization of bit patterns

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Example:

```
/begin COMPU_VTAB       TT              /* name */
                        "engine status conversion"
                        TAB_VERB        /* convers_type */
                        4               /* number_value_pairs */
                        0 "engine off"  /* value pairs */
                        1 "idling"
                        2 "partial load"
                        3 "full load"
/end COMPU_VTAB
/begin COMPU_VTAB       CT_DiagStatus
                        ""
                        TAB_VERB        /* convers_type */
                        3               /* number_value_pairs */
                        0 "C_Fail"
                        1 "C_Pass"
                        2 "C_Indeterminate"
/end COMPU_VTAB
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.41 COMPU_VTAB_RANGE

Prototype:

```
/begin COMPU_VTAB_RANGE
                    ident      Name
                    string     LongIdentifier
                    uint       NumberValueTriples
                    ( float    InValMin
                      float    InValMax
                      string   OutVal)*
                    [-> DEFAULT_VALUE]
/end COMPU_VTAB_RANGE
```

Parameters:

ident	Name	unique identifier in the program for the verbal range based conversion table Note: The name of the conversion table has to be unique within all conversion tables of the ASAM MCD-2 MC MODULE, i.e. there must not be another COMPU_TAB, COMPU_VTAB or COMPU_VTAB_RANGE object with the same identifier in the MODULE.
string	LongIdentifier	comment, description
uint	NumberValueTriples	number of successive value triples for this verbal range based conversion table
float	InValMin	lower limit as float value, needs to be integer ECU value when assigned to “non-float” definitions.
float	InValMax	upper limit as float value, needs to be integer ECU value when assigned to “non-float” definitions.
string	OutVal	display string for the value range

Optional parameters

-> DEFAULT_VALUE	string used as OutVal for display when the ECU value is out of any declared range. This string shall not be selectable for calibration (when writing to the ECU).

Description:

Conversion table for the assignment of display strings to a value range. In particular this is useful for ASAM MCD-2 MC definitions with the data type ‘floating point’ (referred as FLOAT definitions).
For FLOAT definitions, the declared string is displayed for InValMin <= ECU value < InValMax, with InValMin, InValMax as floating point values.
For non-FLOAT definions, the declared string is displayed for InValMin <= ECU value <= InValMax, with InValMin, InVal as integer values.

Note: InValMin and InValMax can have the same value to express an assignment of one ECU value to a string (as in COMPU_VTAB); this is not realistic for floating point (and therefore not supported).

Note: Overlapping ranges may not be declared. The ASAM MCD-2 MC file is invalid in case of overlapping ranges within COMPU_VTAB_RANGE. But still, the upper

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

limit of one range may be the same FLOAT value than the lower limit of the following range in case of a FLOAT definition (see display rules).

Note: When a COMPU_METHOD with COMPU_VTAB_RANGE is used for calibration (writing of values to ECU), the InValMin is used when the assigned STRING (OutVal) is selected in the user interface.

Note: If the optional DEFAULT_VALUE is declared, this string is displayed when the ECU value is out of any declared range. This string shall not be selectable for calibration.

Example:

```
/begin COMPU_VTAB_RANGE  TT        /* name */
                         "engine status conversion"
                         5
                         0 0    "ONE"
                         1 2    "first_section"
                         3 3    "THIRD"
                         4 5    "second_section"
                         6 500  "usual_case"
                         DEFAULT_VALUE "Value_out_of_Range"
/end COMPU_VTAB_RANGE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.42 CONSISTENT_EXCHANGE

Prototype:

```
CONSISTENT_EXCHANGE
```

Description:

This keyword is used to indicate that a structure shall always be handled completely, i.e. it is not allowed to exchange only parts of the structure (e.g. in CDF files). The MC Tool is responsible for ensuring consistency among the structure components.

Example:

```
/begin TYPEDEF_STRUCTURE
  MyStructure             // structure type name
  ""                      // description
  40                      // total size of structure
  CONSISTENT_EXCHANGE
  /begin STRUCTURE_COMPONENT
        TypeName Component1
        0
  /end STRUCTURE_COMPONENT
  /begin STRUCTURE_COMPONENT
        TypeName Component2
        20
  /end STRUCTURE_COMPONENT
/end TYPEDEF_STRUCTURE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.43 CONVERSION

Prototype:

```
CONVERSION          ident       ConversionMethod
```

Parameters:

ident	ConversionMethod	Name of the COMPU_METHOD to be used for converting ECU internal values into physical values

Description:

This keyword can be used to overwrite the default conversion method specified at type definition for a specific instance.

Example:

```
/begin INSTANCE
     MyInstance ""
     MeasureTemplate
     0x2000
     /begin OVERWRITE
         THIS 0
         CONVERSION NewConversion     // overruling the conversion
     /end OVERWRITE
/end INSTANCE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.44 CPU_TYPE

Prototype:

```
CPU_TYPE            string     CPU
```

Parameters:

string	CPU	CPU identifier

Description:

CPU identification

Example:

```
CPU_TYPE                "INTEL 4711"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.45 CURVE_AXIS_REF

Prototype:

```
CURVE_AXIS_REF      ident      CurveAxis
```

Parameters:

ident CurveAxis — Name of the CURVE CHARACTERISTIC that is used to normalize or scale the axis that references the curve.

Note: If the referenced curve axis is an element of an array or a component of a structure, the identifier to be used for the reference shall be the name built according to rules described at INSTANCE.

Note: If (and only if) the axis description is utilized inside a structure definition and the curve referenced by this keyword is a component of the same structure, then it is also allowed to refer to the curve by using the THIS keyword followed by a dot and the component name of the curve instead of using a concrete instance name.

Description:

This keyword is used in conjunction with AXIS_DESCR definitions that use the CURVE_AXIS attribute. It is used to specify the adjustable CURVE CHARACTERISTIC that is used to normalize or scale the axis. See Appendix: B for more details.
Note: The same parameters for MaxAxisPoints apply as those for AXIS_DESCR.

Example:

```
/begin CHARACTERISTIC    FUEL_ADJ            /* name */
                         "Air fuel table"    /* long identifier */
                         MAP                 /* type */
                         0x7140              /* address */
                         DEP_12E             /* deposit */
                         1.0                 /* maxdiff */
                         R_MULT              /* conversion */
                         0.0                 /* lower limit */
                         2.0                 /* upper limit */
  /begin AXIS_DESCR      /* description of X-axis points */
                         CURVE_AXIS  /* curve axis points */
                         SPEED       /* reference to input quantity*/
                         NO_COMPU_METHOD     /* conversion */
                         13          /*maximum number of axis points*/
                         0           /*lower limit */
                         12          /*upper limit */
    CURVE_AXIS_REF       SPD_NORM
  /end AXIS_DESCR
  /begin AXIS_DESCR      /* description of Y-axis points */
                         CURVE_AXIS  /* curve axis points */
                         LOAD        /* reference to input quantity*/
                         NO_COMPU_METHOD     /* conversion */
                         17          /*maximum number of axis points*/
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
                        0          /*lower limit */
                        16         /*upper limit */
      CURVE_AXIS_REF    MAF_NORM
    /end AXIS_DESCR
  /end CHARACTERISTIC

  /begin RECORD_LAYOUT    DEP_12E
        FNC_VALUES        1 FLOAT32_IEEE ROW_DIR DIRECT
  /end RECORD_LAYOUT

  /begin CHARACTERISTIC   SPD_NORM /* name */
                          "Speed normalizing function"
                                   /* long identifier */
                          CURVE    /* type */
                          0x8210   /* address */
                          SPD_DEP  /* deposit */
                          100      /* maxdif */
                          R_NORM   /* conversion */
                          0 6      /* lower limit, upper limit */
    /begin AXIS_DESCR     /* description of X-axis points */
                          STD_AXIS /* standard axis */
                          SPEED    /* reference to input quantity */
                          R_SPEED  /* conversion */
                          7        /* maximum number of axis points*/
                          0        /* lower limit */
                          10000    /* upper limit */
    /end AXIS_DESCR
  /end CHARACTERISTIC

  /begin RECORD_LAYOUT    SPD_DEP
        AXIS_PTS_X        1 FLOAT32_IEEE INDEX_INCR DIRECT
        FNC_VALUES        2 FLOAT32_IEEE ALTERNATE_WITH_X DIRECT
  /end RECORD_LAYOUT

  /begin CHARACTERISTIC   MAF_NORM /* name */
                          "Load normalizing function"
                                   /* long identifier */
                          CURVE    /* type */
                          0x8428   /* address */
                          LOAD_DEP /* deposit */
                          100      /* maxdif */
                          R_NORM   /* conversion */
                          0 16     /* lower limit, upper limit */
    /begin AXIS_DESCR     /* description of X-axis points */
                          STD_AXIS /* standard axis */
                          LOAD     /* reference to input quantity */
                          R_LOAD   /* conversion */
                          17       /* maximum number of axis points*/
                          0.0      /* lower limit */
                          100.0    /* upper limit */
    /end AXIS_DESCR
  /end CHARACTERISTIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.46 CUSTOMER

Prototype:

```
CUSTOMER            string     Customer
```

Parameters:

string	Customer	customer name

Description:

This keyword allows a customer name to be specified.

Example:

```
CUSTOMER                "LANZ - Landmaschinen"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.47 CUSTOMER_NO

Prototype:

```
CUSTOMER_NO          string      Number
```

Parameters:

string	Number	customer number

Description:

Customer number as string.

Example:

```
CUSTOMER_NO              "191188"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.48 DATA_SIZE

Prototype:

```
DATA_SIZE           uint        Size
```

Parameters:

uint	Size	data size in bits

Description:

Data size in bits

Example:

```
DATA_SIZE               16
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.49 DEF_CHARACTERISTIC

Prototype:

```
/begin DEF_CHARACTERISTIC
                    (ident     Identifier)*
/end DEF_CHARACTERISTIC
```

Parameters:

ident	Identifier	Identifier of those adjustable objects that are defined in respective function.

Description:

This keyword can be used to declare some adjustable objects to be defined in respective function (function orientation).

Note: DEF_CHARACTERISTIC may only refer to adjustable objects, i.e. objects of type CHARACTERISTIC, AXIS_PTS and BLOB and objects which are defined via keyword INSTANCE using a corresponding type definitions. This also includes instances and instance components of adjustable structures.

Note: Each object shall be defined only once in one function. Other functions use only references to it.

Example:

```
/begin DEF_CHARACTERISTIC    INJECTION_CURVE
                             DELAY_FACTOR
/end DEF_CHARACTERISTIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.50 DEFAULT_VALUE

Prototype:

```
DEFAULT_VALUE       string     display_string
```

Parameters:

string	display_string

Description:

Optional String which can be applied with COMPU_TAB, COMPU_VTAB and COMPU_VTAB_RANGE, used as OutVal for display when the ECU value is out of any declared range. This string shall not be selectable for calibration (when writing to the ECU).
The use of this keyword excludes the use of the keyword DEFAULT_VALUE_NUMERIC.

Example:

```
DEFAULT_VALUE            "overflow_state"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.51 DEFAULT_VALUE_NUMERIC

Prototype:

```
DEFAULT_VALUE_NUMERIC
                    float       display_value
```

Parameters:

float	display_value

Description:

Optional value which can be applied with COMPU_TAB, used as OutVal for display when the ECU value is out of any declared range. This value is not selectable for calibration (when writing to the ECU). The DEFAULT_VALUE_NUMERIC is handled like a physical value (transferred via ASAM MCD-3 [4]). The use of this keyword excludes the use of the keyword DEFAULT_VALUE.

Example:

```
DEFAULT_VALUE_NUMERIC    999.0
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.52 DEPENDENT_CHARACTERISTIC

Prototype:

```
/begin DEPENDENT_CHARACTERISTIC
                    string     Formula
                    (ident     Characteristic)*
/end DEPENDENT_CHARACTERISTIC
```

Parameters:

string	Formula	Formula to be used for the calculation of the physical value of the characteristic from the physical value of other characteristics. Note: The result is interpreted as internal value of the dependent characteristic.
ident	Characteristic	Identifier of those adjustable objects that are used for the calculation of this characteristic. Note: If the referenced object is an element of an array or a component of a structure, the identifier to be used for the reference shall be the name built according to rules described at INSTANCE.

Description:

This keyword allows dependent characteristics to be specified. For this, other characteristics can be combined into one characteristic whose consistent value is automatically derived by the measurement and calibration system. Upon adjusting one of the characteristics, this characteristic is then also automatically adjusted according to the chosen formula (see also VIRTUAL_CHARACTERISTIC). Consider for example a rectangular triangle with a hypotenuse of length 1,

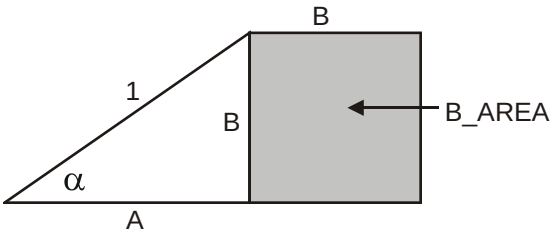


Figure 4 DEPENDENT_CHARACTERISTIC

where the length of the other sides are the characteristics A and B. When adjusting A the characteristic B has to be adjusted accordingly to B = sqrt (1- A*A). The relation between the involved characteristics is described on the physical level. Also other characteristic might depend on B, e.g. B_AREA = B * B. A dependent characteristic should not be adjustable by itself, but only through the adjustment of a characteristic it depends on.
The following example makes clear how the calibration process takes place. Assume for each of the characteristics A, B, and B_AREA a conversion formula of *internal = f(phys) = 100 * phys* and assume that the value A_{int} is 60 (decimal). Then $A_{phys} = A_{int} / 100 = 0.6$. According to the formula B = sqrt (1- A*A), $B_{phys} = 0.8$ and $B_{int} = B_{phys} * 100 = 80$ (decimal). According to B_AREA = B*B, we have $B_AREA_{phys} = 0.64$ and therefore $B_AREA_{int} = 64$ (decimal).

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

The references used in the dependency formula are named X1, X2, X3, … . The reference X1 references the first parameter of the attached parameter list, X2 the second, X3 the third, ….
If there is only one reference used it is allowed to use X instead of X1.

Example:

```
/begin DEPENDENT_CHARACTERISTIC
               „sqrt(1-X1*X1)“
               A
/end DEPENDENT_CHARACTERISTIC

/* Example for ParamB - ParamA */
/begin DEPENDENT_CHARACTERISTIC
               "X2-X1"
               ParamA    /* is referenced by X1 */
               ParamB    /* is referenced by X2 */
/end DEPENDENT_CHARACTERISTIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.53 DEPOSIT

Prototype:

```
DEPOSIT              enum        Mode
```

Parameters:

enum	Mode	Deposit of the axis points of a characteristic curve or map: ABSOLUTE absolute axis points DIFFERENCE difference axis points

Description:

This keyword describes how the axis point values of a curve or map are deposited in memory. The axis points of a characteristic can be deposited in two different ways in the memory:
a) The individual axis point values are deposited as absolute values.
b) The individual axis points are deposited as differences. Each axis point value is determined on the basis of the adjacent axis point (predecessor) and the corresponding difference. As reference point for the first axis point <maxvalue> is used:

1-byte-size: <maxvalue> = 2^8 (256)
2-byte-size: <maxvalue> = 2^{16} (65536)
4-byte-size: <maxvalue> = 2^{32}

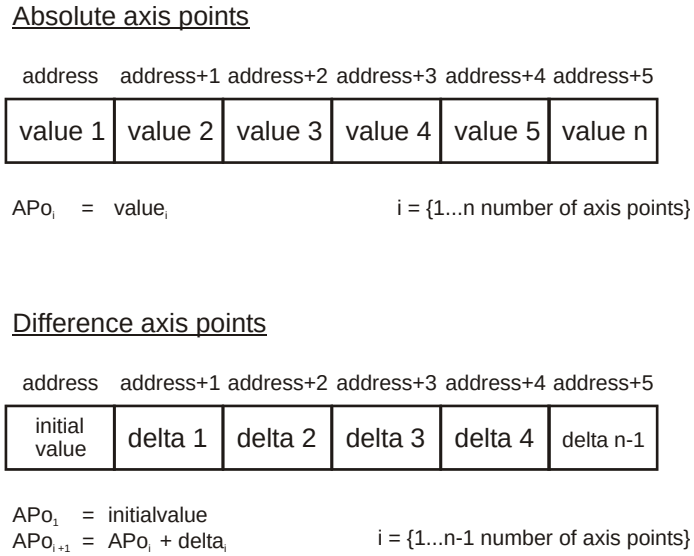


Figure 5 data deposition

Example:

```
DEPOSIT              DIFFERENCE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.54 DISCRETE

Prototype:

```
DISCRETE
```

Description:

This keyword indicates that a measurement object or a measured calibration object has discrete values. The measured values shall not be interpolated between two measure points - e.g. in graphic display windows (e.g. oscilloscope) or for further calculations. This flag can be used e.g. for integer objects describing states. If the keyword is not specified the values are interpreted as continuous ones which can be interpolated between the measure points. The keyword can be used at MEASUREMENT, CHARACTERISTIC and the corresponding TYPEDEF descriptions.

This keyword indicates that a measurement object or a measured calibration object has discrete values. The measured values shall not be interpolated between two measure points - e.g. in graphic display windows (osci) or for further calculations. This flag can be used e.g. for integer objects describing states. If the keyword is not specified the values are interpreted as continuous ones which can be interpolated between the measure points. The keyword can be used at MEASUREMENT and CHARACTERISTIC.

Example:

```
/begin MEASUREMENT
   counter
   "…"
   UBYTE
   NO_COMPU_METHOD
   2
   1
   0
   255
   DISCRETE
/end MEASUREMENT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.55 DISPLAY_IDENTIFIER

Prototype:

```
DISPLAY_IDENTIFIER ident      display_name
```

Parameters:

ident	display_name

Description:

This identifier can be used as a alternative name in the Measurement and Calibration System. DISPLAY_IDENTIFIERs can constitute an alternative set of names.

Note: The display_name does not have to be unique and is not referenced elsewhere. But is recommended that the display identifier shall be unique in order to avoid confusion in the user interface of the MCD system.

Example:

```
DISPLAY_IDENTIFIER      load_engine
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.56 DIST_OP_X / _Y / _Z / _4 / _5

Prototype:

```
DIST_OP_X /_Y /_Z / _4 / _5
                    uint        Position
                    datatype    Datatype
```

Parameters:

uint	Position	Position of the distance operand in the deposit structure.
datatype	Datatype	Data type of the distance operand.

Description:

Description of the distance operand in the deposit structure to compute the axis points for fixed characteristic curves and fixed characteristic maps (see also keyword FIX_AXIS_PAR_DIST). The axis points distribution for fixed characteristic curves or fixed characteristic maps is derived from the two 'offset' and 'distance' parameters as follows:

X_i = Offset + (i - 1)*Distance i = { 1...numberofaxispts }

or

Y_k = Offset + (k - 1)* Distance k = { 1...numberofaxispts }

or

Z_m = Offset + (m - 1)* Distance m = { 1...numberofaxispts }

or

$Z4_n$ = Offset + (n - 1)* Distance n = { 1...numberofaxispts }

or

$Z5_o$ = Offset + (o - 1)* Distance o = { 1...numberofaxispts }

Example:

```
DIST_OP_X                   21
                            UWORD
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.57 ECU

Prototype:

```
ECU                 string     ControlUnit
```

Parameters:

string	ControlUnit	control unit identifier

Description:

String for identification of the control unit.

Example:

```
ECU                          "Steering control"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.58 ECU_ADDRESS

Prototype:

```
ECU_ADDRESS          ulong      Address
```

Parameters:

ulong	Address	Address of the measurement in the memory of the control unit.

Description:

ECU_ADDRESS is used to describe the address of a measurement. It should replace the specific IF_DATA. It can be used in MEASUREMENT only.

Example:

```
ECU_ADDRESS              0x12FE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.59 ECU_ADDRESS_EXTENSION

Prototype:

```
ECU_ADDRESS_EXTENSION int      Extension
```

Parameters:

int	Extension	Address extension of the ECU address

Description:

This keyword is used to specify additional address information. For instance it can be used, to distinguish different address spaces of an ECU (multi-micro controller devices). ECU_ADDRESS_EXTENSION is an optional keyword of MEASUREMENT, AXIS_PTS and CHARACTERISTIC.

Note:

Some calibration interfaces, such as CCP and XCP need an address extension to access ECU data. To avoid the need for additional IF_DATA section at calibration and measurement objects, the keyword ECU_ADDRESS_EXTENSION has been introduced.

Example:

```
/begin MEASUREMENT       N                /* name */
                         "Engine speed"   /* long identifier */
                         UWORD            /* datatype */
                         R_SPEED_3        /* conversion */
                         1                /* resolution */
                         0                /* accuracy */
                         120.0            /* lower limit */
                         8400.0           /* upper limit */
  ECU_ADDRESS            0x12345
  ECU_ADDRESS_EXTENSION  1
/end MEASUREMENT

/begin CHARACTERISTIC    MAX_N            /* name */
                         "max speed"      /* long identifier */
                         VALUE            /* type */
                         0x7140           /* address */
                         DAMOS_Word       /* deposit */
                         100.0            /* maxdiff */
                         R_SPEED          /* conversion */
                         0.0              /* lower limit */
                         5000.0           /* upper limit */
  ECU_ADDRESS_EXTENSION  1
/end CHARACTERISTIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.60 ECU_CALIBRATION_OFFSET

Prototype:

```
ECU_CALIBRATION_OFFSET        long        Offset
```

Parameters:

long	Offset	Offset that has to be added to each address of a characteristic

Description:

ECU_CALIBRATION_OFFSET is used to describe a fixed address offset when accessing characteristics in the control unit due to

- near pointers in calibration objects. Some record layouts include near pointers inside a calibration objects from which the calibration system has to compute the absolute values by adding the ECU_CALIBRATION_OFFSET (CDAMOS)
- variant coding. Some ECU projects include multiple data sets for different engine or vehicle projects served by one common ECU. By using the ECU_CALIBRATION_OFFSET, a selection for project base address can be made

Example:

```
ECU_CALIBRATION_OFFSET        0x1000
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.61 ENCODING

Prototype:

```
ENCODING            enum        Encoding
```

Parameters:

enum	Encoding	Encoding of the string in the ECU program. Possible values: UTF8 UTF16 UTF32

Description:

This keyword specifies the encoding for an adjustable string object.

Note: If no ENCODING is specified, the content of the CHARACTERISTIC is interpreted as array of 8 Bit characters.

Example:

```
ENCODING                UTF8
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.62 EPK

Prototype:

```
EPK                 string     Identifier
```

Parameters:

string	Identifier	EPROM identifier

Description:

EPROM identifier string.

Example:

```
EPK                 "EPROM identifier test"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.63 ERROR_MASK

Prototype:

```
ERROR_MASK          uint64      Mask
```

Parameters:

uint64	Mask	mask to mask out selected bits

Description:

The ERROR_MASK keyword can be used to mask bits of a MEASUREMENT which indicate that the value is in error. The Measurement and Calibration System may apply this mask to display the error status of a measurement value. The error mask is usually a single bit; separate measurements should be defined in situations where each bit indicates a different type of error.

Example:

```
ERRROR_MASK             0x00000001
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.64 EXTENDED_LIMITS

Prototype:

```
EXTENDED_LIMITS     float      LowerLimit
                    float      UpperLimit
```

Parameters:

float	LowerLimit	extended range of table values, lower limit
float	UpperLimit	extended range of table values, upper limit

Note: Depending on the type of conversion, the limit values are interpreted as physical or internal values.
For conversions of type COMPU_VTAB and COMPU_VTAB_RANGE, the limit values are interpreted as internal values. For all other conversion types, the limit values are interpreted as physical values.

Description:

This keyword can be used to specify an extended range of values. In the MCD system, for example, when leaving the standard range of values (mandatory parameters 'lower limit' and 'upper limit' in the data record of the adjustable object) a warning could be generated (extended limits enabled only for "power user").

Example:

```
EXTENDED_LIMITS          0
                         6000.0
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.65 FIX_AXIS_PAR

Prototype:

```
FIX_AXIS_PAR        float       Offset
                    float       Shift
                    uint        Numberapo
```

Parameters:

float	Offset	'offset' parameter to calculate the axis points of fixed characteristic curves or maps (see description).
float	Shift	'shift' parameter to calculate the axis points of fixed characteristic curves or maps (see description).
uint	Numberapo	number of axis points

Description:

Typical of fixed characteristic curves and fixed characteristic maps is that, in contrast with standard and group characteristics, the axis points are not deposited individually in the program data of the ECU program but are derived from the two parameters 'offset' and 'shift'. In the current deposit methods both parameters are contained in the description file. In future deposit methods both methods could well be part of the deposit structure of the adjustable objects.
The axis points of fixed characteristic curves or maps are calculated as follows:

$$X_i = Offset + (i - 1)*2^{Shift} \quad i = \{ 1...numberapo \}$$

or

$$Y_k = Offset + (k - 1)*2^{Shift} \quad k = \{ 1...numberapo \}$$

or

$$Z_m = Offset + (m - 1)*2^{Shift} \quad m = \{ 1...numberapo \}$$

Note: This keyword is equivalent to FIX_AXIS_PAR_DIST but differs in parameter ‘Shift’ (see FIX_AXIS_PAR_DIST).

Example:

```
/* Define axis points 0, 16, 32, 48, 64, 80 */
FIX_AXIS_PAR            0
                        4
                        6
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.66 FIX_AXIS_PAR_DIST

Prototype:

```
FIX_AXIS_PAR_DIST   float      Offset
                    float      Distance
                    uint       Numberapo
```

Parameters:

float	Offset	'offset' parameter to calculate the axis points of fixed characteristic curves or maps (see description).
float	Distance	'distance' parameter to calculate the axis points of fixed characteristic curves or maps (see description).
uint	Numberapo	number of axis points

Description:

Typical of fixed characteristic curves and fixed characteristic maps is that, in contrast with standard and group characteristics, the axis points are not deposited individually in the program data of the ECU program but are derived from the two parameters 'offset' and 'distance'. In the current deposit methods both parameters are contained in the description file. In future deposit methods both methods could well be part of the deposit structure of the adjustable objects.
The axis points of fixed characteristic curves or maps are calculated as follows:

X_i = Offset + (i - 1)*Distance i = { 1...numberapo }

or

Y_k = Offset + (k - 1)*Distance k = { 1...numberapo }

or

Z_m = Offset + (m - 1)*Distance m = { 1...numberapo }

Note: This keyword is equivalent to FIX_AXIS_PAR but differs in parameter ‘Distance’.

Example:

```
FIX_AXIS_PAR_DIST       0
                        100
                        8
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.67 FIX_AXIS_PAR_LIST

Prototype:

```
/begin FIX_AXIS_PAR_LIST
                    (float     AxisPts_Value)*
/end FIX_AXIS_PAR_LIST
```

Parameters:

float	AxisPts_Value	List of "ECU-Original" Values as implied by the ECU algorithm. The number of values must match with the MaxAxisPoints attribute of the AXIS_DESCR referencing FIX_AXIS_PAR_LIST. The COMPU_METHOD assigned to the AXIS_DESCR shall be applied to achieve the actual display values. Note: The data type shall be integer in case of an assignment to a non-float definition).

Description:

Allows the description of any value combination of a virtual axis (FIX_AXIS, axis points not in the ECU memory). Other methods (FIX_AXIS_PAR, FIX_AXIS_PAR_DIST) implicitly assume an interpolation algorithm in the ECU. But axis descriptions are also used e.g. to span status tables.
The values are the input for the COMPU_METHOD assigned to the axis. Even a verbal table could be applied as COMPU_METHOD (i.e. for the axis description of status tables on which no interpolation is applied).

Example:

```
/begin FIX_AXIS_PAR_LIST        2 5 9
/end FIX_AXIS_PAR_LIST
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.68 FIX_NO_AXIS_PTS_X / _Y / _Z / _4 / _5

Prototype:

```
FIX_NO_AXIS_PTS_X /_Y /_Z / _4 / _5
                  uint      NumberOfAxisPoints
```

Parameters:

uint	NumberOfAxisPoints	Dimensioning of characteristic curves or characteristic maps with a fixed number of axis points

Description:

This keyword indicates that all characteristics of type CURVE, MAP, CUBOID, CUBE_4 or CUBE_5 allocate a fixed number of axis points. In a RECORD_LAYOUT data record, this keyword cannot be used simultaneously with the keyword NO_AXIS_PTS_X / _Y / _Z / _4 / _5

Example:

```
FIX_NO_AXIS_PTS_X       17
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.69 FNC_VALUES

Prototype:

```
FNC_VALUES          uint        Position
                    datatype    Datatype
                    enum        IndexMode
                    addrtype    Addresstype
```

Parameters:

uint	Position	position of table values (function values) in the deposit structure (description of sequence of elements in the data record).
datatype	DataType:	data type of the table values
enum	IndexMode:	for characteristic maps, this attribute is used to describe how the 2-dimensional table values are mapped onto the 1-dimensional address space:

ALTERNATE_CURVES

curves: curves which share a common axis are deposited in columns; each row of memory contains values for all the shared axis curves at a given axis breakpoint. Required in order to represent characteristics which correspond to arrays of structures in ECU program code. In the example code below, DT10, DT20, etc are treated as separate curves which may have different conversions or limits:-

maps: alternate curves not supported

cuboid: alternate curves not supported

cube_4: alternate curves not supported

cube_5: alternate curves not supported

```
typedef struct    {
            int DT10;
            int DT20;
            int DT30;
            int DT40;
      } VXP_TYPE;

const VXP_TYPE VX_PLUS_DELAY_TIMES[5] = {
            { 10, 3, 4, 8 },
            { 12, 2, 4, 6 },
            { 17, 9, 5, 8 },
            { 10, 1, 4, 8 },
            { 18, 3, 8, 8 },
      };
```

ALTERNATE_WITH_X

maps: deposited in columns, the columns of table values alternate with the respective X-coordinates.

curves: table values and X-coordinate values are deposited alternating.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

cuboid: alternate with X not supported
cube_4: alternate with X not supported
cube_5: alternate with X not supported

ALTERNATE_WITH_Y
maps: deposited in rows, the rows of table values alternate with the respective Y-coordinates (maps only).
cuboid: alternate with Y not supported
cube_4: alternate with Y not supported
cube_5: alternate with Y not supported

COLUMN_DIR storage of array values in column-major order

ROW_DIR storage of array values in row-major order

addrtype Addresstype addressing of the table values (see enum addrtype).

Description:

Description of the table values (function values) of an adjustable object. If the ALTERNATE option is used, the position parameter of values and axis-points indicates their order. The concepts 'columns' and 'rows' relate to the XY coordinate system (see also Appendix: A).
For characteristic cuboids each XY plane is mapped as above. The cuboid is stored as an array of maps with incremented or decremented Z coordinates. The CUBE_4 is stored as an array of CUBOID with incremented or decremented Z2 coordinates. The CUBE_5 is stored as an array of CUBE_4 with incremented or decremented Z3 coordinates. (Alternate not supported)

Example for ROW_DIR:

A 2 x 3 matrix M = $\begin{bmatrix} a11 & a12 & a13 \\ a21 & a22 & a23 \end{bmatrix}$

would be stored as follows: a_{11}, a_{12}, a_{13}, a_{21},a_{22}, a_{23}

More generally, a matrix a $i \; x \; j \; x \; k$
would be listed as a_{111}, … a_{11k}, a_{121}, … a_{1jk}, a_{211}, … a_{ijk}

Example for COLUMN_DIR:

A 2 x 3 matrix M = $\begin{bmatrix} a11 & a12 & a13 \\ a21 & a22 & a23 \end{bmatrix}$

would be stored as follows: a_{11}, a_{21}, a_{12}, a_{22}, a_{13}, a_{23}

More generally, a matrix a $i \; x \; j \; x \; k$
would be listed as a_{111}, … a_{i11}, a_{121}, … a_{ij1}, a_{112}, … a_{ijk}

Example:

```
FNC_VALUES              7
                        SWORD
                        COLUMN_DIR
                        DIRECT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.70 FORMAT

Prototype:

```
FORMAT                  string      FormatString
```

Parameters:

string	FormatString	display format in %[<length>].<layout> <length> is an optional unsigned integer value, which indicates the overall length; <layout> is a mandatory unsigned integer value, which indicates the decimal places

Description:

This keyword allows a special display format to be specified for MEASUREMENT, CHARACTERISTIC or AXIS_PTS objects resp. for the corresponding TYPEDEF definitions. If exists this display format is used instead of display format specified in respective COMPU_METHOD data record. The format string must always contain at least "%", "." and <layout>.

Example:

```
FORMAT                      "%4.2"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.71 FORMULA

Prototype:

```
/begin FORMULA      string     f(x)
                    [-> FORMULA_INV]
/end FORMULA
```

Parameters:

string f(x) function to calculate the physical value from the control unit internal value. The interpretation proceeds from left to right. Operator preferences, such as power before product/quotient before sum/difference, are taken into account. Brackets are allowed.

System constants which are defined in SYSTEM_CONSTANT can be used here, if their names comply to the restrictions of data type ident. For that the name of the system constant has to be put in “sysc()”. The value of a system constant used in FORMULA must contain either a numerical value or a string that contains a further FORMULA part (recursive replacement of the text in the formula). Endless loop for the system constant usage is not allowed.

The notation of operators and function names is conform to the ANSI C notation.
The following operation symbols can be used:

Basic operations:
+ for sums
- for differences
* for products
/ for quotients

Binary operators: interpretation from left to right
& bitwise AND
| bitwise OR
>> bitwise shift right
<< bitwise shift left
^ bitwise exclusive OR
~ bitwise NOT

Logical operators: interpretation from left to right
&& logical AND
|| logical OR
! logical NOT

Trigonometric functions:
sin(x), cos(x), tan(x)
asin(x), acos (x), atan (x)
sinh(x), cosh(x), tanh(x)

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Exponential function:
exp(x) for base e

Logarithmic functions:
log(x) for base e
log10(x) for base 10

Square root, absolute amount and power:
sqrt(x)
abs(x)
pow(x1, x2)

Internal Value
hex(x)

Physical Value
phy(x)

Optional parameters:

-> FORMULA_INV — function to calculate the control unit internal value from the physical value. This parameter is mandatory in formulas used for the conversion of adjustable objects. It is optional only for measurement objects.

Note: Certain functions in the measurement and calibration system can only be used for those measurement objects for which this parameter is specified (e.g. scalable DAC output, triggering).

Description:

This keyword allows any kind of formula to be specified for the conversion of measurement values, axis points or table values of an adjustable object from their ECU internal format into the physical format. The interpretation of the formula must be supported by a formula interpreter in the operating system.
The references used in the formula are to be named X1, X2, X3, … . The reference X1 references the first input, X2 the second, X3 the third, ….
If there is only one reference used it is allowed to use X instead of X1. One formula can either contain X or X1.

Example 1:

```
/begin FORMULA          "sqrt( 3 - 4*sin(X1) )"
/end FORMULA
```

Example 2:

```
/* Example to explain reference to SYSTEM_CONSTANT */
SYSTEM_CONSTANT "PI" "3.1415"
SYSTEM_CONSTANT "PI_half" "sysc(PI) / 2"
 […]
/begin FORMULA      "sysc(PI_half) * X"
/end FORMULA
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Note: Do not use FORMULA to describe identical, linear, or rational functions.

Note: Some of the FORMULA operators are no longer downward compatible to earlier ASAM MCD-2 MC versions! For details see chapter 2.1.1.

Note: The operators phy(x) and hex(x) shall only be used for the calculation of virtual or dependent parameters or virtual measurements. phy(x) provides the physical value and hex(x) the internal value of the input quantity. If neither phy(x) nor hex(x) is given, the physical value shall be used for calculation.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.72 FORMULA_INV

Prototype:

```
FORMULA_INV          string      g(x)
```

Parameters:

string	g(x)	function for calculation of the control unit internal value from the physical value. The interpretation proceeds from left to right. Operator preferences, such as power before product/quotient before sum/difference, are taken into account. Brackets are allowed. Permissible operation symbols: see keyword FORMULA. System constants which are defined in SYSTEM_CONSTANT can be used here, if their names comply to the restrictions of data type ident. For that the name of the system constant has to be put in “sysc()”. The value of a system constant used in FORMULA must contain either a numerical value or a string that contains a further FORMULA part (recursive replacement of the text in the formula). Endless loop for the system constant usage is not allowed.

Description:

This keyword allows any kind of formula to be specified for the conversion of measurement values, axis points or table values of an adjustable object from their physical format into the ECU internal format. The interpretation of the formula must be supported by a formula interpreter in the operating system.

Note: FORMULA_INV is necessary if used for adjustable objects (CHARACTERISTIC, AXIS_PTS resp. the corresponding TYPEDEF definitions). Only MEASUREMENT objects do not need an inverse formula.

Example:

Inversion function e.g. for keyword FORMULA

```
FORMULA_INV              "asin( sqrt( (3 - X1)/4 ) )"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.73 FRAME

Prototype:

```
/begin FRAME        ident      Name
                    string     LongIdentifier
                    uint       ScalingUnit
                    ulong      Rate
                    [-> FRAME_MEASUREMENT]
                    [-> IF_DATA]*
/end FRAME
```

Parameters:

ident	Name	unique identifier in the program, referencing is based on this 'name' Note: The name of the frame has to be unique within all frames of the ASAM MCD-2 MC MODULE, i.e. there must not be another FRAME object with the same identifier in the MODULE.
string	LongIdentifier	comment, description
uint	ScalingUnit	This parameter defines the basic scaling unit. The following parameter 'Rate' relates on this scaling unit. The value of ScalingUnit is coded as shown in Appendix: C.
ulong	Rate	The maximum refresh rate of the concerning measurement source in the control unit. The unit is defined with parameter 'ScalingUnit'.

Optional parameters:

-> FRAME_MEASUREMENT	Use this keyword to define the frames measurement objects.
-> IF_DATA	Data record to describe interface specific data of the frame. The parameters associated with this keyword have to be described in the ASAM MCD-2 MC metalanguage.

Description:

For the structuring of a car network involving a very large number of measuring channels, function frames can be defined.
These function frames shall be used in the measurement and calibration system to allow the selection lists for the selection of measuring channels to be represented in a structured manner on the basis of functional viewpoints (function orientation).
This will also be used to describe the packaging of measurement data into sources for CAN frames in a network environment.

Note: The resulting measure rate will be evaluated as product of the base unit, described by the ScalingUnit, and the Rate.

Example:

```
/begin FRAME    ABS_ADJUSTM
                          "function group ABS adjustment"
                        3
                        2       /* 2 msec. */
   FRAME_MEASUREMENT    LOOP_COUNTER TEMPORARY_1
/end FRAME
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.74 FRAME_MEASUREMENT

Prototype:

```
FRAME_MEASUREMENT   (ident      Identifier)*
```

Parameters:

ident	Identifier	Identifier of quantity of respective FRAME (reference to measurement object).

Description:

This keyword can be used to define quantities of respective FRAME.

Note: FRAME_MEASUREMENT may only refer to objects of type MEASUREMENT. This includes objects which are defined via keyword INSTANCE using a TYPEDEF_MEASUREMENT definition as well as instances and instance components of measurement structures.

Example:

```
FRAME_MEASUREMENT       WHEEL_REVOLUTIONS
                        ENGINE_SPEED
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.75 FUNCTION

Prototype:

```
/begin FUNCTION     ident      Name
                    string     LongIdentifier
                    [-> ANNOTATION]*
                    [-> DEF_CHARACTERISTIC]
                    [-> FUNCTION_VERSION]
                    [-> IF_DATA]*
                    [-> IN_MEASUREMENT]
                    [-> LOC_MEASUREMENT]
                    [-> OUT_MEASUREMENT]
                    [-> REF_CHARACTERISTIC]
                    [-> SUB_FUNCTION]
/end FUNCTION
```

Parameters:

ident	Name	unique Identifier in the program, referencing is based on this 'name' Note: The name of the function has to be unique within all functions of the ASAM MCD-2 MC MODULE, i.e. there must not be another FUNCTION object with the same identifier in the MODULE.
string	LongIdentifier	comment, description

Optional parameters:

-> ANNOTATION	Set of notes (represented as multi-line ASCII description texts) which are related. Can serve e.g. as application note.
-> DEF_CHARACTERISTIC	This keyword can be used to define those adjustable objects which are defined in respective function.
-> FUNCTION_VERSION	String to define the version of the function. An measurement and calibration tool should be able to handle function oriented characteristic data.
-> IF_DATA	Data record to describe interface specific data of the function. The parameters associated with this keyword have to be described in the ASAM MCD-2 MC
-> IN_MEASUREMENT	Use this keyword to define the input measurement objects of respective function (input variables).
-> LOC_MEASUREMENT	Use this keyword to define the local measurement objects of respective function (local variables: scope is limited to this function).
-> OUT_MEASUREMENT	Use this keyword to define the output measurement objects of respective function (output variables).
-> REF_CHARACTERISTIC	If the function contains references to some adjustable objects, this keyword can be used to describe this references.
-> SUB_FUNCTION	This keyword can be used to describe the function hierarchy. If the respective function is subdivided into subfunctions, use this keyword to define the subfunctions.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Description:

For the structuring of projects involving a very large number of adjustable objects and measuring channels, functions can be defined. These functions shall be used in the measurement and calibration system to allow the selection lists for the selection of adjustable objects and measuring channels to be represented in a structured, hierarchical manner following the order of input on the basis of functional viewpoints (function orientation).

Note: Since ASAP2 version 1.2.0 the references between functions and measurement objects resp. adjustable objects can be described either with keyword CHARACTERISTIC, AXIS_PTS and MEASUREMENT (see FUNCTION_LIST) or with keyword FUNCTION.

Example:

```
/begin FUNCTION                ID_ADJUSTM          /* name */
                               "function group idling adjustment"
  /begin DEF_CHARACTERISTIC    INJECTION_CURVE
  /end DEF_CHARACTERISTIC
  /begin REF_CHARACTERISTIC    FACTOR_1
  /end REF_CHARACTERISTIC
  /begin IN_MEASUREMENT        WHEEL_REVOLUTIONS  ENGINE_SPEED
  /end IN_MEASUREMENT
  /begin OUT_MEASUREMENT       OK_FLAG  SENSOR_FLAG
  /end OUT_MEASUREMENT
  /begin LOC_MEASUREMENT       LOOP_COUNTER  TEMPORARY_1
  /end LOC_MEASUREMENT
  /begin SUB_FUNCTION          ID_ADJUSTM_SUB
  /end SUB_FUNCTION
/end FUNCTION
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.76 FUNCTION_LIST

Prototype:

```
/begin FUNCTION_LIST
                    ident       (Name)*
/end FUNCTION_LIST
```

Parameters:

ident	Name	list of references to higher-order functions (see FUNCTION)

Description:

This keyword can be used to specify a list of 'functions' to which the relevant adjustable object has been allocated (function orientation).

Note: Since ASAP2 version 1.2.0 the keyword FUNCTION comprises some additional features to describe functional structure and dependencies.

Example:

```
/begin FUNCTION_LIST     ID_ADJUSTM
                         FL_ADJUSTM
                         SPEED_LIM
/end FUNCTION_LIST
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.77 FUNCTION_VERSION

Prototype:

```
FUNCTION_VERSION    string     VersionIdentifier
```

Parameters:

string VersionIdentifier short identifier for the version

Description:

String for identification of the version of a function with maximum MAX_STRING characters.

Example:

```
FUNCTION_VERSION          "BG5.0815"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.78 GROUP

Prototype:

```
/begin GROUP        ident      GroupName
                    string     GroupLongIdentifier
                    [-> ANNOTATION]*
                    [-> FUNCTION_LIST]
                    [-> IF_DATA]*
                    [-> REF_CHARACTERISTIC]
                    [-> REF_MEASUREMENT]
                    [-> ROOT]
                    [-> SUB_GROUP]
/end GROUP
```

Parameters:

ident	GroupName	unique identifier of the group Note: The name of the group has to be unique within all groups of the ASAM MCD-2 MC MODULE, i.e. there must not be another GROUP object with the same identifier in the MODULE.
string	GroupLongIdentifier	Comment, description of the group within a grouping mechanism.

Optional parameters:

-> ANNOTATION	Set of notes (represented as multi-line ASCII description texts) which are related. Can serve e.g. as application note.
-> FUNCTION_LIST	This keyword can be used to specify a list of references to functions.
-> IF_DATA	Data record to describe interface specific data of the group. The parameters associated with this keyword have to be described in the ASAM MCD-2 MC
-> REF_CHARACTERISTIC	If the group contains references to some adjustable objects, this keyword can be used to describe these references.
-> REF_MEASUREMENT	If the group contains references to some measurement objects, this keyword can be used to describe these references.
-> ROOT	This keyword indicates that the group constitutes an independent grouping mechanism (root level) which the MCD system may use as a root point for the hierarchical presentation of groups. All groups referenced via SUB_GROUP (including nested references) constitute a set of groups belonging to the grouping mechanism. Examples for such grouping mechanisms : Group Name = {Software_Components, Calibration_Components, Editor_Selection_Lists}
-> SUB_GROUP	This keyword can be used to describe the group hierarchy. If the respective group is subdivided into sub-groups, use this keyword to define the subgroups. In particular, SUB_GROUP references the groups belonging

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

to a grouping mechanism indicated with the optional keyword ROOT (see above).

Description:

These GROUPs shall be used in the measurement and calibration system to provide selection lists (groups) of adjustable objects and measuring channels.
For the structuring of projects involving a very large number of adjustable objects and measuring channels, an unlimited number of grouping mechanisms, each constituted from a root group containing subgroups (including nested references), can be defined. Such root groups are used in the MCD system for initial display of the available groups, as the root of a tree containing the referenced subgroups. Use cases are e.g. software components which define the C file assignment, calibration components which describe the calibration engineer's viewpoint, editor selection lists which can define the presentation of calibration objects and their related measurement quantities.

Example:

```
/begin GROUP              SOFTWARE_COMPONENTS
                          "assignment of the definitions to C files"
                          ROOT
  /begin SUB_GROUP        INJE
                          C6TD
  /end SUB_GROUP
/end GROUP

/begin GROUP              INJE
                          "Subsystem Injection"
  /begin SUB_GROUP        injec1
                          injec2
  /end SUB_GROUP
/end GROUP

/begin GROUP              Injec1
                          "Module filename Injec1"
  /begin REF_CHARACTERISTIC
                          INJECTION_CURVE
  /end REF_CHARACTERISTIC
  /begin REF_MEASUREMENT
                          LOOP_COUNTER
                          TEMPORARY_1
  /end REF_MEASUREMENT
/end GROUP

/begin GROUP              Injec2
                          "Module filename Injec2"
  /begin REF_CHARACTERISTIC
                          INJECTION_ADJUST
  /end REF_CHARACTERISTIC
  /begin REF_MEASUREMENT
                          GAS_INPUT
                          WHEEL_SPEED
  /end REF_MEASUREMENT
/end GROUP


/begin GROUP              C6TD
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
                          "Shift Point Control"
  /begin SUB_GROUP        c6tdvder
                          c6tdertf
  /end  SUB_GROUP
/end GROUP

/begin GROUP              c6tdvder
                          "Module filename c6tdvder"
  /begin  REF_CHARACTERISTIC
                          SHIFT23_CURVE
  /end  REF_CHARACTERISTIC
  /begin REF_MEASUREMENT
                          LOOP_COUN2
                          NO_GEAR
  /end REF_MEASUREMENT
/end GROUP

/begin GROUP              c6tderft
                          "Module filename c6tderft"
  /begin REF_CHARACTERISTIC
                          LUP23_CURVE
  /end  REF_CHARACTERISTIC
  /begin REF_MEASUREMENT
                          TRANSMISSION_SP
                          ENGINE_SPEED
  /end REF_MEASUREMENT
/end GROUP


/begin GROUP              CALIBRATION_COMPONENTS
                          "assignment of the definitions to calibration
                          components"
                          ROOT
  /begin SUB_GROUP
                          Winter_Test
                          Summer_Test
  /end SUB_GROUP
/end GROUP

/begin GROUP              CALIBRATION_COMPONENTS_L4
                          "L4-PCM 2002 cals"
                          ROOT
  /begin SUB_GROUP        LUFT
                          CLOSED_LOOP
  /end SUB_GROUP
/end GROUP

/begin GROUP              LUFT
                          "Cals in LUFT Subsystem"
  /begin REF_CHARACTERISTIC
                          KfLUFT_n_EngSpdThrsh
                          KtLUFT_ScaledVE
                          KaLUFT_AirPerCylCoeff
  /end  REF_CHARACTERISTIC
/end GROUP

/begin GROUP              CLOSED_LOOP
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
                        "Cals in FCLS, FCLP & FCLL Subsystem"
  /begin REF_CHARACTERISTIC
                        KaFCLP_U_O2LeanThrsh
                        KfFCLP_t_O2AgainstMax
  /end REF_CHARACTERISTIC
/end GROUP

/begin GROUP            Winter_Test
                        "Flash this in winter time"
  /begin REF_CHARACTERISTIC
                        GASOLINE_CURVE
  /end  REF_CHARACTERISTIC
/end GROUP

/begin GROUP            Summer_Test
                        "Flash that in summer time"
  /begin REF_CHARACTERISTIC
                        SUPER_CURVE
  /end  REF_CHARACTERISTIC
/end GROUP

/begin GROUP            SOFTWARE_COMPONENTS
                        " L4-PCM 2002 C modules"
                        ROOT
  /begin SUB_GROUP
                        luftkmgr.c
                        fclpkout.c
                        viosmeng.c
  /end SUB_GROUP
/end GROUP

/begin GROUP            luftkmgr.c
                        "Objects in luftkmgr.c"
  /begin REF_CHARACTERISTIC
                        KtLUFT_ScaledVE
  /end REF_CHARACTERISTIC
/end GROUP

/begin GROUP            fclpkout.c
                        "Objects in fclpkout.c"
  /begin REF_CHARACTERISTIC
    KaFCLP_U_O2LeanThrsh
                          KfFCLP_t_O2AgainstMax
  /end REF_CHARACTERISTIC
/end GROUP

/begin GROUP            viosmeng.c
                        "Objects in viosmeng.c"
  /begin REF_CHARACTERISTIC
                        VfVIOS_n_EngSpdLORES
                        VfVIOS_p_AmbientAirPres
  /end REF_CHARACTERISTIC
/end GROUP
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.79 GUARD_RAILS

Prototype:

```
GUARD_RAILS
```

Description:

This keyword is used to indicate that an adjustable CURVE, MAP or AXIS_PTS uses guard rails. The Measurement and Calibration System does not allow the user to edit the outermost values or axis points of the adjustable object, but calculates them as follows:

Table 11 GUARD_RAILS

AXIS_PTS	CURVE	MAP
(X_0) = AXIS_PTS.LowerLimit	$(X_0) = (X_1)$	$(X_i, Y_0) = (X_j, Y_1)$
(X_m) = AXIS_PTS.UpperLimit	$(X_m) = (X_{m-1})$	$(X_i, Y_n) = (X_j, Y_{n-1})$
		$(X_0, Y_j) = (X_1, Y_j)$
		$(X_m, Y_j) = (X_{m-1}, Y_j)$

$0 < i < m$, m = Number of X-axis points
$0 < j < n$, n = Number of Y-axis points

Example:

```
/begin CHARACTERISTIC    F_INJ_CORR  /* name */
                         "Injector correction factor''
                                     /* long identifier */
                         CURVE       /* type */
                         0x7140      /* address */
                         REC12       /* deposit */
                         10.0        /* maxdiff */
                         C_INJF      /* conversion */
                         0.0         /* lower limit */
                         199.0       /* upper limit */
                         GUARD_RAILS /* uses guard rails */
  /begin AXIS_DESCR      /* description of X-axis points */
                         STD_AXIS    /* standard axis points */
                         N           /* reference to input quantity*/
                         C_TEMP      /* conversion */
                         10     /* maximum number of axis points*/
                         -40.0       /* lower limit */
                         150.0       /* upper limit */
  /end AXIS_DESCR
/end CHARACTERISTIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.80 HEADER

Prototype:

```
/begin HEADER       string     Comment
                    [-> PROJECT_NO]
                    [-> VERSION]
/end HEADER
```

Parameters:

string	Comment:	comment, description

Optional parameters:

-> VERSION	version number
-> PROJECT_NO	project number

Description:

Header information on a project. A project can comprise several ECU's or devices.

Example:

```
/begin HEADER           "see also specification XYZ of 01.02.1994"
      VERSION           "BG5.0815"
      PROJECT_NO        M4711Z1
/end HEADER
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.81 IDENTIFICATION

Prototype:

```
IDENTIFICATION      uint        Position
                    datatype    Datatype
```

Parameters:

uint	Position	position of the 'identifier' in the deposit structure.
datatype	Datatype	word length of the 'identifier''

Description:

Description of an 'identifier' in an adjustable object.

Example:

```
IDENTIFICATION          1
                        UWORD
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.82 IF_DATA (Example)

Prototype:

```
/begin IF_DATA      ident         Name
                    [-> ...]
/end IF_DATA
```

Parameters:

ident	Name	identifier of Interface, The prefix "ASAP1B_" is reserved for ASAM and can be not used for proprietary Interfaces.

Optional parameters:

-> ...	Data record to describe interface specific data. The parameters associated with this keyword have to be described in the ASAM MCD-2 MC metalanguage. These parameters describe e.g. the access methods to the measurement data collection, serial communication and so on.

Description:

Definition of interface-specific description data.

Example:

```
/begin IF_DATA
    ASAP1B_EXAMPLE  /* Name of device */
                /* interface-specific parameters described in A2ML */
    /begin DP_BLOB
                0x12129977
                0xFF
    /end DP_BLOB
                /* interface-specific parameters described in A2ML */
    /begin PA_BLOB
                "Pumpenkennfeld"
                1
                2
                17
    /end PA_BLOB
/end IF_DATA
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.83 IN_MEASUREMENT

Prototype:

```
/begin IN_MEASUREMENT (ident   Identifier)*
/end IN_MEASUREMENT
```

Parameters:

ident	Identifier	Identifier of input quantity of respective function (reference to measurement object).

Description:

This keyword can be used to define input quantities of respective function.

Note: IN_MEASUREMENT may only refer to objects of type MEASUREMENT. This includes objects which are defined via keyword INSTANCE using a TYPEDEF_MEASUREMENT definition as well as instances and instance components of measurement structures.

Example:

```
/begin IN_MEASUREMENT    WHEEL_REVOLUTIONS
                         ENGINE_SPEED
/end IN_MEASUREMENT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.84 INPUT_QUANTITY

Prototype:

```
INPUT_QUANTITY      ident      InputQuantity
```

Parameters:

ident	InputQuantity	Name of the MEASUREMENT to be used as input quantity for the corresponding axis.

Description:

This keyword can be used to overwrite the input quantity specified at type definition for a specific axis instance.

Note: If the referenced input quantity is an element of an array or a component of a structure, the identifier to be used for the reference shall be the name built according to rules described at INSTANCE.

Example:

```
/begin INSTANCE
  MyInstance ""
  StructureTemplate
  0x2000
  /begin OVERWRITE
        MapComponent 2
        INPUT_QUANTITY Speed2          // overruling the input quantity
                                       // of the Y axis
  /end OVERWRITE
/end INSTANCE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.85 INSTANCE

Prototype:

```
/begin INSTANCE     ident      Name
                    string     LongIdentifier
                    ident      TypedefName
                    ulong      Address
                    [-> ANNOTATION]*
                    [-> CALIBRATION_ACCESS]
                    [-> DISPLAY_IDENTIFIER]
                    [-> ECU_ADDRESS_EXTENSION]
                    [-> IF_DATA]*
                    [-> LAYOUT]
                    [-> MATRIX_DIM]
                    [-> MAX_REFRESH]
                    [-> MODEL_LINK]
                    [-> OVERWRITE]*
                    [-> READ_WRITE]
                    [-> SYMBOL_LINK]
/end INSTANCE
```

Parameters:

ident Name — Identifier of the instance in the ECU program.

Note: For structure instances, the full identifiers of the measure or calibration components are built by adding the component name(s) to the instance name separated by dot(s). If necessary, array indices are added in '[']' at the end of the instance name or at the end of component name(s). Array indices are added with decimal notation without leading zeros. The numbering starts with 0.
All resulting identifiers must be unique in the MODULE, i.e. there must not be another INSTANCE, AXIS_PTS, BLOB, CHARACTERISTIC or MEASUREMENT object with the same identifier in the MODULE. Resulting identifiers are full identifiers or partial identifiers starting from the root (see Example for valid and invalid identifier names).

Note: The resulting identifiers of structure instance components can be used to refer to measure or calibration objects which are part of a structure – e.g. for usage as input quantity in axes or for assignment to groups / functions.

Note: Against the common rules for valid identifiers, it is not allowed to use '.' and '[', ']' in the name of a structure instance.

String LongIdentifier — Comment, description for the instance. If no description is specified (i.e. empty string), the description of the referenced type is re-used for the instance.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

ident	TypedefName	Identifier of the referenced type definition. This type definition can either be a TYPEDEF_AXIS, TYPEDEF_BLOB, TYPEDEF_CHARACTERISTIC, TYPEDEF_MEASUREMENT or a TYPEDEF_STRUCTURE.
ulong	Address	Start address of the instance in the ECU memory

Optional parameters

-> ANNOTATION	Set of related notes.
-> CALIBRATION_ACCESS	Specifies the access rights for instances of adjustable object types.
-> DISPLAY_IDENTIFIER	Specifies a display name (alternative to the 'Name' attribute).
-> ECU_ADDRESS_EXTENSION	Additional address information, e.g. to distinguish different address spaces of an ECU.
-> IF_DATA	Data record to describe interface specific data of the instance. The parameters associated with this keyword have to be described in the ASAM MCD-2 MC meta language.
-> LAYOUT	Specifies the memory layout of this INSTANCE, if it is a multi-dimensional array (MATRIX_DIM is used inside this INSTANCE with more than one dimension). If the keyword is missing multi-dimensional arrays are interpreted row by row (ROW_DIR).
-> MATRIX_DIM	Defines the dimension if the instance is an array of the referenced type definition.
-> MAX_REFRESH	Maximum refresh rate of this INSTANCE in the control unit. The existence of this keyword implies that the instance content is changed by the control unit.
-> MODEL_LINK	name used in the software model.
-> OVERWRITE	This keyword can be used to overwrite the default properties specified at the referenced type definition. For each structure component of the instance, only one OVERWRITE block is allowed with the same axis number.
-> READ_WRITE	Marks a measurement instance as writeable.
-> SYMBOL_LINK	Reference to a symbol name in a linker map file.

Description:

Specification of a measure object, a calibration object or a structure instance, i.e. a composition of several measure and calibration objects. Which object type is created depends on the referenced type definition.

Note: LAYOUT and MATRIX_DIM at the INSTANCE do not override the corresponding keywords at TYPEDEF_xx. If LAYOUT and MATRIX_DIM are specified both at the INSTANCE and the referenced TYPEDEF_xx then they have to be applied consecutively.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Example:

See TYPEDEF_STRUCTURE

Example for valid and invalid identifier names

The following example illustrates the rule for defining valid A2L identifiers that do not collide with other identifiers of the same namespace.
In this example, two structures and one instance are defined in C (we use C here since it is shorter than the corresponding definition in A2L):

```
typedef struct {
  int a;
  int b;
  } structType1;

typedef struct {
  int c[10];
  structType1 d;
  } structType2;

structType2 myInstance;
```

In the following, find a list of identifiers for INSTANCE, AXIS_PTS, BLOB, CHARACTERISTIC or MEASUREMENT that are allowed to be defined in A2L without collision in the namespace.

```
d.a
myInstance.f
myInstance.d[0]
myInstance.c[1].x
```

In the following, find a list of identifiers which must not be used within the A2L file for INSTANCE, AXIS_PTS, BLOB, CHARACTERISTIC or MEASUREMENT since they would collide with identifiers of the components of the myInstance structure.

```
myInstance
myInstance.d
myInstance.d.a
myInstance.c
myInstance.c[1]
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.86 LAYOUT

Prototype:

```
LAYOUT              enum        IndexMode
```

Parameters:

enum IndexMode: For multi-dimensional arrays, this attribute is used to describe how the array values are mapped onto the one-dimensional address space:

ROW_DIR: storage of array values in row-major order

COLUMN_DIR: storage of array values in column-major order

For an example, see FNC_VALUES.

Description:

This keyword describes the layout of a multi-dimensional array. It can be used at MEASUREMENT, TYPEDEF_MEASUREMENT, STRUCTURE_COMPONENT and INSTANCE.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.87 LEFT_SHIFT

Prototype:

```
LEFT_SHIFT          ulong      Bitcount
```

Parameters:

ulong	Bitcount	Shift ‚Bitcount‘ bits to the left

Description:

The LEFT_SHIFT keyword is only used within the BIT_OPERATION keyword. See description of BIT_OPERATION.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.88 LIMITS

Prototype:

```
LIMITS              float      LowerLimit
                    float      UpperLimit
```

Parameters:

float	LowerLimit	lower limit of the corresponding measure or calibration object, physical value
float	UpperLimit	upper limit of the corresponding measure or calibration object, physical value

Description:

This keyword can be used to overwrite the default limits specified at type definition for a specific instance.

Example:

```
/begin INSTANCE
  MyInstance ""
  ParameterTemplate
  0x2000
  /begin OVERWRITE
        THIS 0
        LIMITS 100 280    // overruling the limits
  /end OVERWRITE
/end INSTANCE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.89 LOC_MEASUREMENT

Prototype:

```
/begin LOC_MEASUREMENT          (ident       Identifier)*
/end LOC_MEASUREMENT
```

Parameters:

ident	Identifier	Identifier of local quantity of respective function (reference to measurement object).

Description:

This keyword can be used to define local quantities of respective function.

Note: LOC_MEASUREMENT may only refer to objects of type MEASUREMENT. This includes objects which are defined via keyword INSTANCE using a TYPEDEF_MEASUREMENT definition as well as instances and instance components of measurement structures.

Example:

```
/begin LOC_MEASUREMENT   LOOP_COUNTER
                         TEMPORARY_1
/end LOC_MEASUREMENT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.90 MAP_LIST

Prototype:

```
/begin MAP_LIST     (ident     Name)*
/end MAP_LIST
```

Parameters:

ident	Name	identifier of a MAP (see CHARACTERISTIC)

Description:

This keyword can be used to specify the list of MAPs which comprise a CUBOID. This keyword is required because CUBOID data will not be at contiguous memory locations if a CUBOID is composed of several MAPs. If MAP_LIST is used at CHARACTERISTIC, only one AXIS_DESCR is accepted. This AXIS_DESCR describes the Z-axis. (The X- and Y-axes are described at the MAPs referenced by MAP_LIST.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.91 MATRIX_DIM

Prototype:

```
MATRIX_DIM          (uint      Dim)+
```

Parameters:

uint	Dim	number of values in this dimension

Description:

This keyword is used to describe the dimensions of a multi-dimensional array of objects. It can be used at the definition of MEASUREMENT and CHARACTERISTIC (resp. at the corresponding type definitions) as well as at STRUCTURE_COMPONENTs and INSTANCE definitions.
The number of Dim entries depends on the number of dimensions. For each dimension one Dim entry is necessary. The first Dim value represents the x dimension.
If NUMBER or ARRAY_SIZE are used in the CHARACTERISTIC or MEASUREMENT the value must be the same as the product of all dimensions of MATRIX_DIM.

Note: This keyword can be used to define a one, two or more dimensional object.

Note: The dimensions value "0" shall not be used.

Note: The dimensions value "1" has a special meaning. This dimension has only one value (set of values). But for data exchange Objects (ASAP3, MCD3, CDF, …) this dimension is respected.

Note: If more than one array dimension is specified for a measure object, for an instance or for a structure component, the layout of the array elements shall be specified with the additional keyword LAYOUT in the corresponding context.

Example:

```
/* Example for a 3-dim object */
MATRIX_DIM               2
                         4
                         3

/* Example for a 2-dim object */
MATRIX_DIM               2
                         4

/* Example for a 1-dim object */
MATRIX_DIM               6

/* Example for a 2-dim object where one dimension has only the size of
1 */
/* For ASAP3 / MCD3 the object type is 2 dimensions */
MATRIX_DIM               6
                         1
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.92 MAX_GRAD

Prototype:

```
MAX_GRAD            float       MaxGradient
```

Parameters:

float	MaxGradient	maximum permissible gradient

Description:

This keyword is used to specify a maximum permissible gradient for an adjustable object in relation to an axis:

$$MaxGrad_x = maximum(absolut((W_{i,k} - W_{i-1,k})/(X_i - X_{i-1})))$$

$$MaxGrad_y = maximum(absolut((W_{i,k} - W_{i,k-1})/(Y_i - Y_{k-1})))$$

Example:

```
MAX_GRAD                200.0
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.93 MAX_REFRESH

Prototype:

```
MAX_REFRESH         uint        ScalingUnit
                    ulong       Rate
```

Parameters:

uint	ScalingUnit	this parameter defines the basic scaling unit. The following parameter 'Rate' relates on this scaling unit. The value of ScalingUnit is coded as shown in Appendix: C.
ulong	Rate	the maximum refresh rate of the concerning measurement object in the control unit. The unit is defined with parameter 'ScalingUnit'

Description:

This optional keyword can be used to specify the maximum refresh rate in the control unit. The resulting refresh rate will be evaluated as product of the base unit, described by the ScalingUnit, and the Rate.

Example:

```
MAX_REFRESH 3
            15  /* Refresh Rate = 15 msec */
MAX_REFRESH 103
            3  /* Refresh Rate = Every third cylinder segment */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.94 MEASUREMENT

Prototype:

```
/begin MEASUREMENT  ident      Name
                    string     LongIdentifier
                    datatype   Datatype
                    ident      Conversion
                    uint       Resolution
                    float      Accuracy
                    float      LowerLimit
                    float      UpperLimit
                    [-> ADDRESS_TYPE]
                    [-> ANNOTATION]*
                    [-> ARRAY_SIZE]
                    [-> BIT_MASK]
                    [-> BIT_OPERATION]
                    [-> BYTE_ORDER]
                    [-> DISCRETE]
                    [-> DISPLAY_IDENTIFIER]
                    [-> ECU_ADDRESS]
                    [-> ECU_ADDRESS_EXTENSION]
                    [-> ERROR_MASK]
                    [-> FORMAT]
                    [-> FUNCTION_LIST]
                    [-> IF_DATA]*
                    [-> LAYOUT]
                    [-> MATRIX_DIM]
                    [-> MAX_REFRESH]
                    [-> MODEL_LINK]
                    [-> PHYS_UNIT]
                    [-> READ_WRITE]
                    [-> REF_MEMORY_SEGMENT]
                    [-> SYMBOL_LINK]
                    [-> VIRTUAL]
/end MEASUREMENT
```

Parameters:

ident	Name	unique identifier in the ECU program Note: The name has to be unique within all measure and adjustable objects of the MODULE, i.e. there must not be another AXIS_PTS, MEASUREMENT, CHARACTERISTIC, BLOB or INSTANCE object with an equal identifier in the same MODULE. Furthermore it is not allowed to have a structure component with equal name in the MODULE. (Rules for building the full identifiers of structure components: see at INSTANCE).
string	LongIdentifier	comment, description
datatype	Datatype	data type of the measurement
ident	Conversion	Reference to the relevant record of the description of the conversion method (see COMPU_METHOD). If there is no conversion method, as in the case of CURVE_AXIS, the parameter ‘Conversion’ should be set to

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

		"NO_COMPU_METHOD" (measurement and calibration systems must be able to handle this case).
uint	Resolution	smallest possible change in bits, recommended value: 1 (This parameter is currently not interpreted by any known MC-System. It can be ignored.)
float	Accuracy	possible variation from exact value in %, recommended value: 0 (This parameter is currently not interpreted by any known MC-System. It can be ignored.)
float	LowerLimit	plausible range of table values, lower limit
float	UpperLimit	plausible range of table values, upper limit

Note: Depending on the type of conversion, the limit values are interpreted as physical or internal values.
For conversions of type COMPU_VTAB and COMPU_VTAB_RANGE, the limit values are interpreted as internal values. For all other conversion types, the limit values are interpreted as physical values.

Optional parameters:

-> ADDRESS_TYPE	Addressing of the MEASUREMENT (see in Table 5 enum addrtype) Note: If the keyword ADDRESS_TYPE is missing DIRECT addressing is used.
-> ANNOTATION	Set of notes (represented as multi-line ASCII description texts) which are related. Can serve e.g. as application note.
-> ARRAY_SIZE	This keyword marks a measurement object as an array of measurement values.
-> BIT_MASK	With deviation from the standard value 0xFFFFFFFF this parameter can be used to mask out bits.
-> BIT_OPERATION	The BIT_OPERATION keyword can be used to perform operation on the masked out value.
-> BYTE_ORDER	With deviation from the standard value this parameter can be used to specify the byte order (Intel format, Motorola format)
-> DISCRETE	This keyword indicates that the measurement values are discrete values which should not be interpolated – e.g. in graphic display windows or further calculations. This flag can be used e.g. for integer objects describing states. If the keyword is not specified the values are interpreted as continuous values which can be interpolated.
-> DISPLAY_IDENTIFIER	Can be used as a display name (alternative to the 'name' attribute).
-> ECU_ADDRESS	Address of the measurement in the memory of the control unit.
-> ECU_ADDRESS_EXTENSION	This keyword is an additional address information. For instance it can be used, to distinguish different address spaces of an ECU (multi-micro controller devices).

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

-> ERROR_MASK	With deviation from the standard value 0x00000000 this parameter can be used to mask bits of a MEASUREMENT which indicate that the value is in error.
-> FORMAT	With deviation from the display format specified in the COMPU_METHOD record referenced by parameter <Conversion> a special display format can be specified to be used to display the measurement values.
-> FUNCTION_LIST	This keyword can be used to specify a list of 'functions' to which this measurement object has been allocated. Note: Since ASAP2 version 1.2.0 the keyword FUNCTION comprises some additional features to describe functional structure and dependencies.
-> IF_DATA	Data record to describe interface specific data of the measurement. The parameters associated with this keyword have to be described in the ASAM MCD-2 MC metalanguage.
-> LAYOUT	For multi-dimensional measurement arrays this keyword can be used to specify the layout of the array. If the keyword is missing, multi-dimensional measurement arrays are interpreted row by row (ROW_DIR).
-> MATRIX_DIM	Shows the size and dimension of a multi-dimensional measurement. If the MATRIX_DIM keyword is used, then the option ARRAY_SIZE is not needed. However, if the keywords ARRAY_SIZE and MATRIX_DIM are both used, the resulting value in ARRAY_SIZE must be the same as xDim * yDim *zDim for MATRIX_DIM. If the keyword is missing the array has only one dimension with the size given at ARRAY_SIZE.
-> MAX_REFRESH	Maximum refresh rate of this measurement in the control unit
-> MODEL_LINK	name used in the software model.
-> PHYS_UNIT	With this keyword a physical unit can be specified for the measurement object if no conversion rule is referenced (NO_COMPU_METHOD). Note: If a conversion rule is referenced the additional usage of PHYS_UNIT overrules the unit specified at the referenced conversion rule
-> READ_WRITE	Keyword to mark this measurement object as writeable
-> REF_MEMORY_SEGMENT	Reference to the memory segment.
-> SYMBOL_LINK	Reference to symbol name within a linker map file.
-> VIRTUAL	For description of a virtual measurement (see VIRTUAL)

Description:

The MEASUREMENT keyword is used to describe the parameters for the processing of a measurement object.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Example:

```
/begin MEASUREMENT      N                 /* name */
                        "Engine speed"    /* long identifier */
                        UWORD             /* datatype */
                        R_SPEED_3         /* conversion */
                        1                 /* resolution */
                        0                 /* accuracy */
                        120.0             /* lower limit */
                        8400.0            /* upper limit */
  PHYS_UNIT             "mph"
  BIT_MASK              0x0FFF
  /begin BIT_OPERATION
     RIGHT_SHIFT        4                 /*4 positions*/
                        SIGN_EXTEND
  /end BIT_OPERATION
     BYTE_ORDER         MSB_FIRST
  REF_MEMORY_SEGMENT    Data2
  /begin FUNCTION_LIST  ID_ADJUSTM
                        FL_ADJUSTM
  /end FUNCTION_LIST
  /begin IF_DATA ISO    SND
                        0x10
                        0x00
                        0x05
                        0x08
                        RCV
                        4
                        long
  /end IF_DATA
/end MEASUREMENT

/begin MEASUREMENT      VdiagStatus       /* name */
                        "VdiagStatus"     /* long identifier */
                        SWORD             /* datatype */
                        CM_DiagSTatus     /* conversion */
                        1                 /* resolution */
                        0                 /* accuracy */
                        -32768            /* lower limit */
                        32767             /* upper limit */
     ECU_ADDRESS        0x003FDFE0
/end MEASUREMENT

/begin MEASUREMENT      VfSpinLoss        /* name */
                        "VfSpinLoss"      /* long identifier */
                        UWORD             /* datatype */
                        CM_RPM            /* conversion */
                        1                 /* resolution */
                        0                 /* accuracy */
                        -4096             /* lower limit */
                        4095.875          /* upper limit */
     ECU_ADDRESS        0x003FE380
/end MEASUREMENT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.95 MEMORY_LAYOUT

Prototype:

```
/begin MEMORY_LAYOUT  enum     PrgType
                      ulong    Address
                      ulong    Size
                      long[5]  Offset
                      [-> IF_DATA]*
/end MEMORY_LAYOUT
```

Parameters:

enum	PrgType	Description of the program segments divided into: PRG_CODE = program code PRG_DATA = program data PRG_RESERVED = other
ulong	Address	Initial address of the program segment to be described.
ulong	Size	Length of the program segment to be described.
long [5]	Offset	In special ECU programs, so-called 'mirrored segments' may occur (Figure 6). A mirrored segment is a copy of another program segment. During adjustment the data changes are introduced in the relevant memory segment as well as in all mirrored segments.

Optional parameters

-> IF_DATA	Data record to describe interface specific data of the memory layout. The parameters associated with this keyword have to be described in the ASAM MCD-2 MC metalanguage.

Description:

This data record is used to describe an ECU program. The description indicates how the emulation memory is divided into the individual segments.

Example:

```
/begin MEMORY_LAYOUT     PRG_RESERVED
                         0x0000
                         0x0400
                         -1 -1 -1 -1 -1
/end MEMORY_LAYOUT
/begin MEMORY_LAYOUT     PRG_CODE
                         0x0400
                         0x3C00
                         -1 -1 -1 -1 -1
/end MEMORY_LAYOUT
/begin MEMORY_LAYOUT     PRG_DATA
                         0x4000
                         0x0200
                         0x10000
                         0x20000
                         -1 -1 -1
/end MEMORY_LAYOUT
/begin MEMORY_LAYOUT     PRG_DATA
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
                        0x4200
                        0x0E00
                        -1 -1 -1 -1 -1
/end MEMORY_LAYOUT
/begin MEMORY_LAYOUT    PRG_DATA
                        0x14200
                        0x0E00
                        -1 -1 -1 -1 -1
/end MEMORY_LAYOUT
/begin MEMORY_LAYOUT    PRG_DATA
                        0x24200
                        0x0E00
                        -1 -1 -1 -1 -1
/end MEMORY_LAYOUT
```

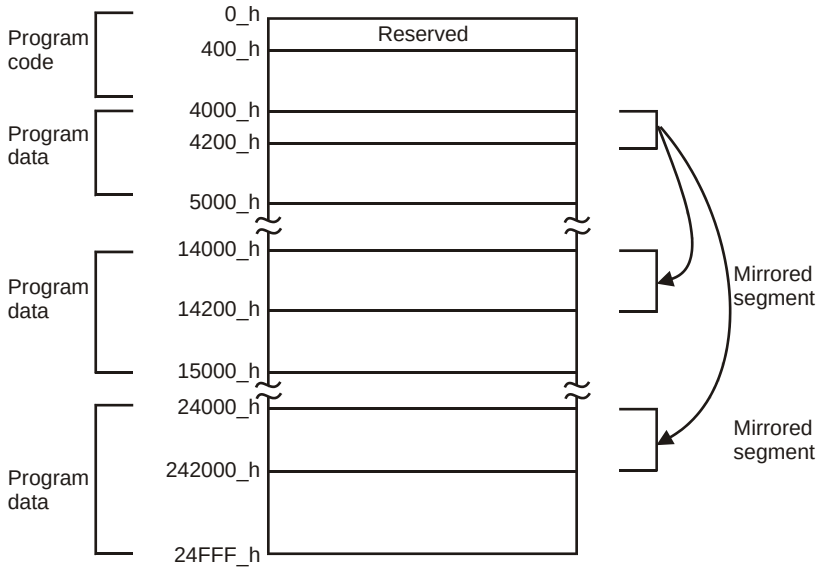


Figure 6 Memory layout (mirrored segments)

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.96 MEMORY_SEGMENT

Prototype:

```
/begin MEMORY_SEGMENT ident    Name
                      string   LongIdentifier
                      enum     PrgType
                      enum     MemoryType
                      enum     Attribute
                      ulong    Address
                      ulong    Size
                      long[5]  Offset
                      [-> IF_DATA]*
/end MEMORY_SEGMENT
```

Parameters:

ident	Name	identifier, name of the memory segment Note: The name of the memory segment has to be unique within all memory segments of the MODULE, i.e. there must not be another MEMORY_SEGMENT with the same identifier in the MODULE.
string	LongIdentifier	comment, description
enum	PrgType	PrgTypes: CALIBRATION_VARIABLES = Values which are changeable by both the ECU and the MC tool. They do not exist in the Hex-file. Use Case: Adaptive characteristics CODE = program code DATA = program data allowed for online calibration EXCLUDE_FROM_FLASH = values which exist in the ECU but do not exist in the HEX-file. OFFLINE_DATA = program data allowed only for offline calibration RESERVED = reserved segments SERAM = program data for serial emulation VARIABLES = program variables
enum	MemoryType	Description of the type of memory used EEPROM = segment of EEPROM EPROM = segment of EPROM FLASH = segment of FLASH RAM = segment of RAM ROM = segment of ROM REGISTER = segment of CPU registers NOT_IN_ECU = Values which are available in the Hex file and the MC tool's data base but not in the ECU's memory. These values cannot be downloaded or uploaded to/from the ECU's memory.
enum	Attribute	attributes INTERN = internal segment (inside CPU)

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

		EXTERN = external segment (outside CPU)
ulong	Address	Initial address
ulong	Size	Length of the segment
long[5]	Offset	Offset address of mirrored segments

Optional Parameters:

-> IF_DATA	Data record to describe interface specific data of the memory segment. The parameters associated with this keyword have to be described in the ASAM MCD-2 MC metalanguage.

Description:

The new keyword MEMORY_SEGMENT is used to replace the existing keyword MEMORY_LAYOUT.
The keywords MEMORY_SEGMENT and MEMORY_LAYOUT can be used in parallel.
The parameter Offset is to be used (as within the former MEMORY_LAYOUT) to describe several mirrored segments.
MEMORY_SEGMENTS shall not overlap mutually to get a linear address space.

Note: Restrictions for the memory type NOT_IN_ECU:
Objects residing within the virtual memory segment are not allowed to have any references to objects in other types of memory segments. The only link allowed between a virtual memory segment and other memory segment types is via a transformer (e.g.: A characteristic (map) within a virtual memory segment shall not use an axis outside the virtual memory segment).

The following table gives a description for some useful combinations of PrgType and MemoryType and their meanings:

Table 12 MEMORY_SEGMENT

Combination	Meaning
CODE / FLASH	Executable code, has to be preserved for download and HEX-file generation.
DATA / FLASH or DATA / EEPROM	Calibration data, can be modified by the user via calibration systems, has to be preserved for download and HEX-file generation.
DATA / RAM	Calibration data, will be modified by ECU and calibration systems, has to be preserved for download and HEX-file generation.
DATA / NOT_IN_ECU	Calibration data which can be modified by the user via MC tools but is only available in the MC tool's data base and in the HEX-file.
RESERVED / FLASH	ECU specific code or data, has to be preserved for HEX-file generation but not for download.
CALIBRATION_VARIABLES / RAM	Calibration data (characteristics), will be modified by ECU and calibration systems. The calibration data is not part of the HEX-file.
OFFLINE_DATA / FLASH	Calibration data, will be modified only without ECU access, online calibration is not allowed.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

VARIABLES / RAM	RAM of the ECU for variables (measurement values and others).
SERAM / RAM	ECU-RAM section available for serial calibration. For usage see also: CALIBRATION_METHOD.

Note: The MemoryType FLASH has been used as synonym for EPROM and ROM

Example:

```
/begin MEMORY_SEGMENT    Data1
                         "Data internal Flash"
                         DATA
                         FLASH
                         INTERN
                         0x4000
                         0x0200
                         -1 -1 -1 -1 -1
/end MEMORY_SEGMENT
/begin MEMORY_SEGMENT    Data2
                         "Data external Flash"
                         DATA
                         FLASH
                         EXTERN
                         0x7000
                         0x2000
                         -1 -1 -1 -1 -1
/end MEMORY_SEGMENT
/begin MEMORY_SEGMENT    VirtualData
                         "Virtual memory segment not in ECU "
                         DATA
                         NOT_IN_ECU
                         EXTERN
                         0xFFFF0000
                         0x00010000
                         -1 -1 -1 -1 -1
/end MEMORY_SEGMENT
/begin MEMORY_SEGMENT    Code1
                         "Code external Flash"
                         CODE
                         FLASH
                         EXTERN
                         0x9000
                         0x3000
                         -1 -1 -1 -1 -1
/end MEMORY_SEGMENT
/begin MEMORY_SEGMENT    ext_Ram
                         "external RAM"
                         DATA
                         RAM
                         EXTERN
                         0x30000
                         0x1000
                         -1 -1 -1 -1 -1
/end MEMORY_SEGMENT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
/begin MEMORY_SEGMENT    int_Ram
                         "internal RAM"
                         DATA
                         RAM
                         INTERN
                         0x0000
                         0x0200
                         -1 -1 -1 -1 -1
/end MEMORY_SEGMENT
/begin MEMORY_SEGMENT    Seram1
                         "emulation RAM 1"
                         SERAM
                         RAM
                         EXTERN
                         0x7000
                         0x1000
                         -1 -1 -1 -1 -1
/end MEMORY_SEGMENT
/begin MEMORY_SEGMENT    Seram2
                         "emulation RAM 2"
                         SERAM
                         RAM
                         INTERN
                         0x8000
                         0x1000
                         -1 -1 -1 -1 -1
/end MEMORY_SEGMENT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.97 MOD_COMMON

Prototype:

```
/begin MOD_COMMON   string     Comment
                    [-> ALIGNMENT_BYTE]
                    [-> ALIGNMENT_FLOAT32_IEEE]
                    [-> ALIGNMENT_FLOAT64_IEEE]
                    [-> ALIGNMENT_INT64]
                    [-> ALIGNMENT_LONG]
                    [-> ALIGNMENT_WORD]
                    [-> BYTE_ORDER]
                    [-> DATA_SIZE]
                    [-> DEPOSIT]
/end MOD_COMMON
```

Parameters:

string	Comment	comment, description

Optional parameters:

-> ALIGNMENT_BYTE	Declares the alignment of bytes in the complete module. The alignment is 1 if parameter is missing.
-> ALIGNMENT_LONG	Declares the alignment of longs in the complete module. The alignment is 4 if parameter is missing.
-> ALIGNMENT_FLOAT32_IEEE	Declares the alignment of 32 bit floats in the complete module. The alignment is 4 if parameter is missing.
-> ALIGNMENT_FLOAT64_IEEE	Declares the alignment of 64 bit floats in the complete module. The alignment is 8 if parameter is missing.
-> ALIGNMENT_INT64	Declares the alignment of int64 in the complete module. The alignment is 8 if parameter is missing.
-> ALIGNMENT_WORD	Declares the alignment of words in the complete module. The alignment is 2 if parameter is missing.
-> BYTE_ORDER	Byte order for the whole device. If this optional parameter is not declared, MSB_LAST (Intel format) is used as a default.
-> DATA_SIZE	Data size in bits
-> DEPOSIT	Standard deposit mode for axis points: ASBOLUTE or DIFFERENCE

Description:

This keyword is used to specify general description data for the module, which are then used as standard in this module. Should other methods be used for an object (e.g. adjustable object or measurement object) of this module, this must then be indicated in the description of the relevant object.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Example:

```
/begin MOD_COMMON       "Characteristic maps always deposited in same
                        mode"
      DEPOSIT           ABSOLUTE
      BYTE_ORDER        MSB_LAST
      DATA_SIZE         16
      ALIGNMENT_BYTE    2
/end MOD_COMMON
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.98 MOD_PAR

Prototype:

```
/begin MOD_PAR      string     Comment
                    [-> ADDR_EPK]*
                    [-> CALIBRATION_METHOD]*
                    [-> CPU_TYPE]
                    [-> CUSTOMER]
                    [-> CUSTOMER_NO]
                    [-> ECU]
                    [-> ECU_CALIBRATION_OFFSET]
                    [-> EPK]
                    [-> MEMORY_LAYOUT]*
                    [-> MEMORY_SEGMENT]*
                    [-> NO_OF_INTERFACES]
                    [-> PHONE_NO]
                    [-> SUPPLIER]
                    [-> SYSTEM_CONSTANT]*
                    [-> USER]
                    [-> VERSION]
/end MOD_PAR
```

Parameters:

string	Comment	comment, description relating to the ECU-specific management data

Optional parameters:

-> ADDR_EPK	Address of EPROM identifier
-> CALIBRATION_METHOD	Declares the implemented calibration methods in the control unit.
-> CPU_TYPE	CPU
-> CUSTOMER	Firm or customer
-> CUSTOMER_NO	Customer number
-> ECU	Control unit
-> ECU_CALIBRATION_OFFSET	Offset that has to be added to each address of a characteristic.
-> EPK	EPROM identifier
-> MEMORY_LAYOUT	Memory layout
-> MEMORY_SEGMENT	Declares the available memory segments.
-> NO_OF_INTERFACES	Number of interfaces
-> PHONE_NO	Phone number of the calibration engineer responsible
-> SUPPLIER	Manufacturer or supplier
-> SYSTEM_CONSTANT	System-defined constants
-> USER	User
-> VERSION	Version identifier

Description:

The MOD_PAR keyword describes the management data to be specified for an device. Except for the comment all parameters are optional.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Example:

```
/begin MOD_PAR           "Note: Provisional release for test purposes
                         only!"
     VERSION             "Test version of 01.02.1994"
     ADDR_EPK            0x45678
     EPK                 EPROM identifier test
     SUPPLIER            "M&K GmbH Chemnitz"
     CUSTOMER            "LANZ-Landmaschinen"
     CUSTOMER_NO         "0123456789"
     USER                "A.N.Wender"
     PHONE_NO            "09951 56456"
     ECU                 "Engine control"
     CPU_TYPE            "Motorola 0815"
     NO_OF_INTERFACES    2
  /begin MEMORY_SEGMENT  ext_Ram
                         "external RAM"
                         DATA
                         RAM
                         EXTERN
                         0x30000
                         0x1000
                         -1 -1 -1 -1 -1
  /begin IF_DATA ASAP1B_KWP2000
  /* ADDRESS_MAPPING     orig_addr    mapping_addr    length */
     ADDRESS_MAPPING     0x4000       0x6000          0x0200
  /end IF_DATA
  /end MEMORY_SEGMENT
  /begin MEMORY_LAYOUT   PRG_RESERVED
                         0x0000
                         0x0400
                         -1 -1 -1 -1 -1
  /end MEMORY_LAYOUT
  /begin MEMORY_LAYOUT   PRG_CODE
                         0x0400
                         0x3C00
                         -1 -1 -1 -1 -1
  /end MEMORY_LAYOUT
  /begin MEMORY_LAYOUT   PRG_DATA
                         0x4000
                         0x5800
                         -1 -1 -1 -1 -1
  /end MEMORY_LAYOUT
  SYSTEM_CONSTANT        "CONTROLLERx constant1" "0.33"
  SYSTEM_CONSTANT        "CONTROLLERx constant2" "2.79"
/end MOD_PAR
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.100 MODEL_LINK

Prototype:

```
MODEL_LINK          string      model_link
```

Parameters:

string	model_link	parameter name in the software model object

Description:

This model_link can be used to reference a software model object the CHARACTERISTIC, AXIS_PTS, BLOB, INSTANCE or MEASUREMENT was derived from.

Note: The model_link does not have to be unique and is not referenced elsewhere in the A2L file. But it is recommended that the model_link is unique in order to avoid confusion in the user interface of the MCD system.

Note: In the model world, the objects may have different names than in the calibration world. To allow a round trip (model, A2L file, data exchange file, model) the model name must be transferred to the calibration world.

The A2L file generation may add the model object name to the A2L file via the MODEL_LINK additionally to the ASAP2 object name. For further process steps the MC tool may transfer the MODEL_LINK to data exchange files. This allows the modeling tools to identify the originally used model object.

Example:

```
MODEL_LINK    "system/sub-system/object.name"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.101 MODULE

Prototype:

```
/begin MODULE       ident      Name
                    string     LongIdentifier
                    [-> A2ML]
                    [-> AXIS_PTS]*
                    [-> BLOB]*
                    [-> CHARACTERISTIC]*
                    [-> COMPU_METHOD]*
                    [-> COMPU_TAB]*
                    [-> COMPU_VTAB]*
                    [-> COMPU_VTAB_RANGE]*
                    [-> FRAME]*
                    [-> FUNCTION]*
                    [-> GROUP]*
                    [-> IF_DATA]*
                    [-> INSTANCE]*
                    [-> MEASUREMENT]*
                    [-> MOD_COMMON]
                    [-> MOD_PAR]
                    [-> RECORD_LAYOUT]*
                    [->TRANSFORMER]*
                    [->TYPEDEF_AXIS]*
                    [->TYPEDEF_BLOB]*
                    [->TYPEDEF_CHARACTERISTIC]*
                    [->TYPEDEF_MEASUREMENT]*
                    [->TYPEDEF_STRUCTURE]*
                    [-> UNIT]*
                    [-> USER_RIGHTS]*
                    [-> VARIANT_CODING]
/end MODULE
```

Parameters:

ident	Name	device identifier
string	LongIdentifier	comment, description

Optional parameters:

-> A2ML	Format description of the interface-specific parameters. Note: The interface-specific parameters must be specified directly after the last mandatory parameter 'long identifier'.
-> AXIS_PTS	Keyword for the description of the axis points
-> BLOB	Description of a blob object
-> CHARACTERISTIC	Keyword for the description of the adjustable objects
-> COMPU_METHOD	Keyword for the description of the conversion method
-> COMPU_TAB	Keyword for the description of the conversion tables
-> COMPU_VTAB	Keyword for the description of the verbal conversion tables
-> COMPU_VTAB_RANGE	Keyword for the description of range-based verbal conversion tables
-> FUNCTION	Keyword for the description of the functions
-> FRAME	Keyword for the declaration of frames

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

-> GROUP	Keyword for the declaration of groups
-> IF_DATA	Data record to describe interface specific data of the device. The parameters associated with this keyword have to be described in the ASAM MCD-2 MC metalanguage.
-> INSTANCE	Description of a measurement or adjustable object created by a type definition
-> MEASUREMENT	Keyword for the description of the measurement objects
-> MOD_COMMON	Module-wide description data
-> MOD_PAR	Keyword for the description of module-specific (device-specific) management data.
-> RECORD_LAYOUT	Keyword for the description of the record layouts
-> TRANSFORMER	Complex calculation of parameter values by an external DLL
-> TYPEDEF_AXIS	Description of a typedef to be used for axes definitions
-> TYPEDEF_BLOB	Description of a typedef to be used for blob definitions
-> TYPEDEF_CHARACTERISTIC	Description of a typedef to be used for characteristic definitions
-> TYPEDEF_MEASUREMENT	Description of a typedef to be used for measurement definitions
-> TYPEDEF_STRUCTURE	Description of a structure typedef
-> UNIT	Keyword for the description of the measurement units
-> USER_RIGHTS	Keyword to reference the groups which constitute access rights.
-> VARIANT_CODING	Keyword to describe the variant coding of adjustable objects.

Description:

The MODULE keyword describes a complete ECU or device with all adjustable and measurement objects, conversion methods and functions. To this, the format description of the interface-specific parameters by the ECU supplier must be added.

Note: It is possible to have a measurement object and a computation method with equal names within the same MODULE. It is NOT possible to have a measurement object and a calibration object with equal names within the same MODULE.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.102 MONOTONY

Prototype:

```
MONOTONY              enum        Monotony
```

Parameters:

enum	Monotony	Description of the monotony:	
		MON_DECREASE	monotonously decreasing
		MON_INCREASE	monotonously increasing
		STRICT_DECREASE	strict monotonously decreasing
		STRICT_INCREASE	strict monotonously increasing
		MONOTONOUS	monotonously in- or decreasing
		STRICT_MON	strict monotonously in- or decreasing
		NOT_MON	no monotony required.

Description:

This keyword can be used to specify the monotony of an adjustment object. The monotony is always related to an axis (see keyword "AXIS_DESCR"). With each adjustment operation the measurement and calibration system (user interface) verifies whether the monotony is guaranteed. Changes that do not correspond to the monotony are not allowed.

Note: Monotony is used in reference to internal values, not physical values.

Note: If the keyword is missing the monotony check is tool dependent.

Example:

```
MONOTONY                MON_INCREASE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.103 NO_AXIS_PTS_X / _Y / _Z / _4 / _5

Prototype:

```
NO_AXIS_PTS_X /_Y /_Z / _4 / _5
                    uint       Position
                    datatype   Datatype
```

Parameters:

uint	Position	Position of the number of axis points in the deposit structure
datatype	Datatype	Data type of the number of axis points

Description:

Description for the axis point count of a curve, map or cube resp. for the dimension of a value block.

Example:

```
NO_AXIS_PTS_X           2
                        UWORD
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.104 NO_OF_INTERFACES

Prototype:

```
NO_OF_INTERFACES    uint        Num
```

Parameters:

uint	Num	Number of interfaces

Description:

Keyword for the number of interfaces

Example:

```
NO_OF_INTERFACES        2
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.105 NO_RESCALE_X

Prototype:

```
NO_RESCALE_X
                    uint        Position
                    datatype    Datatype
```

Parameters:

uint	Position	position of the actual number of rescale axis point value pairs in the deposit structure (description of sequence of elements in the data record).
datatype	DataType	Data type of the number of rescale axis point value pairs

Description:

Actual number of rescaling axis point value pairs.

Example:

```
NO_RESCALE_X            1
                        UBYTE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.106 NUMBER

Prototype:

```
NUMBER              uint        Number
```

Parameters:

uint	Number	Number of available bytes in ECU memory for a CHARACTERISTIC of type ASCII.

Description:

In the CHARACTERISTIC data record, this keyword can be used to specify the number of bytes that are available in ECU memory to store the string when the adjustable object is of type ASCII.

Note: Since strings can have multi-byte encodings such as UTF-8, UTF-16 or UTF-32, Number defines the amount of available memory in bytes and not the number of characters in the string.

Note: Formerly, this keyword was used to specify the size of CHARACTERISTICs of type VAL_BLK. This is obsolete. Use the keyword MATRIX_DIM instead.

Example:

```
NUMBER                          7
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.107 OFFSET_X / _Y / _Z / _4 / _5

Prototype:

```
OFFSET_X /_Y /_Z / _4 / _5
                   uint       Position
                   datatype   Datatype
```

Parameters:

uint	Position	Position of the 'offset' parameter in the deposit structure to compute the axis points for fixed characteristic curves and fixed characteristic maps.
datatype	Datatype	Data type of the 'offset' parameter.

Description:

Description of the 'offset' parameter in the deposit structure to compute the axis points for fixed characteristic curves and fixed characteristic maps (see also keyword FIX_AXIS_PAR). The axis points for fixed characteristic curves or fixed characteristic maps are derived from the two 'offset' and 'shift' parameters as follows:

$$X_i = Offset + (i - 1)*2^{Shift} \quad i = \{ 1...numberofaxispts \}$$

Example:

```
OFFSET_X                16
                        UWORD
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.108 OUT_MEASUREMENT

Prototype:

```
/begin OUT_MEASUREMENT          (ident       Identifier)*
/end OUT_MEASUREMENT
```

Parameters:

ident	Identifier	Identifier of output quantity of respective function (reference to measurement object).

Description:

This keyword can be used to define output quantities of respective function.

Note: OUT_MEASUREMENT may only refer to objects of type MEASUREMENT. This includes objects which are defined via keyword INSTANCE using a TYPEDEF_MEASUREMENT definition as well as instances and instance components of measurement structures.

Example:

```
/begin OUT_MEASUREMENT   OK_FLAG
                         SENSOR_FLAG
/end OUT_MEASUREMENT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.109 OVERWRITE

Prototype:

```
/begin OVERWRITE    ident      Name
                    uint       AxisNumber
                    [-> CONVERSION]
                    [-> EXTENDED_LIMITS]
                    [-> FORMAT]
                    [-> INPUT_QUANTITY]
                    [-> LIMITS]
                    [-> MONOTONY]
                    [-> PHYS_UNIT]
/end OVERWRITE
```

Parameters:

ident	Name	Identifer of the component whose properties are to be overruled. For structure instances, this identifier shall be the name of the corresponding structure component built according to rules described at INSTANCE. For instances of simple types without components or for instance arrays, the keyword THIS must be set for the name.
uint	AxisNumber	If the object to overrule is the axis of a curve or map, the axis number must be set (numbering starts with 1 for the x axis), otherwise 0 must be set.

Optional parameters

-> CONVERSION	Overrules the conversion method of the referenced type definition for this instance.
-> EXTENDED_LIMITS	Overrules the extended limits of the referenced type definition for this instance. This parameter can only be used if the referenced type definition describes an axis or a characteristic.
-> FORMAT	Overrules the format string of the referenced type definition for this instance.
-> INPUT_QUANTITY	Overrules the input quantity of the referenced type definition for this instance. This parameter can only be used if the referenced type definition describes an axis.
-> LIMITS	Overrules the limits of the referenced type definition for this instance.
-> MONOTONY	Overrules the monotony of the referenced type definition for this instance. This parameter can only be used if the referenced type definition describes an axis.
-> PHYS_UNIT	Overrules the physical unit of the referenced type definition for this instance.

Description:

This keyword can be used at INSTANCE to overwrite the default properties specified in the referenced type definition.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Example 1:

```
/begin TYPEDEF_AXIS
  AxisTemplate                  // type name
  ""                            // description
  NO_INPUT_QUANTITY             // input quantity not specified
  RL_AXIS                       // record layout
  100.0                         // maxdiff
  NO_COMPU_METHOD               // no conversion
  10                            // max number of axis points
  0                             // lower limit
  25                            // upper limit
/end TYPEDEF_AXIS

/begin INSTANCE
  Instance1                     // instance name
  ""                            // instance description
  AxisTemplate                  // reference to type definition
  0x1200                        // start address of instance
  /begin OVERWRITE
        THIS 0                  // overruling the object itself
        INPUT_QUANTITY Speed    // overruling the input quantity
        COMPU_METHOD Factor200 // overruling the conversion method
  /end OVERWRITE
/end INSTANCE
```

Example 2:

```
/begin TYPEDEF_CHARACTERISTIC
  MapTemplate                   // type name
  "…"                           // description
  MAP
  RL_MAP                        // record layout
  100.0                         // maxdiff
  NO_COMPU_METHOD               // no conversion
  0                             // lower limit
  25                            // upper limit
  /begin AXIS_DESCR
        STD_AXIS
        NO_INPUT_QUANTITY
        NO_COMPU_METHOD
        10
        0  30
  /end AXIS_DESCR
  /begin AXIS_DESCR
        STD_AXIS
        NO_INPUT_QUANTITY
        NO_COMPU_METHOD
        10
        0  30
  /end AXIS_DESCR
/end TYPEDEF_CHARACTERISTIC

/begin TYPEDEF_STRUCTURE
  MyStructure                   // structure name
  "…"                           // description
  64                            // total size of structure
  /begin STRUCTURE_COMPONENT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
        MyMap MapTemplate
        0                       // address offset
    /end STRUCTURE_COMPONENT
  /end TYPEDEF_STRUCTURE

  /begin INSTANCE
    MyInstance                  // instance name
    ""                          // instance description
    MyStructure                 // reference to type definition
    0x1200                      // start address of instance

    /begin OVERWRITE
        MyMap 1                 // overruling the X axis of
                                // the component MyMap
        INPUT_QUANTITY Speed    // overruling the input quantity
    /end OVERWRITE

    /begin OVERWRITE
        MyMap 2                 // overruling the Y axis of
                                // the component MyMap
        CONVERSION SpeedConv    // overruling the conversion method
    /end OVERWRITE

    /begin OVERWRITE
        MyMap 0                 // overruling the map object
        CONVERSION Conv2        // overruling the conversion method
    /end OVERWRITE

  /end INSTANCE
```

Example 3:

```
  /begin INSTANCE
    MyInstanceArray             // instance name
    ""                          // instance description
    MyStructure                 // reference to type definition
    0x1200                      // start address of instance
    MATRIX_DIM 3
    /begin OVERWRITE
        THIS[0].MyMap 1         // overruling the X axis of
                                // the component MyMap of the
                                // first array element
        INPUT_QUANTITY Speed1   // overruling the input quantity
    /end OVERWRITE
    /begin OVERWRITE
        THIS[1].MyMap 2         // overruling the Y axis of
                                // the component MyMap of the
                                // second array element
        INPUT_QUANTITY Speed2   // overruling the input quantity
    /end OVERWRITE
  /end INSTANCE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.110 PHONE_NO

Prototype:

```
PHONE_NO            string      Telnum
```

Parameters:

string	Telnum	phone number

Description:

This keyword is used to specify a phone number, e.g. of the calibration engineer responsible.

Example:

```
PHONE_NO                "09498 594562"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.111 PHYS_UNIT

Prototype:

```
PHYS_UNIT           string     Unit
```

Parameters:

string	Unit	Physical unit.

Description:

With this keyword a physical unit can be specified for a measure or calibration object if no conversion rule is used (NO_COMPU_METHOD). If a conversion rule is used, the additional usage of PHYS_UNIT overrules the default unit specified at the referenced conversion rule. The keyword can be used at AXIS_PTS, AXIS_DESCR, CHARACTERISTIC and MEASUREMENT, as well as at the corresponding type definitions.

Example:

```
PHYS_UNIT           "°C"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.112 PROJECT

Prototype:

```
/begin PROJECT      ident      Name
                    string     LongIdentifier
                    [-> HEADER]
                    [-> MODULE]*
/end PROJECT
```

Parameters:

ident	Name	Project identifier in the program
string	LongIdentifier	Comment, description

Optional parameters:

-> HEADER	Project header
-> MODULE	This keyword is used to describe the devices belonging to the project.

Description:

Project description with project header and all devices belonging to the project. The PROJECT keyword covers the description of several control units, and possibly also of several suppliers.

Example:

```
/begin PROJECT          RAPE-SEED ENGINE
                        "Engine tuning for operation with rape oil"
  /begin HEADER         "see also specification XYZ of 01.02.1994"
     VERSION            "BG5.0815"
     PROJECT_NO         M4711Z1
  /end HEADER

  /include ENGINE_ECU.A2L    /* Include for engine control module */
  /include ABS_ECU.A2L       /* Include for ABS module */
/end PROJECT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.113 PROJECT_NO

Prototype:

```
PROJECT_NO          ident       ProjectNumber
```

Parameters:

ident	ProjectNumber	Short identifier of the project number

Description:

String used to identify the project number with maximum MAX_IDENT characters.

Example:

```
PROJECT_NO              M4711Z1
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.114 READ_ONLY

Prototype:

```
READ_ONLY
```

Description:

This keyword is used to indicate that an adjustable object cannot be changed (but can only be read).

Example:

```
/begin CHARACTERISTIC    KI "I-share for speed limitation"
                         VALUE          /* type: fixed value */
                         0x408F         /* address */
                         DAMOS_FW       /* deposit */
                         0.0            /* max_diff */
                         FACTOR01       /* conversion */
                         0.0            /* lower limit */
                         255.0          /* upper limit */

  /* interface-specific parameters: address location, addressing */
  /begin IF_DATA "DIM"   EXTERNAL
                         DIRECT
  /end IF_DATA
  /begin FUNCTION_LIST   V_LIM          /* Reference to functions */
  /end FUNCTION_LIST
  READ_ONLY
/end CHARACTERISTIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.115 READ_WRITE

Prototype:

```
READ_WRITE
```

Description:

This keyword is used to mark a measurement object to be writeable.

Example:

```
/begin MEASUREMENT      N                /* name */
                        "Engine speed"   /* long identifier */
                        UWORD            /* datatype */
                        R_SPEED_3        /* conversion */
                        1                /* resolution */
                        0                /* accuracy */
                        120.0            /* lower limit */
                        8400.0           /* upper limit */
  READ_WRITE
  /begin IF_DATA ISO    SND
                        0x10
                        0x00
                        0x05
                        0x08
                        RCV
                        4
                        long
  /end IF_DATA
/end MEASUREMENT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.116 RECORD_LAYOUT

Prototype:

```
/begin RECORD_LAYOUT  ident    Name
                     [-> ALIGNMENT_BYTE]
                     [-> ALIGNMENT_FLOAT32_IEEE]
                     [-> ALIGNMENT_FLOAT64_IEEE]
                     [-> ALIGNMENT_INT64]
                     [-> ALIGNMENT_LONG]
                     [-> ALIGNMENT_WORD]
                     [-> AXIS_PTS_X/_Y/_Z/_4/_5]
                     [-> AXIS_RESCALE_X]
                     [-> DIST_OP_X/_Y/_Z/_4/_5]
                     [-> FIX_NO_AXIS_PTS_X/_Y/_Z/_4/_5]
                     [-> FNC_VALUES]
                     [-> IDENTIFICATION]
                     [-> NO_AXIS_PTS_X/_Y/_Z/_4/_5]
                     [-> NO_RESCALE_X]
                     [-> OFFSET_X/_Y/_Z/_4/_5]
                     [-> RESERVED]*
                     [-> RIP_ADDR_W/_X/_Y/_Z/_4/_5]
                     [-> SRC_ADDR_X/_Y/_Z/_4/_5]
                     [-> SHIFT_OP_X/_Y/_Z/_4/_5]
                     [-> STATIC_ADDRESS_OFFSETS]
                     [-> STATIC_RECORD_LAYOUT]
/end RECORD_LAYOUT
```

Parameters:

ident Name	Identification of the record layout, which is referenced via this 'name'. Note: The name of the record layout has to be unique within all record layouts of the ASAM MCD-2 MC MODULE, i.e. there must not be another RECORD_LAYOUT object with the same identifier in the MODULE.

Optional parameters:

-> ALIGNMENT_BYTE	Declares the alignment of bytes for all characteristics which use this record layout. If the keyword is missing, the alignment defined at MOD_COMMON is used.
-> ALIGNMENT_FLOAT32_IEEE	Declares the alignment of 32 bit floats for all characteristics which use this record layout. If the keyword is missing, the alignment defined at MOD_COMMON is used.
-> ALIGNMENT_FLOAT64_IEEE	Declares the alignment of 64 bit floats for all characteristics which use this record layout. If the keyword is missing, the alignment defined at MOD_COMMON is used.
-> ALIGNMENT_INT64	Declares the alignment of int64 for all characteristics which use this record layout. If the keyword is

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

missing, the alignment defined at MOD_COMMON is used.

-> ALIGNMENT_LONG
Declares the alignment of longs for all characteristics which use this record layout.
If the keyword is missing, the alignment defined at MOD_COMMON is used.

-> ALIGNMENT_WORD
Declares the alignment of words for all characteristics which use this record layout. . If the keyword is missing, the alignment defined at MOD_COMMON is used.

-> AXIS_PTS_X / _Y / _Z / _4 / _5
Describes where the X, Y, Z, Z4 or Z5 axis points are deposited in memory.

-> AXIS_RESCALE_X
Describes where the rescale mapping for the X axis is deposited in memory.

-> DIST_OP_X / _Y / _Z / _4 / _5
'Distance' parameter to compute the axis points of fixed Characteristics.

-> FIX_NO_AXIS_PTS_X / _ Y / _Z / _4 / _5
Indicates that a fixed number of axis points is allocated.

Note: In a RECORD_LAYOUT data record, this keyword may not be used simultaneously with the keyword 'NO_AXIS_PTS_X / _Y / _Z / _4 / _5.

-> FNC_VALUES
This keyword describes how the table values (function values) of the adjustable object are deposited in memory.

-> IDENTIFICATION
This keyword is used to describe that an 'identifier' is deposited in a specific position in the adjustable object.

-> NO_AXIS_PTS_X / _Y / _Z / _4 / _5
Describes at which position the current X/Y/Z/4/5-dimension of the adjustable object is deposited in memory.

Note: This parameter is intended to be used for record layouts describing objects with dynamic dimension. For curves, maps and cubes, it describes the current number of axis points and the corresponding value must not exceed the maximum number of axis points defined at the axis description. For value blocks, it represents its current dimension and the corresponding value must not exceed the maximum dimension defined via keyword MATRIX_DIM at the value block.

-> NO_RESCALE_X
Describes at which position the parameter ‘current number of rescale pairs’ for the axis is deposited (see AXIS_RESCALE_X).

-> OFFSET_X / _Y / _Z / _4 / _5
Offset to compute the axis points of fixed Characteristics.

-> RESERVED
This keyword can be used to skip specific elements in the adjustable object whose meaning must not be

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

	interpreted by the measurement and calibration system (e.g. for extensions: new parameters in the adjustable objects).
-> RIP_ADDR_W	Final result (table value) of the ECU-internal interpolation. (output value)
-> RIP_ADDR_X / _Y / _Z / _4 / _5	Is used to describe at which position the address of this RIP_X, RIP_Y, RIP_Z, RIP_Z4 or RIP_Z5 quantity is deposited, which contains the current value of the ECU-internal interpolation. (input values)
-> SHIFT_OP_X / _Y / _Z / _4 / _5	Shift operand to compute the axis points of fixed Characteristics.
-> SRC_ADDR_X / _Y / _Z / _4 / _5	Describes at which position the address of the input quantity of the axis is deposited in memory.
-> STATIC_ ADDRESS_OFFSETS	For calibration objects with dynamic number of axis points this keyword indicates that the start addresses of axes and function values do not change when removing or inserting axis points, however the addresses of the single data cells will change if the dimension of the map changes. See STATIC_ADDRESS_OFFSETS.
-> STATIC_RECORD_LAYOUT	For calibration objects with dynamic number of axis points this keyword indicates that the calibration object **does not** compact or expand data when removing resp. inserting axis points. All record layout elements are stored at the same address as for the max. number of axis points specified at the calibration object - independent of the actual number of axis points. If the parameter STATIC_RECORD_LAYOUT is missing, the calibration objects referencing this record layout **do** compact / extend data when removing resp. inserting axis points and the addresses of the record layout elements depend on the actual number of axis points.

Description:

The 'RECORD_LAYOUT' keyword is used to specify the various data structures of the adjustable objects in the memory. The structural buildup of the various adjustable object types must be described in such a way that a standard measurement and calibration system will be able to process all adjustable object types (reading, writing, operating point display etc.).
In particular, if the ALTERNATE option is used with FNC_VALUES, the position parameter determines the order of values and axis points.

Note: To describe the record layouts, use is made of a predefined list of parameters which may be part of an adjustable object (characteristic) in the emulation memory. This list represents the current status of the record layouts. With each change or extension of the record layouts contained in this predefined list of parameters the ASAM MCD-2 MC description file format must be modified accordingly.

Note: The keywords describing axis parameters for CUBE_4 and CUBE_5 are extended by _X, _Y, _Z, _4, _5. This allows an easier understanding which dimension the axis description belongs to. In the textual description the axes for

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

CUBE_4 and CUBE_5 are named X, Y, Z, Z4, Z5. This allows textual description without a reference to a keyword (Z4-axis describes the axis better as 4-axis)

Note: For CUBOID, CUBE_4 and CUBE_5 the RECORD_LAYOUT supports only one dedicated way to sort the data in the memory. These objects are always stored as array of MAP with incremented or decremented axes. The exchange of dimensions (e.g. X, Z, Y, Z5, Z4) in the memory is not supported.

Note: If no STATIC_RECORD_LAYOUT is defined and a dynamic number of axis points is used, the number of axis points (NO_AXIS_PTS_?) has to be located in the ECU memory before the axis points (AXIS_PTS_?) and the function values (FNC_VALUES).

Note: The position numbers inside a record layout are used in ascending order. This means no gaps are included and no double usage of position numbers is allowed. The start number itself is not defined. For the position the following sub keywords are relevant:

- AXIS_PTS_X / _Y / _Z / _4 / _5
- AXIS_RESCALE_X
- DIST_OP_X / _Y / _Z / _4 / _5
- FNC_VALUES
- IDENTIFICATION
- NO_AXIS_PTS_X / _Y / _Z / _4 / _5
- NO_RESCALE_X
- OFFSET_X / _Y / _Z / _4 / _5
- RESERVED
- RIP_ADDR_W / _X / _Y / _Z / _4 / _5
- SHIFT_OP_X / _Y / _Z / _4 / _5
- SRC_ADDR_X / _Y / _Z / _4 / _5

Note: When describing common axis, use the keyword AXIS_PTS_X/_Y/_Z/_4/_5 at the corresponding RECORD_LAYOUT. Do not use AXIS_RESCALE_X for this.

Example:

```
/begin RECORD_LAYOUT     DAMOS_KF
      FNC_VALUES         7 SWORD COLUMN_DIR DIRECT
      AXIS_PTS_X         3 SWORD INDEX_INCR DIRECT
      AXIS_PTS_Y         6 UBYTE INDEX_INCR DIRECT
      NO_AXIS_PTS_X      2 UBYTE
      NO_AXIS_PTS_Y      5 UBYTE
      SRC_ADDR_X         1
      SRC_ADDR_Y         4
      ALIGNMENT_BYTE     2
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     RESCALE_SST
      NO_RESCALE_X       1 UBYTE
      RESERVED           2 BYTE
      AXIS_RESCALE_X     3 UBYTE 5 INDEX_INCR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     SHORTINT
      FNC_VALUES         1 SBYTE ROW_DIR DIRECT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     BYTE
      FNC_VALUES         1 UBYTE ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     INTEGER
      FNC_VALUES         1 SWORD ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     WORD
      FNC_VALUES         1 UWORD ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     LONGINT
      FNC_VALUES         1 SLONG ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     LONGWORD
      FNC_VALUES         1 ULONG ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     2D_structure_table_int
      NO_AXIS_PTS_X      1 UWORD
      FNC_VALUES         2 SWORD ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     2D_structure_table_word
      NO_AXIS_PTS_X      1 UWORD
      FNC_VALUES         2 UWORD ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     2D_structure_table_byte
      NO_AXIS_PTS_X      1 UBYTE
      RESERVED           2 BYTE
      FNC_VALUES         3 UBYTE ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     2D_structure_table_shortint
      NO_AXIS_PTS_X      1 UBYTE
      RESERVED           2 BYTE
      FNC_VALUES         3 SBYTE ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     3D_structure_table_int
      NO_AXIS_PTS_X      1 UWORD
      NO_AXIS_PTS_Y      2 UWORD
      FNC_VALUES         3 SWORD ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     3D_structure_table_word
      NO_AXIS_PTS_X      1 UWORD
      NO_AXIS_PTS_Y      2 UWORD
      FNC_VALUES         3 UWORD ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     3D_structure_table_byte
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
      NO_AXIS_PTS_X      1 UBYTE
      NO_AXIS_PTS_Y      2 UBYTE
      RESERVED           3 BYTE
      FNC_VALUES         4 UBYTE ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     3D_structure_table_shortint
      NO_AXIS_PTS_X      1 UBYTE
      NO_AXIS_PTS_X      2 UBYTE
      RESERVED           3 BYTE
      FNC_VALUES         4 SBYTE ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     2D_array_table_int
      FNC_VALUES         1 SWORD ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     2D_array_table_word
      FNC_VALUES         1 UWORD ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     2D_array_table_byte
      FNC_VALUES         1 UBYTE ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     2D_array_table_shortint
      FNC_VALUES         1 SBYTE ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     3D_array_table_int
      FNC_VALUES         1 SWORD ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     3D_array_table_word
      FNC_VALUES         1 UWORD ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     3D_array_table_byte
      FNC_VALUES         1 UBYTE ROW_DIR DIRECT
/end RECORD_LAYOUT

/begin RECORD_LAYOUT     3D_array_table_shortint
      FNC_VALUES         1 SBYTE ROW_DIR DIRECT
/end RECORD_LAYOUT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.117 REF_CHARACTERISTIC

Prototype:

```
/begin REF_CHARACTERISTIC
                    (ident     Identifier)*
/end REF_CHARACTERISTIC
```

Parameters:

ident	Identifier	Identifier of those objects that are referred to the respective function or group.

Description:

This keyword can be used to assign adjustable objects to the respective function or group.

Note: REF_CHARACTERISTIC may only refer to adjustable objects, i.e. objects of type CHARACTERISTIC, AXIS_PTS and BLOB and objects which are defined via keyword INSTANCE using a corresponding type definition. This also includes instances and instance components of adjustable structures.

Example:

```
/begin REF_CHARACTERISTIC    ENG_SPEED_CORR_CURVE
/end REF_CHARACTERISTIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.118 REF_GROUP

Prototype:

```
/begin REF_GROUP    (ident     Identifier)*
/end REF_GROUP
```

Parameters:

ident	Identifier	Identifier of those groups which are referred in USER_RIGHTS

Description:

This keyword can be used to refer groups which control the access rights of users logging into an MCD system.

Example:

```
/begin REF_GROUP          GROUP_1
                          GROUP_2
/end REF_GROUP
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.119 REF_MEASUREMENT

Prototype:

```
/begin REF_MEASUREMENT          (ident       Identifier)*
/end REF_MEASUREMENT
```

Parameters:

ident	Identifier	Identifier of those measure objects which are assigned to the group.

Description:

This keyword can be used to define measure objects which are assigned to the respective group.

Note: REF_MEASUREMENT may only refer to objects of type MEASUREMENT. This includes objects which are defined via keyword INSTANCE using a TYPEDEF_MEASUREMENT definition as well as instances and instance components of measurement structures.

Example:

```
/begin REF_MEASUREMENT   LOOP_COUNTER
                         TEMPORARY_1
/end REF_MEASUREMENT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.120 REF_MEMORY_SEGMENT /DEPRECATED

Prototype:

```
REF_MEMORY_SEGMENT  ident      Name
```

Parameters:

ident	Name	Name of memory segments

Description:

The reference to a memory segment is used in characteristics and measurements.

Note: This keyword is deprecated and only available for downward compatibility.

Example:

```
REF_MEMORY_SEGMENT      Data1
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.121 REF_UNIT

Prototype:

```
REF_UNIT            ident      Unit
```

Parameters:

ident	Unit	reference to the data record which describes a measurement unit

Description:

This keyword can be used to reference to the data record which describes a measurement unit.
REF_UNIT may only refer to objects of type UNIT.

Note: The string parameter Unit of a COMPU_METHOD is a redundant information because the record referenced by REF_UNIT contain it too. Just for the purpose of compatibility with previous versions of ASAM MCD-2 MC the parameter REF_UNIT is optional.

Example:

```
/begin COMPU_METHOD   Velocity
                      "conversion method for velocity"
                      RAT_FUNC
                      "%6.2"
                      "[km/h]"
             COEFFS   0 100 0 0 0 1
           REF_UNIT   kms_per_hour   /* new (optional) parameter */
/end COMPU_METHOD
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.122 RESERVED

Prototype:

```
RESERVED            uint       Position
                    datasize   DataSize
```

Parameters:

uint	Position	Position of the reserved parameter in the deposit structure
datasize	DataSize	Word length of the reserved parameter.

Description:

This keyword can be used to skip specific elements in an adjustable object whose meaning must not be interpreted by the measurement and calibration system (e.g. for extensions: new parameters in the adjustable objects).

Example:

```
RESERVED                7
                        LONG
```

Note: Only BYTE, WORD and LONG are valid datasize values. A datatype (UBYTE, SBYTE, UWORD, etc.) cannot be used in place of the datasize.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.123 RIGHT_SHIFT

Prototype:

```
RIGHT_SHIFT        ulong      Bitcount
```

Parameters:

ulong	Bitcount	Shift ‚Bitcount‘ bits to the right

Description:

The RIGHT_SHIFT keyword is only used within the BIT_OPERATION keyword. See description of BIT_OPERATION.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.124 RIP_ADDR_W / _X / _Y / _Z / _4 / _5

Prototype:

```
RIP_ADDR_W / _X / _Y / _Z / _4 / _5
                   uint       Position
                   datatype   Datatype
```

Parameters:

uint	Position	Position of the address to the result of the ECU-internal interpolation (see below) in the deposit structure.
datatype	Datatype	Data type of the address

Description:

When the ECU program accesses a characteristic curve it determines an output value based on an input quantity. First it searches the adjacent axis points of the current value of the input quantities (Xi, Xi+1 or Yi, Yi+1 or Zi, Zi+1 or Z4i, Z4i+1 or Z5i, Z5i+1). The output value is derived from these axis points and the allocated table values by means of interpolation. This produces an 'intermediate result' known as the RIP_X / _Y / _Z / _4 / _5 quantity (Result of Interpolation), which describes the relative distance between the current value and the adjacent axis points (see Figure 7). The output value is derived from these axis points and the two allocated table values by means of interpolation. This produces as intermediate results the quantities RIP_X and RIP_Y, which describe the distance between the current value and the adjacent axis points:

$$RIP_X = (X_{current} - X_i)/(X_{i+1} - X_i)$$

For a characteristic map the ECU program determines this intermediate result both in the X-direction and in the Y-direction. For a characteristic cuboid the result in the direction of all three axes are calculated.

$$RIP_Y = (Y_{current} - Y_k)/(Y_{k+1} - Y_k)$$

$$RIP_Z = (Z_{current} - Z_m)/(Z_{m+1} - Z_m)$$

For a characteristic curve the result of the interpolation is calculated as follows:

$$RIP_W = W_i + (RIP_X * (W_{i+1} - W_i)$$

for a characteristic map as follows:

$$RIP_W = (W_{i,k} * (1 - RIP_X) + W_{i+1,k} * RIP_X)) * (1 - RIP_Y) +$$
$$(W_{i,k+1} * (1 - RIP_X) + W_{i+1,k+1} * RIP_X)) * RIP_Y$$

and for a characteristic cuboid as follows:

Interpolation for the map Z = m

$$RIP_W_m = (W_{i,k,m} * (1 - RIP_X) + W_{i+1,k,m} * RIP_X)) * (1 - RIP_Y) +$$
$$(W_{i,k+1,m} * (1 - RIP_X) + W_{i+1,k+1,m} * RIP_X)) * RIP_Y$$

Interpolation for the map Z = m+1

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

$RIP_W_{m+1} = (W_{i,k,m+1} * (1 - RIP_X) + W_{i+1,k,m+1} * RIP_X)) * (1 - RIP_Y) +$

$(W_{i,k+1,m+1} * (1 - RIP_X) + W_{i+1,k+1,m+1} * RIP_X)) * RIP_Y$

Interpolation in Z direction between the two points RIP_W_m and RIP_W_{m+1}.

$RIP_W = RIP_W_m + (RIP_Z * (RIP_W_{m+1} - RIP_W_m)$

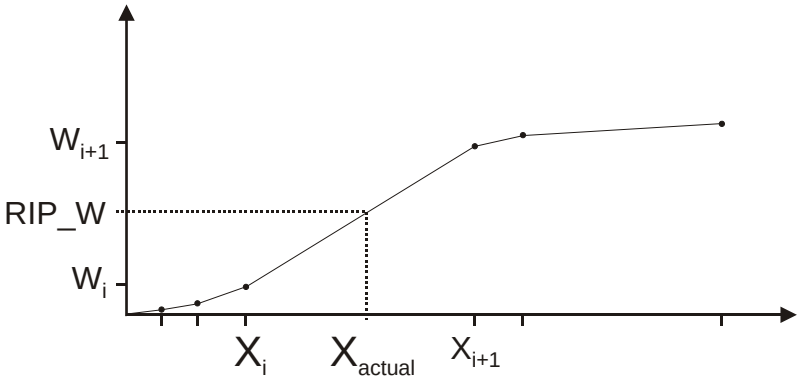


Figure 7 Linear interpolation for a characteristic curve

Example:

```
RIP_ADDR_X              19
                        UWORD
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.125 ROOT

Prototype:

```
ROOT
```

Parameters:

none

Description:

This keyword ROOT indicates that the related group is presented as a root of a navigation tree in the group selection mechanism of the MCD system. The keyword ROOT can indicate that groups referred to this root group constitute a grouping mechanism.

Example:

```
/begin GROUP              SOFTWARE_COMPONENTS
                          "assignment of the definitions to C files"
  ROOT
  /begin SUB_GROUP        INJE
                          C6TD
  /end SUB_GROUP
/end GROUP
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.126 SHIFT_OP_X / _Y / _Z / _4 / _5

Prototype:

```
SHIFT_OP_X / _Y / _Z / _4 / _5
                    uint        Position
                    datatype    Datatype
```

Parameters:

uint	Position	Position of the shift operand in the deposit structure.
datatype	Datatype	Data type of the shift operand.

Description:

Description of the shift operand in the deposit structure to compute the axis points for fixed characteristic curves and fixed characteristic maps (see also keyword FIX_AXIS_PAR). The axis points distribution for fixed characteristic curves or fixed characteristic maps is derived from the two 'offset' and 'shift' parameters as follows:

$$X_i = Offset + (i - 1)*2^{Shift} \quad i = \{ 1...numberofaxispts \}$$

Example:

```
SHIFT_OP_X              21
                        UWORD
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.127 SIGN_EXTEND

Prototype:

```
SIGN_EXTEND
```

Parameters:

none

Description:

The SIGN_EXTEND keyword is only used within the BIT_OPERATION keyword. See description of BIT_OPERATION.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.128 SI_EXPONENTS

Prototype:

```
SI_EXPONENTS        int         Length
                    int         Mass
                    int         Time
                    int         ElectricCurrent
                    int         Temperature
                    int         AmountOfSubstance
                    int         LuminousIntensity
```

Parameters:

int	Length	exponent of the base dimension length with unit metre
int	Mass	exponent of the base dimension mass with unit kilogram
int	Time	exponent of the base dimension time with unit second
int	ElectricCurrent	exponent of the base dimension electric current with unit ampere
int	Temperature	exponent of the base dimension thermodynamic temperature with unit kelvin
int	AmountOfSubstance	exponent of the base dimension amount of substance with unit mole
int	LuminousIntensity	exponent of the base dimension luminous intensity with unit candela

Description:

Specification of the seven base dimensions required to define an extended SI unit.

Example:

```
/begin UNIT
                        newton
                        "extended SI unit for force"
                        "[N]"
                        EXTENDED_SI
        SI_EXPONENTS    1 1 -2 0 0 0 0     /*[N] = [m]*[kg]*[s]^-2 */
/end UNIT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.129 SRC_ADDR_X / _Y / _Z / _4 / _5

Prototype:

```
SRC_ADDR_X / _Y / _Z / _4 / _5
                   uint       Position
                   datatype   Datatype
```

Parameters:

uint	Position	Position of the address of the input quantity in the deposit structure.
datatype	Datatype	Data type of the address.

Description:

Description of the address of the input quantity in an adjustable object

Example:

```
SRC_ADDR_X              1
                        UWORD
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.130 STATIC_ADDRESS_OFFSETS

Prototype:

```
STATIC_ ADDRESS_OFFSETS
```

Parameters:

none

Description:

This keyword is used to indicate that the start addresses of axes and function values of an adjustable object do not change when removing or inserting axis points. All record layout elements are stored at the same address offsets as for the max. number of axis points specified at the calibration object - independent of the current number of axis points.
In difference to STATIC_RECORD_LAYOUT the FNC_VALUES are handled dynamically. I.e., the addresses of the single data cells change if the dimension of the map changes.

Note: The interpolation routine `GetMapValue(map, xIndex, yIndex)` (See Example 1 of STATIC_RECORD_LAYOUT) may have no access to the structure type information. In this case the FNC_VALUES are to be handled dynamic to allow `GetMapValue()` to calculate the addresses of the single cells correct.

Note: It is not allowed to use STATIC_ADDRESS_OFFSETS and STATIC_RECORD_LAYOUT at the same RECORD_LAYOUT.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.131 STATIC_RECORD_LAYOUT

Prototype:

```
STATIC_RECORD_LAYOUT
```

Description:

This keyword is used to indicate that an adjustable object with dynamic number of axis points **does not** compact or expand data when removing resp. inserting axis points. All record layout elements are stored at the same address as for the max. number of axis points specified at the calibration object - independent of the current number of axis points The FNC_VALUES are handled as static as well. I.e., the addresses of the single data cells do not change if the dimension of the map changes.

Example 1 (requires STATIC_RECORD_LAYOUT):

```
struct {
   unsigned short noX;      /* number of x axis coordinates */
   unsigned short noY;      /* number of y axis coordinates */
   unsigned char xAxis[10]; /* MaxAxisPoints = 10 */
   unsigned char yAxis[5];  /* MaxAxisPoints = 5 */
   unsigned char values[10][5];
} mapBeingNotCompact;

#define GetXCoordinate(map, index) (map.xAxis[index])
#define GetYCoordinate(map, index) (map.yAxis[index])
#define GetMapValue(map, xIndex, yIndex) (map.value[xIndex][yIndex])
```

This example leads to the following description:

```
/begin CHARACTERISTIC
   mapBeingNotCompact "This curve compacts data"
   MAP
   0x0815
   mapLayoutNotCompact
   0.0
   NO_COMPU_METHOD
   0.0
   255.0
   /begin AXIS_DESCR
      STD_AXIS
      NO_INPUT_QUANTITY
      NO_COMPU_METHOD
      10
      0 200
   /end AXIS_DESCR
   /begin AXIS_DESCR
      STD_AXIS
      NO_INPUT_QUANTITY
      NO_COMPU_METHOD
      5
      0 200
   /end AXIS_DESCR
/end CHARACTERISTIC

/begin RECORD_LAYOUT
   mapLayoutNotCompact
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
   NO_AXIS_PTS_X 1 UWORD
   NO_AXIS_PTS_Y 2 UWORD
   AXIS_PTS_X 3 UBYTE INDEX_INR DIRECT
   AXIS_PTS_Y 4 UBYTE INDEX_INR DIRECT
   FNC_VALUES 5 UBYTE ROW_DIR DIRECT
   STATIC_RECORD_LAYOUT
/end RECORD_LAYOUT
```

Example 2 (does NOT require STATIC_RECORD_LAYOUT):

```
stuct mapBeingCompact_t {
   unsigned short noX;          /* number of x axis coordinates */
   unsigned short noY;          /* number of y axis coordinates */
   unsigned char values[65];    /* 10 x points, 5 y points  */
                                /* 10x5 map values */
} mapBeingCompact;

#define GetXCoordinate(map, index) (map.values[index])
#define GetYCoordinate(map, index) (map.values[map.noX + index])
#define GetMapValue(map, xIndex, yIndex) \
    (map.values[map.noX + map.noY + xIndex + (yIndex * map.noX)])
```

This example leads to the following description:

```
/begin CHARACTERISTIC
   mapBeingCompact "This curve doesn't compact data"
   MAP
   0x0815
   mapLayoutCompact
   0.0
   NO_COMPU_METHOD
   0.0
   255.0
   /begin AXIS_DESCR
      STD_AXIS
      NO_INPUT_QUANTITY
      NO_COMPU_METHOD
      10
      0 200
   /end AXIS_DESCR
   /begin AXIS_DESCR
      STD_AXIS
      NO_INPUT_QUANTITY
      NO_COMPU_METHOD
      5
      0 200
   /end AXIS_DESCR
/end CHARACTERISTIC

/begin RECORD_LAYOUT
   mapLayoutCompact
   NO_AXIS_PTS_X 1 UWORD
   NO_AXIS_PTS_Y 2 UWORD
   AXIS_PTS_X 3 UBYTE INDEX_INR DIRECT
   AXIS_PTS_Y 4 UBYTE INDEX_INR DIRECT
   FNC_VALUES 5 UBYTE ROW_DIR DIRECT
/end RECORD_LAYOUT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.132 STATUS_STRING_REF

Prototype:

```
STATUS_STRING_REF   ident      ConversionTable
```

Parameters:

ident ConversionTable	Reference to a verbal conversion table (COMPU_VTAB or COMPU_VTAB_RANGE)

Description:

This keyword is used to split up the value range of ECU internal values into a numerical and a verbal part. The verbal part can be used to visualize status information (e.g. “Sensor not connected”). It is used at COMPU_METHOD.

The conversion table referenced by the keyword STATUS_STRING_REF must not define a default value.

Note: The MC tool at first checks whether the internal value is in the range defined at the STATUS_STRING_REF conversion table. In this case, the tool displays the corresponding text. Otherwise, the MC tool uses the regular computation method.
The MC tool must not respect the limits when evaluating the STATUS_STRING_REF.

Note: PHYS Values defined by STATUS_STRING_REF are not selectable for calibration. To ensure this, the values referenced by STATUS_STRING_REF must be outside the ECU internal limits of all calibration objects (CHARACTERISTIC, AXIS_PTS) using the corresponding conversion method.

Example:

```
/begin COMPU_METHOD      CM_LINFUNC_SENSOR_A      /* name */
                         "conversion method for Sensor A"
                         LINEAR           /* convers_type */
                         "%4.0"           /* display format */
                         "rpm"            /* physical unit */
                         COEFFS_LINEAR  2.0 5.0
                         STATUS_STRING_REF CT_SensorStatus
/end COMPU_METHOD

/begin COMPU_VTAB        CT_SensorStatus
                         ""
                         TAB_VERB         /* convers_type */
                         2                /* number_value_pairs */
                         0x00 "Sensor not Connected"
                         0xFF "Sensor defect"
/end COMPU_VTAB
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.133 STEP_SIZE

Prototype:

```
STEP_SIZE           float      StepSize
```

Parameters:

float	StepSize	delta

Note: The values are to be interpreted as physical values.

Description:

This keyword can be used to define a value which is added to or subtracted from a CHARACTERISTIC, AXIS_PTS or AXIS_DESCR data value when using up/down keys while calibrating.

Example:

```
STEP_SIZE               0.025
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.134 STRUCTURE_COMPONENT

Prototype:

```
/begin STRUCTURE_COMPONENT
                    ident      Name
                    ident      TypedefName
                    ulong      AddressOffset
                    [-> LAYOUT]
                    [-> MATRIX_DIM]
/end STRUCTURE_COMPONENT
```

Parameters:

ident	Name	Identifier of the component. This identifier can be used to build the full name of structure components. It has to be unique within the referring structure definition, i.e. there must not be another STRUCTURE_COMPONENT with the same identifier in the referring TYPEDEF_STRUCTURE. Note: Against the common rules for valid identifiers, it is not allowed to use '.' and '[', ']' in the name of a structure component.
ident	TypedefName	Identifier of the referenced type definition. This type definition can either be a TYPEDEF_AXIS, TYPEDEF_CHARACTERISTIC, TYPEDEF_BLOB, TYPEDEF_MEASUREMENT or a TYPEDEF_STRUCTURE. Note: It is not allowed to mix up components which represent measure objects and components which represent adjustable objects inside one structure. That means a structure can contain either measure objects only (defined via TYPEDEF_MEASUREMENT) or adjustable objects only (defined via TYPEDEF_AXIS, TYPEDEF_CHARACTERISTIC or TYPEDEF_BLOB).
ulong	AddressOffset	Defines the address offset of the structure component inside the parent structure.

Optional parameters

-> LAYOUT	Specifies the layout for multi-dimensional arrays. If the keyword is missing, multi-dimensional arrays are interpreted row by row (ROW_DIR).
-> MATRIX_DIM	Defines the dimension if an array of the referred components is used inside the structure.

Description:

Specification of a structure component by referencing a type definition. References to structure components must not lead to circle dependencies. The address of the structure component is calculated by the start address of the structure instance and the offsets inside the structure.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

It is not allowed to mix up components which represent measure objects and components which represent adjustable objects inside one structure.

Note: Usage of STRUCTURE_COMPONENT must not lead to circular dependencies between type definitions

Example:

```
See TYPEDEF_STRUCTURE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.135 SUB_FUNCTION

Prototype:

```
/begin SUB_FUNCTION (ident      Identifier)*
/end SUB_FUNCTION
```

Parameters:

ident	Identifier	Reference to function record. This function record is declared as subfunction of respective function.

Description:

This keyword can be used to define the hierarchical structure of functions.
Note: SUB_FUNCTION may only refer to objects of type FUNCTION.

Example:

```
/begin SUB_FUNCTION      ID_ADJUSTM_SUB
/end SUB_FUNCTION
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.136 SUB_GROUP

Prototype:

```
/begin SUB_GROUP    (ident      Identifier)*
/end SUB_GROUP
```

Parameters:

ident	Identifier	Reference to a group record. This group record is declared as sub-group of the respective GROUP.

Description:

This keyword can be used to define the hierarchical structure of groups. In particular, a set of groups referenced from a root group (with optional keyword ROOT) constitute a grouping mechanism.

Example:

```
/begin  SUB_GROUP        ID_ADJUSTM_SUB
/end SUB_GROUP
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.137 SUPPLIER

Prototype:

```
SUPPLIER            string     Manufacturer
```

Parameters:

string	Manufacturer	Name of the ECU manufacturer

Description:

String used to identify the manufacturer or supplier.

Example:

```
SUPPLIER                "Smooth and Easy"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.138 SYMBOL_LINK

Prototype:

```
SYMBOL_LINK          string      SymbolName
                     long        Offset
```

Parameters:

string	SymbolName	Name of the symbol within the corresponding linker map file
long	Offset	Offset of the Symbol relative to the symbol's address in the linker map file

Description:

This keyword can be used to specify the name of a symbol within a linker map file that corresponds to the respective CHARACTERISTIC, MEASUREMENT, or AXIS_PTS of the A2L file. The offset parameter defines the offset of the element defined in the A2L file relative to the address of the corresponding symbol of the linker map file (e.g. for arrays). Using this information, an automatic address update can be performed according to a provided linker map file.

Example:

```
SYMBOL_LINK              "_VehicleSpeed"    /* Symbol name */
                         0                  /* Offset */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.139 SYSTEM_CONSTANT

Prototype:

```
SYSTEM_CONSTANT     string     Name
                    string     Value
```

Parameters:

string	Name	system constant identifier
string	Value	value of the system constant as a string

Description:

System-defined constant.

Note: If the system constant shall be used in conversion formulas the name of the system constant has to be unique within all system constants of the MODULE, i.e. there must not be another system constant with the same name in the MODULE. Furthermore, the name must comply to the restrictions of data type ident and the value of the system constant must contain either a numerical value or a string that contains a further formula part.

Example:

```
SYSTEM_CONSTANT          "CONTROLLER_CONSTANT12"
                         "2.7134"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.140 TRANSFORMER

Prototype:

```
/begin TRANSFORMER  ident      Name
                    string     Version
                    string     Executable32
                    string     Executable64
                    ulong      Timeout
                    enum       Trigger
                    ident      InverseTransformer
                    [->TRANSFORMER_IN_OBJECTS]
                    [->TRANSFORMER_OUT_OBJECTS]
/end TRANSFORMER
```

Parameters:

ident	Name	Unique identifier of the TRANSFORMER definition. Note: The name of the transformer object has to be unique within all transformer objects of the ASAM MCD-2MC MODULE, i.e. there must not be another TRANSFORMER object with the same identifier in the MODULE. NO_INVERSE_TRANSFORMER may not be used here.
string	Version	Version of the required transformer.
string	Executable32	Name of the 32bit DLL file of the transformer algorithm. If the DLL is not specified the string is empty. The file extension is included in this name, without any path information.
string	Executable64	Name of the 64 bit DLL file of the transformer algorithm. If the DLL is not specified the string is empty. The file extension is included in this name, without any path information.
ulong	Timeout	Timeout value in [ms] specifying the maximum expected calculation time of the called transformer. After the timeout value is reached the MC tool may take actions to stop a running transformer. A timeout value of zero milliseconds indicates that the timeout is not specified for this TRANSFORMER and may not be used by the MC tool.
enum	Trigger	Specifies the trigger behavior of the TRANSFORMER. Possible triggers are: ON_CHANGE – Triggers the TRANSFORMER whenever a referenced input object in IN_OBJECTS is changed ON_USER_REQUEST – Triggers the TRANSFORMER whenever the user issues the trigger request in the MC tool.
Ident	InverseTransformer	Name of the TRANSFROMER which performs the inverse operation of this TRANSFORMER. If no inverse operation is available put NO_INVERSE_TRANSFORMER here.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Optional Parameters

-> TRANSFORMER_IN_OBJECTS	List of all objects (e.g. CHARTERISTICs, AXIS_PTS, BLOBs) which are used as input values for this TRANSFORMER
-> TRANSFORMER_OUT_OBJECTS	List of all objects (e.g. CHARTERISTICs, AXIS_PTS, BLOBs) which will be changed/updated by running this TRANSFORMER

Description:

This keyword specifies all parameters needed to perform a transformation between
- CHARACTERISTIC
- AXIS_PTS
- BLOB
- INSTANCE (based on TYPEDEF_AXIS, TYPEDEF_CHARACTERISTIC, TYPEDEF_STRUCTURE or TYPEDEF_BLOB).

Notes:

MEASUREMENTs can be used neither as an input nor output of a transformer.

No multistage transformer chain is allowed e.g. it is not allowed that an output of one transformer is the input of another one (exception: inverse transformers which are mutually exclusive when run).

No dependencies between transformers are allowed.

No circular chain of transformers is allowed.

For structures only complete structures are allowed as inputs and outputs. Single structure components or array elements are not allowed. If a transformer needs only a single member the whole structure is passed to the transformer. The transformer is responsible to extract the required member from the structure in this use case.

Only input and output objects are allowed for transformers which are compatible to the CDF standard. Which version of the CDF standard is used is up to the implementer of the TRANSFORMER. Care has to be taken that the TRANSFORMER and the MC tool implement the same CDF version.

Example:

```
/begin MEMORY_SEGMENT    FIM_CALIBRATION
                         "Special calibration data area handled via
                         transformation for FIM Matrix"
                         DATA
                         NOT_IN_ECU
                         EXTERN
                         0xFFFF0000
                         0x00010000
                         -1 -1 -1 -1 -1
/end MEMORY_SEGMENT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
/begin MEMORY_SEGMENT    STANDARD_CALIBRATION
                         "Standard calibration data area"
                         DATA
                         FLASH
                         EXTERN
                         0x00010000
                         0x00010000
                         -1 -1 -1 -1 -1
/end MEMORY_SEGMENT

/begin CHARACTERISTIC    link.err_sensor_voltage.fid_actuator_control
                         "FIM calibration for function Actuator
                         Control"
                         VALUE
                         0xFFFF0100            /* Located in virtual
                                               segment */
                         REC_S1VAL_20_u1
                         1.
                         CNV_S_2_RANGE_____5_CM
                         0.
                         255.
                         DISPLAY_IDENTIFIER    LINK.ERR_SENSOR_VOLTAGE.
                                               FID_ACTUATOR_CONTROL
                         FORMAT                "%1.0"
/end CHARACTERISTIC

/begin BLOB              fim_matrix_compressed
                         "FIM matrix in encoded data format used by ECU
                         FIM algorithm"
                         0x00011000 /*Located in ECU memory segment*/
                         20000
                         CALIBRATION_ACCESS    NO_CALIBRATION      /*
                                               Inhibit the user from
                                               calibrating it */
/end BLOB

/begin TRANSFORMER       FIM_TRANSFORMER_TOOL
                         "1.0.0.24"              /* Version info */
                         "fim_transformer_x32.dll" /* 32bit DLL */
                         ""                      /* 64bit DLL */
                         25000                   /* timeout in [ms] */
                         ON_USER_REQUEST
                         FIM_TRANSFORMER_TOOL_REVERSE
                         /begin TRANSFORMER_IN_OBJECTS
                           link.err_sensor_voltage.fid_actuator_control
                         /end TRANSFORMER_IN_OBJECTS
                         /begin TRANSFORMER_OUT_OBJECTS
                           fim_matrix_compressed
                         /end TRANSFORMER_OUT_OBJECTS
/end TRANSFORMER
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
/begin TRANSFORMER        FIM_TRANSFORMER_TOOL_REVERSE
                          "1.0.0.2"            /* Version info */
                          "fim_transformer_reverse_x32.dll"
                                               /* 32bit DLL */
                          "fim_transformer_reverse_x64.dll "
                                               /* 64bit DLL */
                          20000                /* timeout in [ms] */
                          ON_USER_REQUEST
                          FIM_TRANSFORMER_TOOL
                          /begin TRANSFORMER_IN_OBJECTS
                            fim_matrix_compressed
                          /end TRANSFORMER_IN_OBJECTS
                          /begin TRANSFORMER_OUT_OBJECTS
                            link.err_sensor_voltage.fid_actuator_control
                          /end TRANSFORMER_OUT_OBJECTS
/end TRANSFORMER

/begin COMPU_METHOD _     CNV_S_2_RANGE_____5_CM
                          "Possible error stati"
                          TAB_VERB
                          "%3.0"
                          "-"
                          COMPU_TAB_REF _CNV_S_2_RANGE_____5
/end COMPU_METHOD

/begin COMPU_VTAB _       CNV_S_2_RANGE_____5
                          "Error Stati"
                          TAB_VERB
                          4
                          0 "ACTIVE"
                          2 "NO_ERROR"
                          4 "LAST_FAILED"
                          8 "FAILED"
                          DEFAULT_VALUE "INVALID"
/end COMPU_VTAB

/begin RECORD_LAYOUT _    REC_S1VAL_20_u1
                          FNC_VALUES 1 UBYTE COLUMN_DIR DIRECT
/end RECORD_LAYOUT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.141 TRANSFORMER_IN_OBJECTS

Prototype:

```
/begin TRANSFORMER_IN_OBJECTS (ident       Name)*
/end TRANSFORMER_IN_OBJECTS
```

Parameters:

ident	Name	Name of a object used as input of the TRANSFORMER

Note: For restrictions on the allowed object types see chapter TRANSFORMER.

Description:

This keyword is used to specify the input object list for a TRANSFORMER.

Example:

```
/begin TRANSFORMER_IN_OBJECTS     CHAR_1
                                  BLOB_1
/end TRANSFORMER_IN_OBJECTS
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.142 TRANSFORMER_OUT_OBJECTS

Prototype:

```
/begin TRANSFORMER_OUT_OBJECTS (ident        Name)*
/end TRANSFORMER_OUT_OBJECTS
```

Parameters:

ident	Name	Name of a object used as output of the TRANSFORMER

Note: For restrictions on the allowed object types see chapter TRANSFORMER.

Description:

This keyword is used to specify the output object list for a TRANSFORMER.

Example:

```
/begin TRANSFORMER_OUT_OBJECTS        OUT_CHAR_1
                                      OUT_ID_124
/end TRANSFORMER_OUT_OBJECTS
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.143 TYPEDEF_AXIS

Prototype:

```
/begin TYPEDEF_AXIS ident      Name
                    string     LongIdentifier
                    ident      InputQuantity
                    ident      RecordLayout
                    float      MaxDiff
                    ident      Conversion
                    uint       MaxAxisPoints
                    float      LowerLimit
                    float      UpperLimit
                    [-> BYTE_ORDER]
                    [-> DEPOSIT]
                    [-> EXTENDED_LIMITS]
                    [-> FORMAT]
                    [-> MONOTONY]
                    [-> PHYS_UNIT]
                    [-> STEP_SIZE]
/end TYPEDEF_AXIS
```

Parameters:

ident	Name	Type identifier in the ECU program. This identifier is used for referencing and therefore has to be unique within all type definitions of the MODULE, i.e. there must not be another TYPEDEF_xx description with the same identifier in the MODULE.
string	LongIdentifier	Comment, description for the type or for the axes derived from this type definition.
ident	InputQuantity	Reference to an input quantity to be used for the axes derived from this type definition. The referenced input quantity must be a measurement (i.e. described either with keyword MEASUREMENT or with keyword INSTANCE using a TYPEDEF_MEASUREMENT description). If there is no input quantity assigned, set this parameter to NO_INPUT_QUANTITY.
ident	RecordLayout	Reference to the corresponding RECORD_LAYOUT which describes the data layout in the ECU memory.
float	MaxDiff	Maximum float with respect to the adjustment of a table value
ident	Conversion	Reference to a COMPU_METHOD which describes the conversion from ECU internal values into physical values and vice versa. If no conversion method is used, this parameter shall be set to NO_COMPU_METHOD.
uint	MaxAxisPoints	Maximum number of axis points.
float	LowerLimit	Plausible range of axis point values, lower limit (see also definition of AXIS_PTS)
float	UpperLimit	Plausible range of axis point values, upper limit (see also definition of AXIS_PTS)

Optional parameters:

-> BYTE_ORDER	Specifies the byte order (Intel, Motorola) where the standard value does not apply.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

-> DEPOSIT	Specifies the deposit of the axis points (absolute values or difference values), see definition of AXIS_PTS.
-> EXTENDED_LIMITS	Specifies an extended range of values see definition of AXIS_PTS.
-> FORMAT	Specifies the display format if the one described at the corresponding COMPU_METHOD shall not be used.
-> MONOTONY	Specifies the monotonous behavior.
-> PHYS_UNIT	Specifies the physical unit for the axis points if no conversion rule is referenced or if the physical unit of the referenced conversion method shall not be used.
-> STEP_SIZE	Specifies a delta value which is added to or subtracted from the current value when using up/down keys while calibrating.

Description:

Specification of a type definition for AXIS_PTS objects. This type definition can be used either to describe several axes with equal properties without duplicating redundant information or to describe AXIS_PTS objects which are part of structures.

Example:

```
/begin TYPEDEF_AXIS
  T_AXIS_N                    // type name
  "axis points for speed"     // description
  N                           // input quantity
  DAMOS_SST                   // reference to record layout
  100.0                       // maxdiff
  R_SPEED                     // reference to conversion method
  21                          // maximum number of axis points
  0.0                         // lower limit
  5800.0                      // upper limit
/end TYPEDEF_AXIS

/begin INSTANCE
  AXIS1                       // name for the axis points
  ""                          // use type description
  T_AXIS_N                    // reference to type definition
  0x1004080                   // start address
/end INSTANCE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.144 TYPEDEF_BLOB

Prototype:

```
/begin TYPEDEF_BLOB ident      Name
                    string     LongIdentifier
                    ulong      Size
                    [-> ADDRESS_TYPE]
/end TYPEDEF_BLOB
```

Parameters:

ident	Name	Type identifier in the ECU program. This identifier is used for referencing and therefore has to be unique within all type definitions of the MODULE, i.e. there must not be another TYPEDEF_xx with the same identifier in the MODULE.
string	LongIdentifier	Comment, description for the type or for the blob objects derived from this type definition
ulong	Size	Blob size, number of bytes

Optional parameters

-> ADDRESS_TYPE	Addressing of the TYPEDEF_BLOB (see in Table 5 enum addrtype) Note: If the keyword ADDRESS_TYPE is missing DIRECT addressing is used.

Description:

Specification of a type definition for BLOB objects. This type definition can be used either to describe several objects with equal properties without duplicating the redundant information or to describe BLOB objects which are part of structures.

Example:

```
/begin TYPEDEF_BLOB
  T_BLOB                    // type name
  "binary blob"             // description
  1024                      // number of bytes in blob
/end TYPEDEF_BLOB

/begin INSTANCE
  FIM_Blob                  // name for blob object
  "blob for FIM matrix"     // use own description
  T_BLOB                    // reference to type definition
  0x1004080                 // start address
/end INSTANCE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.145 TYPEDEF_CHARACTERISTIC

Prototype:

```
/begin TYPEDEF_CHARACTERISTIC
                    ident      Name
                    string     LongIdentifier
                    enum       Type
                    ident      RecordLayout
                    float      MaxDiff
                    ident      Conversion
                    float      LowerLimit
                    float      UpperLimit
                    [-> AXIS_DESCR]*
                    [-> BIT_MASK]
                    [-> BYTE_ORDER]
                    [-> DISCRETE]
                    [-> ENCODING]
                    [-> EXTENDED_LIMITS]
                    [-> FORMAT]
                    [-> MATRIX_DIM]
                    [-> NUMBER]
                    [-> PHYS_UNIT]
                    [-> STEP_SIZE]
/end TYPEDEF_CHARACTERISTIC
```

Parameters:

ident	Name	Type identifier in the ECU program. This identifier is used for referencing and therefore has to be unique within all type definitions of the MODULE, i.e. there must not be another TYPEDEF_xx with the same identifier in the MODULE.
string	LongIdentifier	Comment, description for the type or for the characteristic objects derived from this type definition.
enum	Type	Possible object types, see description of CHARACTERISTIC
ident	RecordLayout	Reference to the corresponding RECORD_LAYOUT which describes the data layout in the ECU memory.
float	MaxDiff	Maximum float with respect to the adjustment of a table value.
ident	Conversion	Reference to a COMPU_METHOD which describes the conversion from ECU internal values into physical values and vice versa. If no conversion method shall is used, this parameter shall be to NO_COMPU_METHOD.
float	LowerLimit	Plausible range of values, lower limit (see definition of CHARACTERISTIC)
float	UpperLimit	Plausible range of values, upper limit (see definition of CHARACTERISTIC)

Optional parameters

-> AXIS_DESCR	Specifies the axes parameters for curves and maps, see definition of CHARACTERISTIC
-> BIT_MASK	Specifies a bit mask for the handling of single bits.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

-> BYTE_ORDER	Specifies the byte order (Intel, Motorola) where the standard value does not apply.
-> DISCRETE	Indicates that the characteristic values are discrete values which should not be interpolated.
-> ENCODING	This keyword specifies the type of encoding used when the TYPEDEF_CHARACTERISTIC is of type ASCII. If no ENCODING is specified, the content is interpreted as string of 8 Bit characters.
-> EXTENDED_LIMITS	Specifies an extended range of values, see definition of CHARACTERISTIC.
-> FORMAT	Specifies the display format if the one described at the corresponding COMPU_METHOD shall not be used.
-> MATRIX_DIM	Specifies the size of multi-dimensional characteristic objects.
-> NUMBER	For string objects only: specifies the number of characters.
-> PHYS_UNIT	Specifies the physical unit for the value(s) if no conversion rule is referenced or if the physical unit of the referenced conversion method shall not be used.
-> STEP_SIZE	Specifies a delta value which is added to or subtracted from the current value when using up/down keys while calibrating.

Description:

Specification of a type definition for CHARACTERISTIC objects. This type definition can be used either to describe several objects with equal properties without duplicating the redundant information or to describe CHARACTERISTIC objects which are part of structures.

Example:

```
/begin TYPEDEF_CHARACTERISTIC
  T_TIME_MS                    // type name
  "time in ms"                 // description
  VALUE                        // object type
  RL_VALUE_ULONG               // record layout
  100.0                        // maxdiff
  NO_COMPU_METHOD              // no conversion
  10                           // lower limit
  65535                        // upper limit
  PHYS_UNIT "ms"               // physical unit
/end TYPEDEF_CHARACTERISTIC

/begin INSTANCE
  FilterTime                        // name for measure object
  "delay time for filtering"        // description
  T_TIME_MS                         // reference to type definition
  0x1004080                         // start address
  MATRIX_DIM 5                      // array size: 5 time values
/end INSTANCE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.146 TYPEDEF_MEASUREMENT

Prototype:

```
/begin TYPEDEF_MEASUREMENT
                    ident      Name
                    string     LongIdentifier
                    datatype   Datatype
                    ident      Conversion
                    uint       Resolution
                    float      Accuracy
                    float      LowerLimit
                    float      UpperLimit
                    [-> ADDRESS_TYPE]
                    [-> BIT_MASK]
                    [-> BIT_OPERATION]
                    [-> BYTE_ORDER]
                    [-> DISCRETE]
                    [-> ERROR_MASK]
                    [-> FORMAT]
                    [-> LAYOUT]
                    [-> MATRIX_DIM]
                    [-> PHYS_UNIT]
/end TYPEDEF_MEASUREMENT
```

Parameters:

ident	Name	Type identifier in the ECU program. This identifier is used for referencing and therefore has to be unique within all type definitions of the MODULE, i.e. there must not be another TYPEDEF_xx with the same identifier in the MODULE.
string	LongIdentifier	Comment, description for the type or for the measure objects derived from this type definition.
datatype	Datatype	Data type of the measure objects derived from this type definition.
ident	Conversion	Reference to a COMPU_METHOD which describes the conversion from ECU internal values into physical values and vice versa. If there is no conversion method available set this parameter to NO_COMPU_METHOD.
uint	Resolution	Smallest possible change in bits; recommended value: 1 (This parameter is currently not interpreted by any known MC-System. It can be ignored.)
float	Accuracy	Possible variation from exact value in %,recommended value: 0 (This parameter is currently not interpreted by any known MC-System. It can be ignored.)
float	LowerLimit	Plausible range of values, lower limit (see definition of MEASUREMENT)
float	UpperLimit	Plausible range of values, upper limit (see definition of MEASUREMENT)

Optional parameters:

-> ADDRESS_TYPE	Addressing type (see definition of MEASUREMENT)

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

-> BIT_MASK	Specifies a bit mask for the handling of single bits.
-> BIT_OPERATION	Specifies an operation on the masked out value.
-> BYTE_ORDER	Specifies the byte order (Intel, Motorola) where the standard value does not apply.
-> DISCRETE	Indicates that the measure values are discrete values which should not be interpolated.
-> ERROR_MASK	Mask bits which indicate that the value is in error.
-> FORMAT	Specifies the display format if the one described at the corresponding COMPU_METHOD shall not be used.
-> LAYOUT	Specifies the memory layout of multi-dimensional measurement arrays (see definition of MEASUREMENT)
-> MATRIX_DIM	Specifies the size of multi-dimensional measurement arrays. (see definition of MEASUREMENT)
-> PHYS_UNIT	Specifies the physical unit for the value(s) if no conversion rule is referenced or if the physical unit of the referenced conversion method shall not be used.

Description:

Specification of a type definition for MEASUREMENT objects. This type definition can be used either to describe several objects with equal properties without duplicating the redundant information or to describe MEASUREMENT objects which are part of structures.

Example:

```
/begin TYPEDEF_MEASUREMENT
  T_Register                    // type name
  "register content"            // description
  UWORD                         // data type
  RegisterConversion            // conversion method
  1                             // resolution
  0                             // accuracy
  0                             // lower limit
  4294967295                    // upper limit
/end TYPEDEF_MEASUREMENT

/begin INSTANCE
  Register_A11                  // name for measure object
  ""                            // use description of type definition
  T_Register                    // reference to type definition
  0xC0000094                    // start address
/end INSTANCE
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.147 TYPEDEF_STRUCTURE

Prototype:

```
/begin TYPEDEF_STRUCTURE
                    ident      Name
                    string     LongIdentifier
                    ulong      Size
                    [-> ADDRESS_TYPE]
                    [-> CONSISTENT_EXCHANGE]
                    [-> STRUCTURE_COMPONENT]*
/end TYPEDEF_STRUCTURE
```

Parameters:

ident	Name	Type identifier in the ECU program. This identifier is used for referencing and therefore has to be unique within all type definitions of the MODULE, i.e. there must not be another TYPEDEF_xx with the same identifier in the MODULE.
string	LongIdentifier	Comment, description for the type or for the structure instances derived from this type definition.
ulong	Size	Total size of the structure in bytes including sub structures and structure components. Note: The size information is necessary for address calculation if arrays of structures are used. Therefore the value for size must be including alignment information.

Optional parameters

-> ADDRESS_TYPE	Addressing of the TYPEDEF_STRUCTURE (see in Table 5 enum addrtype) Note: If the keyword ADDRESS_TYPE is missing DIRECT addressing is used. Note: This keyword can be used to define pointers to structures
-> CONSISTENT_EXCHANGE	Indicates that the structure is always handled complete.
-> STRUCTURE_COMPONENT	Defines a structure component by referencing a type definition Note: It is not allowed to mix up components which represent measure objects and components which represent adjustable objects inside one structure. That means a structure can contain either measure objects only (defined via TYPEDEF_MEASUREMENT) or adjustable objects only (defined via TYPEDEF_AXIS, TYPEDEF_CHARACTERISTIC or TYPEDEF_BLOB).

Description:

Specification of a structured type which is composed of single elements and sub structures.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Example:

C Code:

```
typedef struct {
   unsigned long     A;
   unsigned char     dummy1[30];
   unsigned long     B[10];
   unsigned char     dummy2[10];
} MyStructure;

MyStructure Instance1;
MyStructure Instance2[2][3];
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

ASAP2 representation:

```
/begin TYPEDEF_CHARACTERISTIC
  MyUnsignedLongType            // type name
  ""                            // description
  VALUE                         // object type
  RL_VALUE_ULONG                // record layout
  100.0                         // maxdiff
  NO_COMPU_METHOD               // no conversion
  10                            // lower limit
  65535                         // upper limit
/end TYPEDEF_CHARACTERISTIC

/begin TYPEDEF_STRUCTURE
  MyStructure                   // structure name
  "…"                           // description
  84                            // total size of structure
                                // including dummies
  /begin STRUCTURE_COMPONENT
        A MyUnsignedLongType
        0                       // address offset
  /end STRUCTURE_COMPONENT
  // structure component <dummy1> not needed in ASAP2
  /begin STRUCTURE_COMPONENT
        B MyUnsignedLongType
        34                      // address offset
                                // (respecting A and dummy1)
        MATRIX_DIM 10
  /end STRUCTURE_COMPONENT
  // structure component <dummy2> not needed in ASAP2
/end TYPEDEF_STRUCTURE

/begin INSTANCE
  Instance1                     // instance name
  ""                            // instance description
  MyStructure                   // reference to type definition
  0x1200                        // start address of instance
/end INSTANCE

/begin INSTANCE
  Instance2                     // instance name
  ""                            // instance description
  MyStructure                   // reference to type definition
  0x2030                        // start address of instance
  MATRIX_DIM 2 3                // array dimensions
  LAYOUT COLUMN_DIR             // memory layout
/end INSTANCE

/begin GROUP MyGroup ""
  /begin REF_CHARACTERISTIC
        Instance2[0][0].A Instance1
  /end REF_CHARACTERISTIC
/end GROUP
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.148 UNIT

Prototype:

```
/begin UNIT         ident      Name
                    string     LongIdentifier
                    string     Display
                    enum       Type
                    [-> REF_UNIT]
                    [-> SI_EXPONENTS]
                    [-> UNIT_CONVERSION]
/end UNIT
```

Parameters:

ident	Name	identifier in the program, referencing is based on this name Note: The name of the unit has to be unique within all units of the ASAM MCD-2 MC MODULE, i.e. there must not be another UNIT object with the same identifier in the MODULE.
string	LongIdentifier	comment, description
string	Display	string to be used to display the measurement unit of a physical value
enum	Type	Type of the Unit: DERIVED measurement unit derived from another one referenced by the optional parameter REF_UNIT. EXTENDED_SI extended SI unit. Recommendation: The principle of describing "real" measurement units is to refer to SI units. Therefore, this relationship should be given by using the optional parameter SI_EXPONENTS.

Optional Parameters:

-> SI_EXPONENTS	This keyword is used to specify the exponents of the seven base dimensions required to define an extended SI unit.
-> REF_UNIT	This keyword is used to reference to another measurement unit from which the one using REF_UNIT is derived.
-> UNIT_CONVERSION	This keyword is used to specify the linear relationship between two measurement units.

Description:

Specification of a measurement unit.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Example:

```
/begin UNIT
                        metres_per_second
                        "extended SI unit for velocity"
                        "[m/s]"
                        EXTENDED_SI
        SI_EXPONENTS    1 0 -1 0 0 0 0     /* [m] * [s]^-1 */
/end UNIT

/begin UNIT
                        kms_per_hour
                        "derived unit for velocity: kilometres per
                        hour"
                        "[km/h]"
                        DERIVED
            REF_UNIT    metres_per_second
    UNIT_CONVERSION     3.6  0.0     /* y [km/h] = (60*60/1000) * x
                                        [m/s] + 0.0 */
/end UNIT

/begin UNIT
                        miles_per_hour
                        "derived unit for velocity: miles per hour"
                        "[mph]"
                        DERIVED
            REF_UNIT    metres_per_second
    UNIT_CONVERSION     2.237  0.0   /* y [mph] = (60*60/1609) * x
                                        [m/s] + 0.0 */
/end UNIT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.149 UNIT_CONVERSION

Prototype:

```
UNIT_CONVERSION     float       Gradient
                    float       Offset
```

Parameters:

float	Gradient, Offset	gradient and offset of the linear relationship between two measurement units: f(x) = gradient * x + offset

Description:

Specification of the linear relationship between two measurement units given by describing the conversion from the referenced unit to the derived unit:

derived_unit = f(referenced_unit)

The referenced measurement unit had to be specified with parameter REF_UNIT.

Example:

```
/begin UNIT
                        kelvin
                        "base unit for temperature: Kelvin"
                        "[K]"
                        EXTENDED_SI
        SI_EXPONENTS    0 0 0 0 1 0 0
/end UNIT

/begin UNIT
                        degC
                        "unit for temperature: degree Celsius"
                        "[°C]"
                        DERIVED
            REF_UNIT    kelvin
     UNIT_CONVERSION    1.0 -273.15  /* y [°C] = 1.0 * x [K] + (-
                                        273.15) */
/end UNIT
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.150 USER

Prototype:

```
USER                    string      UserName
```

Parameters:

string	UserName	Name of the user

Description:

Specification of the user name.

Example:

```
USER                    "Nigel Hurst"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.151 USER_RIGHTS

Prototype:

```
/begin USER_RIGHTS  ident      UserLevelId
                    [-> READ_ONLY]
                    [-> REF_GROUP]*
/end USER_RIGHTS
```

Parameters:

ident	UserLevelId	unique identifier in the program, referencing is based on this 'name' Note: The name of the USER_RIGHTS has to be unique within all USER_RIGHTS of the ASAM MCD-2 MC MODULE, i.e. there must not be another USER_RIGHTS object with the same identifier in the MODULE. When a user logs into the MCD system, a UserLevelId is assigned.

Optional parameters:

-> REF_GROUP:	Reference to groups. Only the CHARACTERSITIC and MEASUREMENT members of the referenced groups including the members of nested subgroups (and functions nested in such groups) are visible to the user of the MCD system. If the READ_ONLY attribute is set, the CHARACTERISTICs are visible but not available for calibration (not tunable). Note: The restrictions are applied by the MCD system as a global filter in the user interface (active for all manual selection or calibration operations). When navigating by GROUPs, only the GROUPs declared in USER_RIGHTS need to be provided in the selection list.
-> READ_ONLY:	This keyword can be used to define all characteristics of the groups referenced by this USER_RIGHT statement as READ_ONLY (not tunable). Use Case : A group can be defined to specify a set of characteristics to be not tunable for a group of users (control of access rights). In order to achieve this, the group is referenced in a USER_RIGHT statement with the READ_ONLY attribute, related to the user group. When a login to the MCD system identifies the user as member of a group for which the USER_RIGHT statement contains the READ_ONLY attribute, all CHARACTERISTICs of this group shall be treated as if the READ_ONLY attribute was directly related to the CHARACTERISTICs.

Description:

This keyword can be used to define groups accessible for certain users. All USER-RIGHTS groups are listed to the user who can select one of these groups. All measurements and characteristics belonging to that group and its subgroups (and sub

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

subgroups and so on) are accessible (i.e. visible) to the user. The keyword READ_ONLY is used to define the referred group(s) as containing characteristics that are only readable but not writeable (i.e. they can not be adjusted). This property is also inherited by subgroups, i.e. if a group is marked as READ_ONLY all its subgroups (with respect to that USER RIGHT) are also only READ_ONLY.

Example:

```
/begin  USER_RIGHTS      calibration_engineers
  /begin REF_GROUP       group_1
  /end REF_GROUP
/end  USER_RIGHTS

/begin  USER_RIGHTS      measurement_engineers
  /begin REF_GROUP       group_1
  /end REF_GROUP
  READ_ONLY
/end USER_RIGHTS

/begin  GROUP            group_1
  /begin REF_CHARACTERISTIC
                         KF1
                         KF2
  /end REF_CHARACTERISTIC
  /begin REF_MEASUREMENT
                         NMOT
                         TMOT
  /end REF_MEASUREMENT
/end  GROUP
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.152 VAR_ADDRESS

Prototype:

```
/begin VAR_ADDRESS  (ulong     Address)*
/end VAR_ADDRESS
```

Parameters:

ulong	Address	Start address of one variant of variant coded adjustable object.

Description:

This keyword can be used to define a list of start addresses of variant coded adjustable objects (see keyword VAR_CHARACTERISTIC). The number of addresses agrees with number of valid combinations of adjustable objects variant criteria (forbidden combinations excluded). The order of addresses corresponds to the order of variant criteria defined with parameter 'CriterionName' at keyword VAR_CHARACTERISTIC. The priority of index increment is according to the following rules:

- the priority of index increment is inverse to the order of variant criteria definition at keyword VAR_CHARACTERISTIC, e.g.:
- the first variant criterion has the lowest priority
- the last variant criterion has the highest priority

The following example describes the order of addresses of an adjustable object depending on three variant criterions with 'L', 'N', and 'M' criterion values:

Example:

Crit1 = { $Val_{1,1}$, $Val_{1,2}$, ...$Val_{1,L}$)
Crit2 = { $Val_{2,1}$, $Val_{2,2}$, ...$Val_{2,M}$)
Crit3 = { $Val_{3,1}$, $Val_{3,2}$, ...$Val_{3,N}$)

Corresponding address list:

Address[0]	= Address ($Val_{1,1}$, $Val_{2,1}$, $Val_{3,1}$)
Address[1]	= Address ($Val_{1,1}$, $Val_{2,1}$, $Val_{3,2}$)
:	
Address[N - 1]	= Address ($Val_{1,1}$, $Val_{2,1}$, $Val_{3,N}$)
Address[N]	= Address ($Val_{1,1}$, $Val_{2,2}$, $Val_{3,1}$)
Address[N + 1]	= Address ($Val_{1,1}$, $Val_{2,2}$, $Val_{3,2}$)
	:
Address[N + N - 1]	= Address ($Val_{1,1}$, $Val_{2,2}$, $Val_{3,N}$)
	:

Example:

```
/begin VAR_ADDRESS
                        0x8840
                        0x8858
                        0x8870
                        0x8888
/end VAR_ADDRESS
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.153 VAR_CHARACTERISTIC

Prototype:

```
/begin VAR_CHARACTERISTIC
                    ident      Name
                    (ident     CriterionName)*
                    [-> VAR_ADDRESS]
/end VAR_CHARACTERISTIC
```

Parameters:

ident	Name	Identifier of variant coded adjustable object (refers to CHARACTERISTIC or AXIS_PTS record).
ident	CriterionName	Corresponding to each combination of variant criteria defined with this parameter the control unit software contains variants of concerning adjustable object.

Optional Parameters:

-> VAR_ADDRESS	Definition of start address of adjustable objects variants.

Description:

This keyword defines one adjustable object to be variant coded, i.e. this adjustable objects is multiple deposited in control unit software corresponding to the assigned variant criteria. The number of variants results on valid combinations (forbidden combinations excluded) of variant criteria.

Note: If an AXIS_PTS object is variant coded, the curves and maps using this common axis must be variant coded in the same way, i.e. they have to refer the same variant criterions in the same order.

Example:

```
/begin VAR_CHARACTERISTIC      /* define NLLM as variant coded */
                       NLLM
                       Gear  Car
  /* gear box including the 2 variants "Manual" and "Automatic" */
  /* car body including the 3 variants "Limousine",  "Kombi" and */ /*
  "Cabrio" */

  /* four addresses corresponding to the four valid combinations */
  /* of criterion 'Gear' and 'Car' (see example for VAR_CRITERION)*/
  /begin VAR_ADDRESS
                       0x8840
                       0x8858
                       0x8870
                       0x8888
  /end VAR_ADDRESS
/end VAR_CHARACTERISTIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.154 VAR_CRITERION

Prototype:

```
/begin VAR_CRITERION  ident    Name
                      string   LongIdentifier
                      (ident   Value )*
                      [-> VAR_MEASUREMENT]
                      [-> VAR_SELECTION_CHARACTERISTIC]
/end VAR_CRITERION
```

Parameters:

ident	Name	Identifier of variant criterion.
string	LongIdentifier	Comment to describe the variant criterion.
ident	Value	Enumeration of criterion values. Note: The variant values have to be unique within the variant criterion, i.e. it is not allowed to use the same variant name more than once. The variant values must also be assigned to selector values in the verbal conversion table of the variant selector (see VAR_SELECTION_CHARACTERISTIC resp. VAR_MEASUREMENT).

Optional Parameters:

-> VAR_MEASUREMENT	This keyword can be used to specify a special measurement object. This measurement object indicates with its current value the variant which has effect on running control unit software.
-> VAR_SELECTION_CHARACTERISTIC	This keyword is used to specify a special characteristic to change the variant of software which is running on control unit.

Description:

This keyword describes a variant criterion, i.e. some adjustable objects are multiple deposited in control unit software corresponding to the enumeration of variant criterion values.

Note: The keywords VAR_SELECTION_CHARACTERISTIC and VAR_MEASUREMENT must not be used concurrently inside one variant criterion.

Example:

```
/* variant criterion "Car body" with three variants */
/begin VAR_CRITERION            Car
                                "Car body"
                                /*Enumeration of criterion values*/
                                Limousine  Kombi  Cabrio
                VAR_MEASUREMENT    S_CAR
/end VAR_CRITERION
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.155 VAR_FORBIDDEN_COMB

Prototype:

```
/begin VAR_FORBIDDEN_COMB
                    ( ident    CriterionName
                      ident    CriterionValue )*
/end VAR_FORBIDDEN_COMB
```

Parameters:

ident	CriterionName	Identifier of variant criterion.
ident	CriterionValue	Value of variant criterion ' CriterionName '.

Description:

This keyword describes a forbidden combination of values of different variant criteria.

Example:

```
/* forbidden variant combination (doesn't exist in control unit software): */
/begin VAR_FORBIDDEN_COMB
   Car Limousine  /* variant value 'Limousine' of criterion 'Car'  */
   Gear Manual    /* variant value 'Manual' of criterion 'Gear'  */
/end VAR_FORBIDDEN_COMB
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.156 VAR_MEASUREMENT

Prototype:

```
VAR_MEASUREMENT     ident       Name
```

Parameters:

ident	Name	Identifier of measurement object which indicates the current criterion value. This parameter refers to a MEASUREMENT record of description file.

Description:

This keyword can be used to specify a special measurement object. This measurement object indicates with its current value the variant which has effect on running control unit software. To be able to map the current object value to the variant the referenced measurement object must have a conversion method of type COMPU_VTAB and the strings defined at the conversion table must correspond to the criterion values at the VAR_CRITERION record. This impliciteley rules out, that one measurement object is used as variant selector for different variant criterions.

Note: VAR_MEASUREMENT may only refer to objects of type MEASUREMENT.

Example:

```
/begin COMPU_VTAB        V_GEAR_BOX
                         "variants of criterion ""Type of Gear Box"""
                         TAB_VERB
                         3
                         17 "Limousine"
                         39 "Kombi"
                         41 "Cabrio"
                         DEFAULT_VALUE "unknown"
/end COMPU_VTAB


/begin VAR_CRITERION     Car
                         "Car body"
                         Limousine  Kombi  Cabrio
     VAR_MEASUREMENT     S_GEAR_BOX
/end VAR_CRITERION
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.157 VAR_NAMING

Prototype:

```
VAR_NAMING          enum       Tag
```

Parameters:

enum	Tag	Format of variant extension (index). Possible values: NUMERIC variant extension is a number (integer: 0,1,2,3...).

Description:

This keyword defines the format of variant extension (index) of adjustable objects name. The extension is used at MCD to distinguish the different variants of adjustable objects.

Example:

```
/* variant extension: see example VAR_CRITERION*/
VAR_NAMING              NUMERIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.158 VAR_SELECTION_CHARACTERISTIC

Prototype:

```
VAR_SELECTION_CHARACTERISTIC
                    ident      Name
```

Parameters:

ident	Name	Identifier of characteristic object which indicates the current criterion value. This parameter refers to a CHARACTERISTIC record of description file.

Description:

This keyword can be used to specify a special characteristic object. This characteristic object changes with its current value the variant which has effect on running control unit software.
To be able to map the current object value to the variant the referenced characteristic object must have a conversion method of type COMPU_VTAB and the strings defined at the conversion table must correspond to the criterion values at the VAR_CRITERION record. This impliciteley rules out, that one characteristic object is used as variant selector for different variant criterions.

Note: VAR_SELECTION_CHARACTERISTIC may only refer to objects of type CHARACTERISTIC. This includes objects which are defined via keyword INSTANCE and instance components using a TYPEDEF_CHARACTERISTIC definition.

Example:

```
/begin COMPU_VTAB          V_GEAR_BOX
                           "variants of criterion ""Type of Gear Box"""
                           3
                           17 "Limousine"
                           39 "Kombi"
                           41 "Cabrio"
                           DEFAULT_VALUE "unknown"
/end COMPU_VTAB

/begin VAR_CRITERION              Car
                                  "Car body"
                                  Limousine  Kombi  Cabrio
VAR_SELECTION_CHARACTERISTIC      S_GEAR_BOX
/end VAR_CRITERION
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.159 VAR_SEPARATOR

Prototype:

```
VAR_SEPARATOR       string     Separator
```

Parameters:

string	Separator	This parameter defines the separating symbol of variant extension.

Description:

This keyword can be used to define the separating symbol between the two parts of adjustable objects name: 1.) identifier 2.) variant extension.

Note: The identifier of description record of variant coded adjustable objects contains no variant extension. The extension is needed to distinguish the variants at MCD.

Example:

```
VAR_SEPARATOR                "."     /* example: "PUMKF.1"  */
/* three parts of variant coded adjustable objects name:    */
/* 1.) Identifier of adjustable object:    "PUMKF"     */
/* 2.) Separator:             "." (decimal point)      */
/* 3.) Variants extension:   "1"     */
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.160 VARIANT_CODING

Prototype:

```
/begin VARIANT_CODING
                    [-> VAR_CHARACTERISTIC]*
                    [-> VAR_CRITERION]*
                    [-> VAR_FORBIDDEN_COMB]*
                    [-> VAR_NAMING]
                    [-> VAR_SEPARATOR]
/end VARIANT_CODING
```

Optional Parameters:

-> VAR_CHARACTERISTIC	This keyword defines one adjustable object to be variant coded, i.e. this adjustable objects is multiple deposited in control unit software corresponding to the assigned variant criteria.
-> VAR_CRITERION	This keyword describes a variant criterion, i.e. some adjustable objects are multiple deposited in control unit software corresponding to the enumeration of variant criterion values
-> VAR_FORBIDDEN_COMB	This keyword describes a forbidden combination of different variant criteria.
-> VAR_NAMING	This keyword defines the format of variant extension (index) of adjustable objects name (index is used at MCD to distinguish the variants
-> VAR_SEPARATOR	This keyword can be used to define the separating symbol between the two parts of adjustable objects name: 1.) identifier 2.) variant extension. Note: The identifier of description record of variant coded adjustable objects contains no variant extension. This extension is needed to distinguish the variants at MCD

Description:

The information of variant coding is grouped to this keyword. Variant coding means, that control unit software contains several variants (copies) of some adjustable objects, whereas description file contains only one record to describe. In a real ECU only one variant is in use, depending on car-specific parameters.

Example:

```
/begin VARIANT_CODING
        VAR_SEPARATOR    "."          /* PUMKF.1 */
        VAR_NAMING       NUMERIC

   /* variant criterion  "Car body" with three variants */
   /begin VAR_CRITERION  Car
                         "Car body"
         Limousine Kombi  Cabrio
         VAR_SELECTION_CHARACTERISTIC CAR_BOX
   /end VAR_CRITERION
   /* variant criterion "Type of gear box" with two variants */
   /begin VAR_CRITERION  Gear
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
        "Type of gear box"
        Manual  Automatic
        VAR_SELECTION_CHARACTERISTIC S_GEAR_BOX
    /end VAR_CRITERION

    /begin VAR_FORBIDDEN_COMB         /* forbidden: Limousine-Manual*/
                        Car Limousine
                        Gear Manual
    /end VAR_FORBIDDEN_COMB
    /begin VAR_FORBIDDEN_COMB         /* forbidden: Cabrio-Automatic*/
                        Car Cabrio
                        Gear Automatic
    /end VAR_FORBIDDEN_COMB
    /begin VAR_CHARACTERISTIC
                        PUMKF       /*define PUMKF as variant coded*/
                        Gear        /* Gear box variants */
      /begin VAR_ADDRESS
                        0x7140
                        0x7168
    /end VAR_ADDRESS
    /end VAR_CHARACTERISTIC
    /begin VAR_CHARACTERISTIC
                        NLLM        /*define NLLM as variant coded */
                        Gear  Car   /*car body and gear box variants*/
      /begin VAR_ADDRESS
                        0x8840
                        0x8858
                        0x8870
                        0x8888
        /end VAR_ADDRESS
    /end VAR_CHARACTERISTIC
/end VARIANT_CODING
```

Table 13 Example of NLLM - variants coding

Type of gear box / Car body	MANUAL	AUTOMATIC
Limousine	doesn't exist	NLLM.3 (address = 0x8870)
Kombi	NLLM.1 (address = 0x8840)	NLLM.4 (address = 0x8888)
Cabrio	NLLM.2 (address = 0x8858)	doesn't exist

Table 14 Example of PUMKF - variants coding

Type of gear box	
MANUAL	**AUTOMATIC**
PUMKF.0 (address = 0x7140)	PUMKF.1 (address = 0x7168)

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.161 VERSION

Prototype:

```
VERSION              string     VersionIdentifier
```

Parameters:

string VersionIdentifier short identifier for the version

Description:

String for identification of the version with maximum MAX_STRING characters.

Example:

```
VERSION                    "BG5.0815"
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.162 VIRTUAL

Prototype:

```
/begin VIRTUAL      (ident     MeasuringChannel)*
/end VIRTUAL
```

Parameters:

ident	MeasuringChannel	Reference to a measurement (MEASUREMENT) or a virtual measurement (MEASUREMENT, VIRTUAL)

Note: If the referenced measure object is an element of an array or a component of a structure, the identifier to be used for the reference shall be the name built according to rules described at INSTANCE.

Description:

This keyword allows virtual measurements to be specified. For this, constants, measurements and virtual measurements can be combined into one quantity. The list specified with the VIRTUAL keyword indicates the quantities to be linked (reference). These quantities are combined into one measurement by means of a single conversion formula. The conversion formula must be capable of processing several input quantities.

The references used in the computation formula are named X1, X2, X3, … . The reference X1 references the first parameter of the attached parameter list, X2 the second, X3 the third, ….
If there is only one reference used it is allowed to use X instead of X1.

Example:

```
/begin MEASUREMENT      PHI_FIRING          /* Name */
                        "Firing angle"      /* Long identifier */
                        UWORD               /* Data type */
                        R_PHI_FIRING        /* Conversion */
                        1                   /* Resolution */
                        0                   /* Accuracy */
                        120.0               /* Lower limit */
                        8400.0              /* Upper limit */
  /*Quantities to be linked: 2 measurements */
  /begin VIRTUAL        PHI_BASIS
                        PHI_CORR
  /end VIRTUAL
/end MEASUREMENT

/begin COMPU_METHOD     R_PHI_FIRING        /* Name */
                        "Addition of two measurements"
                        FORM                /* Convers_type */
                        "%4.2"              /* Display format */
                        "GRAD_CS"           /* physical unit */
  /begin FORMULA        "X1 + X2"           /* X1 -> PHI_BASIS */
                                            /* X2 -> PHI_CORR */
  /end FORMULA
/end COMPU_METHOD
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

4.7.163 VIRTUAL_CHARACTERISTIC

Prototype:

```
/begin VIRTUAL_CHARACTERISTIC
                    string     Formula
                    (ident     Characteristic)*
/end VIRTUAL_CHARACTERISTIC
```

Parameters:

string	Formula	Formula to be used for the calculation of the initial physical value of the characteristic from the physical value of other characteristics. Note: The result is interpreted as internal value of the virtual characteristic.
ident	Characteristic	Identifier of those adjustable objects that are used for the calculation of this characteristic. Note: If the referenced object is an element of an array or a component of a structure, the identifier to be used for the reference shall be the name built according to rules described at INSTANCE.

Description:

This keyword allows to define characteristics that are not deposited in the memory of the control unit, but can be used to indirectly calibrate other characteristic values in the control unit, if these are declared to be dependent on this characteristic. The introduction of virtual characteristic is therefore useful for saving memory in the case the calibration with dependent characteristics is used.

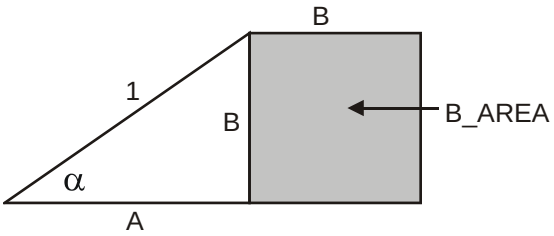


Figure 8 VIRTUAL_CHARACTERISTIC

For the initial value of the virtual characteristic must be derived from the values of other characteristics. The mechanism to implement this is the same as for dependent characteristics by a list of characteristics and a formula, e.g. α = asin(B). Also B might be virtual, i.e. its value has to be derived from B_AREA.
The following example makes clear how the calibration process takes place. When the virtual characteristic α is initialized, the value of α is calculated from the value of B. Therefore B_{Int} is read from the ECU and $B_{phys} = B_{Int}/100$ is computed. Assuming the value $B_{Int} = 80$, $B_{phys} = 0.8$ and $\alpha_{phys} = \text{asin}(B_{phys}) = 53.13$. Since virtual characteristics are not in the memory of an ECU, α_{Int} and α_{phys} may coincide if the datatype for α_{Int} is chosen an float datatype and the conversion formula is the identity (one to one formula).

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

The references used in the virtual dependency formula are named X1, X2, X3, … . The reference X1 references the first parameter of the attached parameter list, X2 the second, X3 the third, ….
If there is only one reference used it is allowed to use X instead of X1.

Example:

```
/begin VIRTUAL_CHARACTERISTIC
                        „sin(X1)“
                        B
/end VIRTUAL_CHARACTERISTIC

/* Example for ParamB - ParamA */
/begin DEPENDENT_CHARACTERISTIC
              "X2-X1"
              ParamA    /* is referenced by X1 */
              ParamB    /* is referenced by X2 */
/end DEPENDENT_CHARACTERISTIC
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

5 Include mechanism

For the description of projects involving several control units or measurement and calibration devices of various manufacturers the Include statement can be used. There is no restriction where to use include because it is a simple text replacement mechanism.

/include <filename>

The parameter <filename> may include a relative or absolute path or a UNC path. Relative paths are relative to the location of the ASAM MCD-2 MC file containing the /include statement. The relative path uses backslashes without escape sequences.

This statement allows several description files to be integrated into one project description. The filename may be put between quotation marks. If the filename contains spaces or path information the quotation marks are required.

Example:

```
/*****************File PROJECT1.A2L*******************************/
ASAP2_VERSION 1
              70
/begin PROJECT          RAPE-SEED ENGINE "Engine tuning for operation
                        with rape oil"
     /begin HEADER      "General project description"
        VERSION         "0815"
        PROJECT_NO      1188
     /end HEADER

     /include "C:\ENG_ECU.A2L"
     /include "..\includes\ABS_ECU.A2L"
     /include "SPEC_ECU.A2L"
     /include "\\MyServer\VariableDescriptions\ESP_ECU.A2L"
/end PROJECT
/*****************End of file PROJECT1.A2L*********************/
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

6 ASAM MCD-2 MC Metalanguage

6.1 General

Between MCD System and the devices or ECU's an interface is used. The description data in the measurement and calibration system are divided into two categories:

1. Parameters that are used by the control interface.
2. Parameters that are only analyzed by the driver and whose meaning is hidden to the control interface (interface-specific parameters). They are transferred to the driver as a binary block.

These two measures should make it possible that new interface module types can be handled without having to introduce any changes in the control part of the measurement and calibration system but simply by incorporating a new driver.
For the description of the interface-specific parameters a description language (ASAM MCD-2 MC metalanguage, in short AML) will be defined on the following pages. Each manufacturer can specify a special set of parameters for their own interface module types (format description). Using this format description (in AML) the standardized measurement and calibration system must be capable of reading in the *interface-specific parameters* of the description file and transferring them to the drivers.

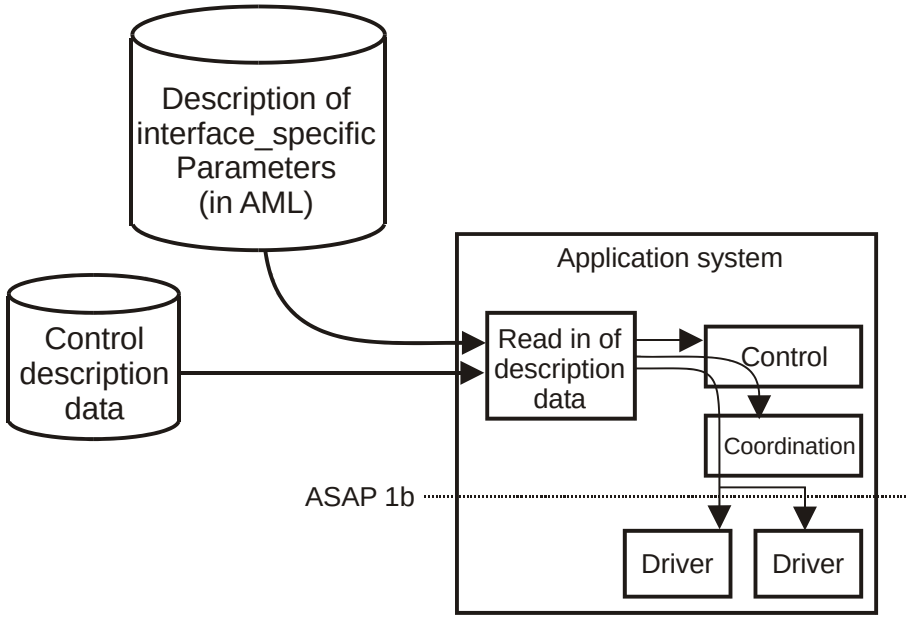


Figure 9 Schematic data flow of description data

For parameterization of the drivers and for access to the adjustable and measurement objects different parameters have to be used within the various drivers. The user interface, which does not need to know these parameters, in fact only requires a description of the data types to be able to read in these interface-specific parameters. This description, based on the ASAM MCD-2 MC metalanguage described hereafter, occurs either INLINE within the description file, or in separate files. In the second case the Include statement can be used to integrate the description of interface- specific parameters:

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
/begin MODULE ...
        /include <filename>
  ....
/end MODULE
```

6.2 Format of the ASAM MCD-2 MC metalanguage

Table 15 BNF Terminology

Symbol	Meaning
<...>	Place marker
[...]	optional part
|	or (in the exclusive sense)

To describe the grammar of the ASAM MCD-2 MC metalanguage, an extended Backus-Naur format is used:

The non-terminals are listed in the first column of Table 16 Grammar in the extended Backus-Naur format, the production rules are given in the second column.

A terminal (keyword) is written bold and is enclosed within quotes "**struct**".

An optional part is enclosed within square brackets: [identifier]**.**

Alternative parts are separated by a pipe symbol: "char" | "int" | "long".

Non-formally defined parts are in italics. These are in particular:

- *tag:* is used to define a keyword of the ASAM MCD-2 MC metalanguage by means of a character sequence enclosed within double inverted commas.
- *identifier:* identifier for the definition of data structures.
- *keyword:* an identifier for an enumeration
- *constant:* a numerical constant

Note: User defined *tags* and *keywords* must meet the rules for an ident (see 4.3 Name Spaces). Within the AML own name spaces are used. In this case it is allowed to reuse ASAM MCD-2 MC keyword names. The definitions from the AML are exclusively valid in IF_DATA. Outside IF_DATA only the keywords according to chapter 4.7 are valid.

Table 16 Grammar in the extended Backus-Naur format

Non-terminals	Production rule	Explanation
Declaration	type_definition "**;**" | block_definition "**;**"	definition of a data structure to be used for defining a data record of the description file.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Non-terminals	Production rule	Explanation
type_definition	type_name	type definition
type_name	predefined_type_name | struct_type_name | taggedstruct_type_name | taggedunion_type_name | enum_type_name	type name
predefined_type_name	"**char**" | "**int**" | "**long**" | "**int64**" | "**uchar**" | "**uint**" | "**uint64**" | "**ulong**" | "**double**" | "**float**"	predefined type name
block_definition	"**block**" *tag* member | "**block**" *tag* "**(**" member "**)***"	definition of a block. A block consists of a special begin keyword ("/begin"), a keyword identifying the record type (e.g. "FUNCTION_LIST"), the relevant data record and an end keyword ("/end"). Nested blocks are also possible.
enum_type_name	"**enum**" [*identifier*] "**{**" enumerator_list "**}**" | "**enum**" *identifier*	definition of an enumeration
enumerator_list	Enumerator | Enumerator "**,**" enumerator_list	list of enumeration values
Enumerator	*keyword* ["**=**" *constant*]	an enumeration, value range 32 Bit signed
struct_type_name	"**struct**" [*identifier*] "**{**" struct_member_list "**}**" | **"struct"** *identifier*	definition of data records of the ASAM MCD-2 MC description file with fixed sequence of the data record elements.
struct_member_list	struct_member | struct_member struct_member_list	list of structure members

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Non-terminals	Production rule	Explanation
struct_member	member "**;**"	member of the structure
member	type_name [array_specifier]	a member of a data type
array_specifier	"**[**" *constant* "**]**" | "**[**" *constant* "**]**" array_specifier	the length of an array
taggedstruct_type_name	"**taggedstruct**" [*identifier*] "**{**" taggedstruct_member_list "**}**" | "**taggedstruct**" *identifier*	definition of data records of the ASAM MCD-2 MC description file whose elements can specified in a random sequence. All elements are optional and each element is identified by its tag. For the description of lists with a variable number of elements, the symbols "(" and ")*" are used. The sequences identified by these symbols can be repeated any number of times.
taggedstruct_member_list	taggedstruct_member | taggedstruct_member taggedstruct_member_list	list of members of a taggedstruct
taggedstruct_member	taggedstruct_definition "**;**" | "**(**" taggedstruct_definition "**)*;**" | block_definition "**;**" "**(**" block_definition "**)*;**"	a member of a taggedstruct
taggedstruct_definition	*tag* [member] | *tag* "**(**" member "**)***"	an entry of a taggedstruct
taggedunion_type_name	"**taggedunion**" [*identifier*] "**{**" taggedunion_member_list "**}**" | "**taggedunion**" *identifier*	definition of variants in data records of the ASAM MCD-2 MC description file. Similar to the 'union' data type used in programming language C, the ASAM MCD-2 MC description file allows only one variant to be specified at a time in a 'taggedunion'. Each variant is assigned a

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Non-terminals	Production rule	Explanation
		tag for identification purposes (see *tag*).
taggedunion_member_list	tagged_union_member | tagged_union_member taggedunion_member_list	list of members of a taggedunion
tagged_union_member	*tag* [member] ";" | block_definition ";"	a member of a taggedunion

Hint: "(" <content> ")*;" describes a sequence with base type <content>. The "(" and ")*;" are not symbols of the BNF but terminals. The brackets indicate that for any instance the <content> can be repeated 0 till n times.

6.3 Designing AML-file

This chapter describes how to design an AML-file for interface-specific data.
To be compatible with ASAM MCD-2 MC a tag ”IF_DATA” must be defined in the AML-file. This tag is then used by the MCD tool to interpret the data that is written in the various IF_DATA-fields in the ASAM MCD-2 MC file.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

7 Terms and Definitions

For the purposes of this ASAM standard the following terms and definitions apply (in alphabetical order).

A2L File	File that contains an ASAP2 description
Adjustable Object	Different types of parameters where the values can be modified. See keywords CHARACTERISTIC, AXIS_PTS, BLOB, INSTANCE.
ASAM MCD-2 MC	This document, standard to describe ECU software for MC Tools.
ASAM MCD-2 MC metalanguage	Formal description language for ECU interfaces and additional, non-standardized software functionality.
ASAP2	Market name of ASAM MCD-2 MC
Conversion	Transformation between numerical values (ECU internal) and physical values (model world).
EPROM identifier	String in the data area of the ECU program that describes the program.
Fixed axis	Axis where the axis point values are not part of the ECU program but calculated by the MC Tool.
Function	For the structuring of ASAP2 projects according to the ECU software functionality.
Group	For the structuring of ASAP2 projects independent of the ECU software functionality
Group axis	The axis of a curve or map is separated from the output values of the curve or map and can be referenced by multiple curves or maps. Other names used in the field are: “Common axis”, “Shared axis”, “Referenced axis”.
Interface-specific data	Description of the ECU interface used to communicate with the MC Tool. See keyword IF_DATA (Example).
MC Tool	Measurement & Calibration software that is connected to the ECU and uses the A2L file to get information about the ECU and its interfaces.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

8 Symbols and Abbreviated Terms

A2L	ASAM MCD-2 MC language
AE	Automotive Electronics
AML	ASAM MCD-2 MC metalanguage
ASCII	American Standard for Character Information Interchange
CAN	Control Area Network
CCP	CAN Calibration Protocol
ECU	Electronic Control Unit
HW interface	Hardware interface, interface converter
IF	Interface
MCD	Measuring, Calibration and Diagnostics
ODX	Open Diagnostic data eXchange
ROM	Read-Only Memory
XCP	Universal Measurement and Calibration Protocol

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

9 Bibliography

[1] ASAM e.V.; CDF - Calibration Data Format V2.0.0; 2006

[2] ASAM e.V.; MDF - Measurement Data Format V4.1.0; 2012

[3] ASAM e.V.; FIBEX - Field Bus Data Exchange Format V4.1.1; 2013

[4] ASAM e.V.; MCD-3 MC API for Measurement and Calibration server V3.0; 2011

[5] ASAM e.V. ASAP3 Application Systems Interface Specification; 1999

[6] ASAM e.V.; ODX - Diagnostic Data Model Specification V2.2; 2008

[7] ASAM e.V.; Seed&Key and Checksum Calculation API V1.0

[8] ISO/DIS 9141; Road vehicles -- Diagnostic systems –

Part 1: Requirements for interchange of digital information, 1989

Part 2: CARB requirements for interchange of digital information; 1994

Part 3: Verification of the communication between vehicle and OBD II scan tool; 1998

[9] ISO/DIS 14229; Road vehicles -- Unified diagnostic services (UDS) – Part 1: Specification and requirements; 2006

[10] ISO/DIS 14230; Road vehicles -- Diagnostic systems -- Keyword Protocol 2000 – Part 1: Physical layer; 1999

Part 2: Data link layer; 1999

Part 3: Application layer; 1999

Part 4: Requirements for emission-related systems; 2000

[11] ISO/DIS 15765; Road vehicles -- Diagnostics on Controller Area Networks (CAN) – Part 1: General information; 2004

Part 2: Network layer services; 2004

Part 3: Implementation of unified diagnostic services (UDS on CAN); 2004

Part 4: Requirements for emissions-related systems; 2005

[12] ISO: ISO/DIS 22901-1; Road vehicles — Open diagnostic data exchange — Part 1: Data model specification; 2008

[13] ISO: ISO/DIS 22900-2; Road vehicles — Modular vehicle communication interface (MVCI) —Part 2: Diagnostic protocol data unit application programmer interface (D-PDU API); 2006

[14] ISO: ISO/DIS 22900-3; Road vehicles - Modular vehicle communication interface (MVCI) - Part 3: Diagnostic server application programming interface (D-Server API); 2007

[15] ASAM e.V.; MDX Meta Data Exchange Format for Software Module Sharing Version 1.2.0; 2013

[16] ASAM e.V.; Container Catalog XML Model Specification Version 3.0.0; 2004

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Appendix: A. IEEE-Floating-Point-Format

Normative

A.1. 32-Bit Format

Table 17 IEEE-Floating-Point-Format (32-Bit)

Sign	Biases Exponent					Significant						
S	e7	e6	...	e1	e0	B1	b2	b3	...	b21	b22	b23
31						23						0

Representation of real numbers: $(-1)^s * 2^E * b_{0\Delta} b_1 b_2 b_3 ... b_{23}$

s: 0 or 1
E: any integer between -126 and +127 (E = e - 127)
b_i: 0 or 1 (where $b_0 = 1$)

$$\text{RealNumber} = (-1)^s * 2^{(-127)+\sum_{i=0}^{7}(e_i * 2^i)} * \sum_{i=0}^{23}\left(\frac{b_i}{2^i}\right) \quad \text{where } b_0 = 1$$

A.2. 64-Bit Format

Table 18 IEEE-Floating-Point-Format (64-Bit)

Sign	Biases Exponent					Significant						
S	e10	e9	...	e1	e0	b1	b2	b3	...	b50	b51	b52
63						52						0

Representation of real numbers: $(-1)^s * 2^E * b_{0\Delta} b_1 b_2 b_3 ... b_{52}$

s: 0 or 1
E: any integer between -1022 and +1023 (E = e - 1023)
b_i: 0 or 1 (where $b_0 = 1$)

$$\text{RealNumber} = (-1)^s * 2^{(-1023)+\sum_{i=0}^{10}(e_i * 2^i)} * \sum_{i=0}^{52}\left(\frac{b_i}{2^i}\right) \quad \text{where } b_0 = 1$$

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Appendix: B. Using Reference Curves as Normalization Axes for Maps

Informative

B.1. Background

B.1.1. General

In certain calibration environments, the use of the same curves for normalization of two or more maps can save time during calibration and ROM storage while providing calibration control of resolution. A map specifies an axis as a CURVE_AXIS (See keywords AXIS_DESCR and CURVE_AXIS_REF) with a reference to the name of an existing curve defined elsewhere in the symbolic information. Typically, two such curves would be specified, one to normalize the map in the X direction, and one to normalize the map in the Y direction. Because of the interpolation within the curve and then in the map, the number of pairs in the curve does not have to match the number of points along the corresponding map axis.

B.1.2. Overall technique

Two curves are needed; one for the map X direction and one for the map Y direction. The X values for the curves must be ordered such that their values are always staying the same or increasing. The output of each curve are indices used to determine four cells within a map, whose values are interpolated to result in a single Z value. Since the output of a curve is a floating-point value, the value is truncated to determine one cell in the map, with the adjacent cell one larger. The interpolation is done twice in the X direction (once for each row) resulting in two Z values. These two Z values are then interpolated to result in a final Z value. The interpolation uses the identical-slope method, where for two existing known points, (X1,Y1) and (X2,Y2), the corresponding Y value for a given X value that lies on the line between the two points is given by: Y = Y1 + (X-X1)((Y2-Y1)/(X2-X1))

B.1.3. Determining the map indices

When the input value to a reference curve is less than or equal to the lowest X value, the result is set to the Y value corresponding to the lowest X value. When the input value to a curve is greater than or equal to the highest X value, the result is set to the Y value corresponding to the highest X value. When the input value matches an X value, the result is set to the corresponding Y value. Otherwise, the result is determined by interpolation between the two adjacent pairs whose X values form boundaries around the input value.

B.1.4. Determining the map normalized value

The map cells are artificially numbered with integers beginning with 0 in both the X and Y direction. Two floating-point values are obtained from two curves, one for the X direction and one for the Y direction.
If the X direction floating-point value is less than 0, then the corresponding two column indices are set to 0. Otherwise the first column index is set to the whole number portion of the value and the second column index is set to one larger. Now, if the second column index is greater than or equal to the number of columns in the map, the first and second column indices are set to the number of columns minus 1.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

If the Y direction floating-point value is less than 0, then the corresponding two row indices are set to 0. Otherwise the first row index is set to the whole number portion of the value and the second row index is set to one larger. Now, if the second row index is greater than or equal to the number of rows in the map, the first and second row indices are set to the number of rows minus 1.
The intersection of the rows and columns determine four cells whose values are interpolated to result in a single Z value. The two values at the intersecting columns of one row are interpolated to obtain an intermediate result, and likewise for the other row. The two intermediate results are then interpolated in the Y direction to come up with the final Z result.

B.2. Example

The following example shows a map (Z_MAP) with 7 columns and 6 rows. The normalization curve for the column is a referenced curve (X_NORM) that contains 5 pairs. The normalization curve for the row is a referenced curve (Y_NORM) that contains 4 pairs. The number of pairs for each normalizing curve does not match the number of row and columns in the map. An input value of 850.0 into X_NORM produces a Y output of 3.9, and an input value of 60.0 for Y_NORM produces a Y output of 1.7. The four points in the table to interpolate would be the intersection of columns 3 and 4, and rows 1 and 2. First the two points in row 1, columns 3 and 4 (5.6, 3.2) are interpolated to get a value of 3.44 . Then the two points in row 2, column 3 and 4 (2.2, 1.6) are interpolated to get a value of 1.66. These two resulting values are then interpolated to produce a final Z value of 2.194.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

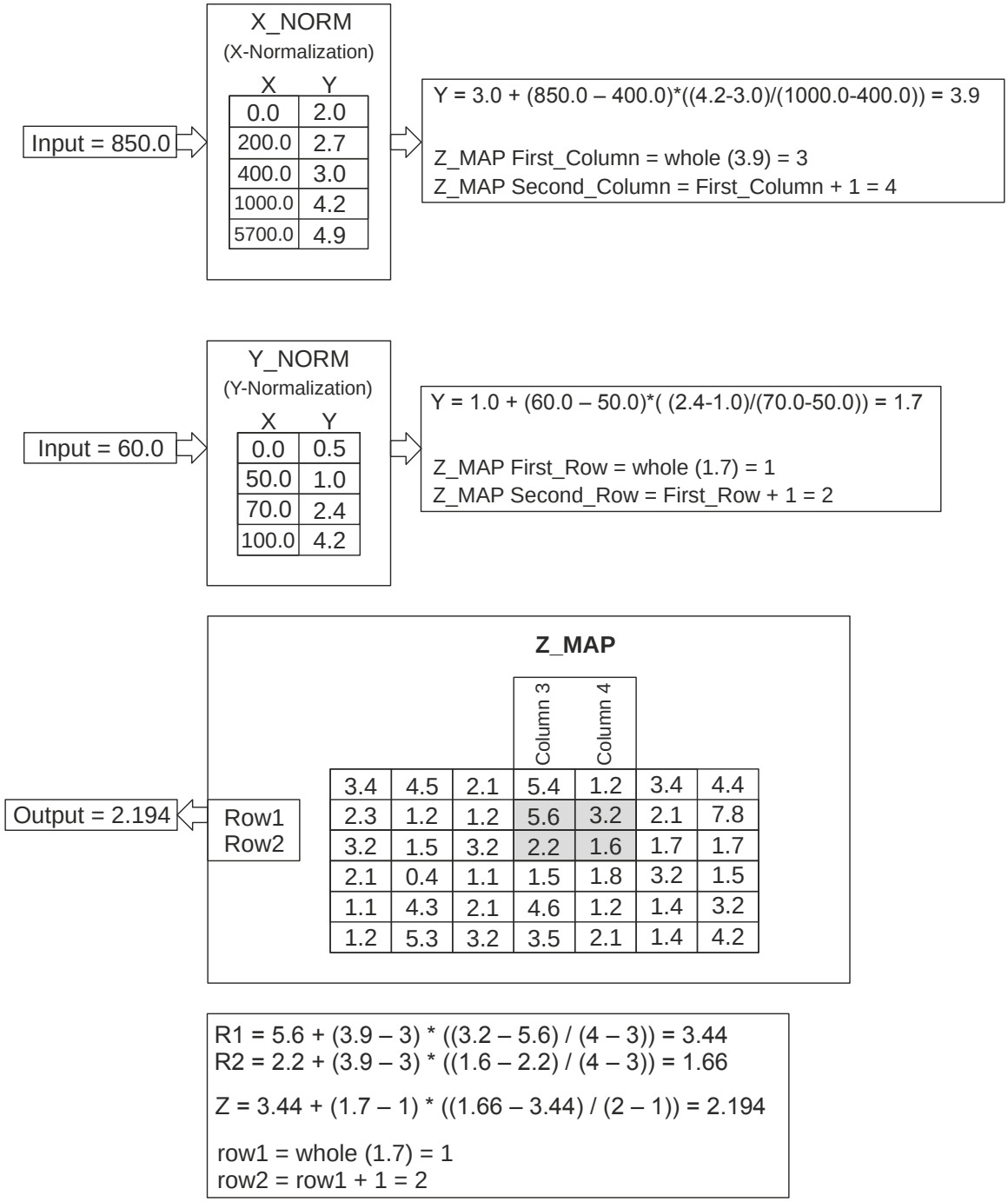


Figure 10 Map with referenced normalization curves

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Appendix: C. Codes for scaling units (CSE)

Normative

Table 19 contains the values for scaling units, based on [15].
The resulting measurement rate will be evaluated as product of the Base unit and the Rate.

Table 19 Codes for scaling units (CSE)

ScalingUnit	Base unit	Comment
0	1 µsec	Time
1	10 µsec	Time
2	100 µsec	Time
3	1 msec	Time
4	10 msec	Time
5	100 msec	Time
6	1 sec	Time
7	10 sec	Time
8	1 min	Time
9	1 hour	Time
10	1 day	Time
20	1 fs (femto second)	Time
21	10 fs (femto second)	Time
22	100 fs (femto second)	Time
23	1 ps (pico second)	Time
24	10 ps (pico second)	Time
25	100 ps (pico second)	Time
26	1 ns (nano second)	Time
27	10 ns (nano second)	Time
28	100 ns (nano second)	Time
100	Angular degrees	Angle Rate defines the number of degrees (360°) after that a trigger occurs
101	Revolutions 360 degrees	Revolutions Rate defines the number of revolutions after that a trigger occurs
102	Cycle 720 degrees	Cycles e.g. in case of IC engines Rate defines the number of cycles (cycle of 4-stroke cycle

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

ScalingUnit	Base unit	Comment
		engine) after that a trigger occurs
103	Cylinder segment	Combustion e.g. in case of IC engines Rate defines the cylinder number (Combustion) the trigger is related to
1000	Non deterministic	Without fixed scaling, The value from RATE is not interpreted and the Rate is set to "0"

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Appendix: D. Transformer

Normative

D.1. Basics

In order to be able to handle highly optimized, non human readable ECU calibration control data within an application system, an additional level of representation of this data is introduced. This representation is used by the application engineer to read and calibrate only. If the ECU control algorithm uses an update of this representation a transformation into the ECU control algorithm representation (a BLOB) has to be done first.

The human readable representation is modeled with the already existing A2L objects – namely CHARACTERISTICs and related (e.g. AXIS_PTS). Although this CHARACTERISTICs are not available as instances in the ECU's memory, they are available in the MC tool's data base only. This approach has several advantages:

- Calibration and Calibration data merging can be done on the level of the human readable representation. This also limits the number of tools which have to support the transformation between the two representations
- In order to avoid tool runtime issues (e.g. usability on GUI level when displaying such a specific characteristic) the trigger for a transformation has to be done only in case of a change of the characteristic AND its need to update it in the ECU's memory. Otherwise the value of the characteristic can be taken from the tools data base. This approach can also be considered as an offline calibration process but with still available online calibration.
- The impact to application tools is considered to be minimized as already available mechanisms are used

In order to be able to distinguish between calibration objects available in the ECU's memory and objects available in the tools data base only, these objects are grouped within a MEMORY_SEGMENT with a PrgType NOT_IN_ECU. This attribute value of parameter enum PrgType could be considered as a counterpart approach of PrgType CALIBRATION_VARIABLES.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

The following figure illustrates this approach.

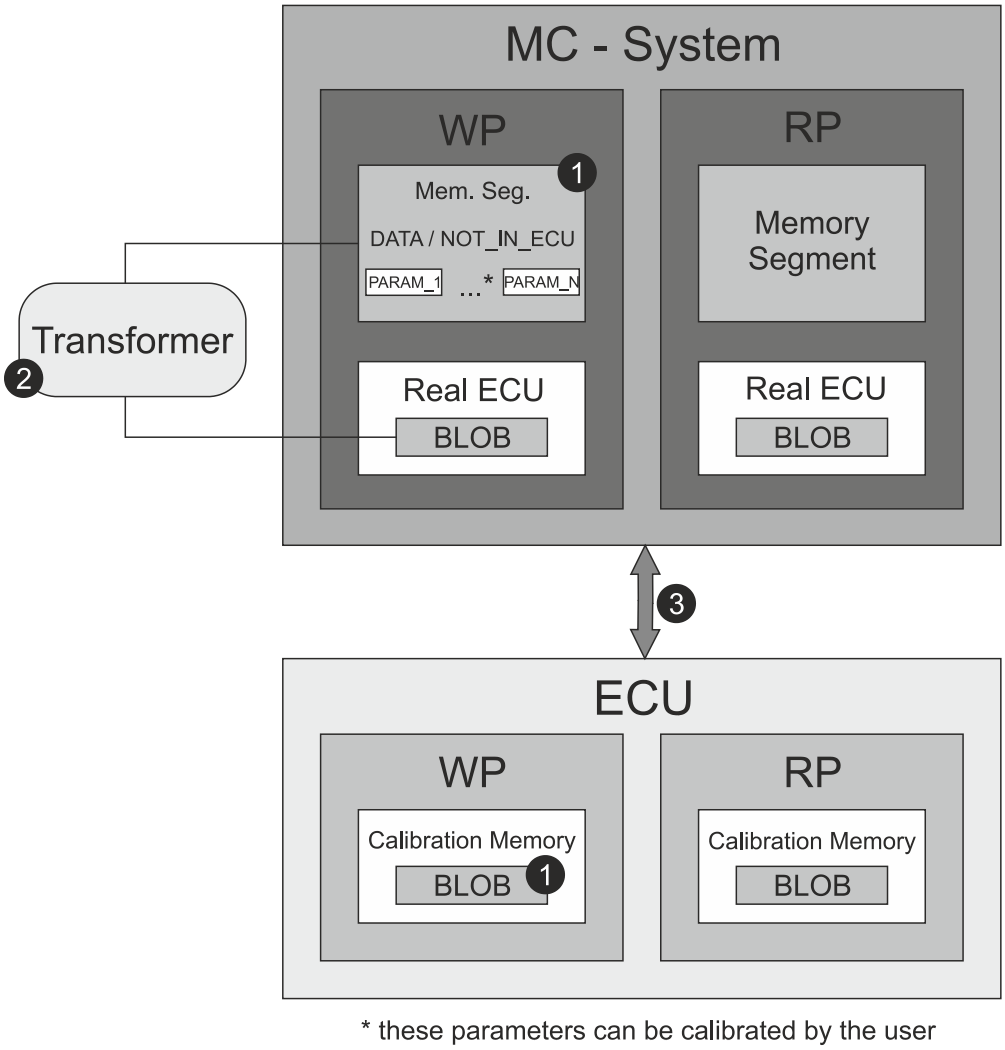


Figure 11 Workflow between BLOB's and Transformer

Workflow (Transformation)

❶ Trigger by user, value change or tool init

[❸] Synchronization between ECU and MC-System
- Upload

❷ Bidirectional Transformation
- Update of Memory BLOB by Virtual Memory Segment Data or
- Update of Virtual Memory Segment Data by Memory BLOB

[❸] Synchronization between MC-System and ECU
- Download

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

D.2. Transformer DLL API definition and rules

D.2.1. General API description

The transformer DLL is implemented as a standard MS Windows DLL conforming to the CDECL calling convention. Depending on the used MC tools the DLL must be available as 32/64 bit DLL. All used strings in the API are Unicode encoded.
If the DLL has further dependencies to any other files it has to check if the dependencies are fulfilled. This check needs to work with only the DLL present.
As the MC tool needs some basic information about the DLL and the implemented transformation algorithm the DLL provides an information record. The information record is evaluated by the MC tool before using the transformation algorithm.
If the MC tool detects that the DLL is not compatible with the currently used A2L or the MC tool itself it raises an error and provides an error text.
The compatibility check is executed by the DLL method `IsTransformationPossible()`.
To transfer the input data to the transformer DLL and to receive the output data from the DLL the CDF file format is used. As big amounts of data might be exchanged between the DLL and the MC tool the input data is written to a file. Only the file name is passed to the DLL. The DLL returns the output data also in a file. The MC tool is responsible to create the input file and generate an output file name the DLL has to use. The input and output file names are generated as URLs. After the DLL computed the result and delivered it to the MC tool the MC tool is responsible to clean up any generated files (including the file used as output file from the DLL).
If problems occur while the DLL is running the created output file is invalid and an error code and message is provided. A mismatch between the output file and OUT_OBJECTS is considered as an error. The same applies for the input file and IN_OBJECTS.

Note: To reduce the dependencies to other libraries it is recommended to link statically against these.

Note: All string parameters shall be handled case sensitive.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

D.2.2. Detailed C transformer API definition

D.2.2.1. Typedefs

All structures which are used in the interface between the transformer DLL and the MC-System shall not have alignment gaps.

```
typedef enum TransformerStatus
{
        OK                      = 0,
        ILLEGAL_ARGUMENTS       = 1,
        TRANSFORMATION_FAILED   = 2,
        VERSION_NOT_SUPPORTED   = 3
} TransformerStatus;
```

Note: The enumeration TransformerStatus can be used inside the DLL. As the size of an enumeration is not uniquely defined the DLL interface uses unsigned short to transfer the enumeration values.

Table 20 Members of TransformerStatus

Name	Description
OK	transformation was successful and the output data from the DLL can be used by the MC tool
ILLEGAL_ARGUMENTS	The DLL was called with the wrong arguments.
TRANSFORMATION_FAILED	The requested transformation failed.
VERSION_NOT_SUPPORTED	The requested transformation is not supported by the transformer

```
typedef struct _TransformerDescription
{
        unsigned short    StructLength;
        unsigned short    MaxIdentLength;
        unsigned short    MaxStringLength;
        wchar_t           Id[MAX_IDENT+1];
        wchar_t           Version[MAX_STRING+1];
        wchar_t           Description[4096];
} TransformerDescription;
```

Table 21 Parameter structure TransformerDescription

Name	Description
StructLength	Length in byte of this structure. Required: 10762 (2 + 2 + 2 + 2*(MAX_IDENT+1) + 2*(MAX_STRING+1) + 2*4096)
MaxIdentLength	Value of MAX_IDENT
MaxStringLength	Value of MAX_STRING
Id	Unique name of the transformer as a null terminated string. It is the same name as

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

	the one in the A2L in the TRANSFORMER keyword.
Version	Implemented version of the transformer as a null terminated string. The version comparison is case sensitive.
Description	A short description of what the transformer does as a null terminated string

Note: The members MaxIdentLength and MaxStringLength are needed to check for compatibility in case the values of MAX_IDENT or MAX_STRING change in future versions of ASAM MCD-2 MC.

```
typedef struct DllDescription
{
        unsigned short   StructLength;
        wchar_t          SupportedCDFVersion[10];
        unsigned short   NumberOfTransformers;
} DllDescription;
```

Table 22 Parameter structure DllDescription

Name	Description
StructLength	Length in byte of this structure. Required: 24 (2 + 2*10 + 2)
SupportedCDFVersion	CDF format version which is supported by the transformers in the DLL as a null terminated string. The string has the same format and content as the root category in the CDF specification. Both CDF files have to be conforming to this CDF version.
NumberOfTransformers	Overall number of transformers implemented in the DLL.

```
typedef struct TransformationData
{
        unsigned short   StructLength;
        const wchar_t*   TransformerId;
        const wchar_t*   TransformerResourcesPathUrl;
        const wchar_t*   CdfInputUrl;
        const wchar_t*   CdfOutputUrl;
} TransformationData;
```

Table 23 Parameter structure TransformationData

Name	Description
StructLength	Length in byte of this structure. Required for 32Bit: 18 (2 + 4 + 4 + 4 + 4) Required for 64Bit: 34 (2 + 8 + 8 + 8 + 8)

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

TransformerId	Name of the transformer as null terminated string. It has to be the same name as returned in “TransformerDescription.Id“ and has to match the name from A2L.
TransformerResourcesPathUrl	URL of the path where the transformer resource (configuration) files are located (provided by the MC tool). The called transformer uses it as a pointer to a null terminated string. If there are no resource files an empty string is used.
CdfInputUrl	URL of the input file (provided by the MC tool) for the called transformer as a pointer to a null terminated string.
CdfOutputUrl	URL of the output file (generated by the transformer) for the called transformer as a pointer to a null terminated string.

```
typedef struct TransformerResult
{
        unsigned short     StructLength;
        unsigned short     Status;
        wchar_t            Message[4096];
} TransformerResult;
```

Table 24 Parameter structure TransformerResult

Name	Description
StructLength	Length in byte of this structure. Required: 8196 (2 + 2 + 2*4096)
Status	Result of the executed transformation Note: Values related to enumeration `TransformerStatus`
Message	Detailed error message if “status” is not equal to “OK”.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

D.2.2.2. IsTransformationPossible DLL Function

Function Call:

```
unsigned short      IsTransformationPossible
                    ( const wchar_t*          TransformerId,
                      const wchar_t*          TransformerVersion);
```

Parameters:

const wchar_t*	TransformerId	Name of the transformer as null terminated string.
const wchar_t*	TransformerVersion	Version information as defined at A2L keyword TRANSFORMER.

Return Value:

unsigned short	1, if the transformer can do the transformation, otherwise 0

Functional Description:

Check if the transformer can do the transformation corresponding to the A2L file entry.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

D.2.2.3. GetTransformerDescription DLL Function

Function Call:

```
void                GetTransformerDescription
                    ( unsigned short          Index,
                      TransformerDescription  *Description,
                      TransformerResult       *Result);
```

Parameters:

unsigned short	Index	The wanted transformer is selected by Index.
TransformerDescription	*Description	Description of the selected transformer
TransformerResult	*Result	Information about the success of the DLL function

Return Value:

Nothing

Functional Description:

Returns in the TransformerDescription a detailed description of an implemented transformer.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

D.2.2.4. GetDllDescription DLL Function

Function Call:

```
void                GetDllDescription
                    ( DllDescription          *Description,
                      TransformerResult       *Result);
```

Parameters:

DllDescription	*Description	Description of the DLL content
TransformerResult	*Result	Information about the success of the DLL function

Return Value:

Nothing

Functional Description:

Returns in the DllDescription a detailed description of the currently loaded transformer DLL.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

D.2.2.5. ExecuteTransformation DLL Function

Function Call:

```
void                ExecuteTransformation
                    ( TransformationData      *TransformationData,
                      TransformerResult       *Result);
```

Parameters:

TransformationData	* TransformationData	Description of the transformation
TransformerResult	*Result	Information about the success of the DLL function

Return Value:

Nothing

Functional Description:

Execute the transformation requested in TransformationData and return the result.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Appendix: E. ASAP2 Container

Normative

To work with automotive ECU it is necessary to handle different description-, result-, documentation-, data-files. All files are normally versioned and related on each other. To help the calibration engineers it is necessary to support an easy way to handle the sorting, storing and exchange of complete sets of files. As these files are normally huge a compression is helpful.

E.1. Container

As solution a file container with a structured order is needed. As container format a standard ZIP container is used. All files are copied into the container. The file can be stored together in one directory or structured in subdirectories. In the last case the references in the catalogue must include this path also. The container description based on [16].

The intension is that one container describes one ECU. Nevertheless it is possible to add different software revisions or data variants into one container.

Note: If the container includes the path it is allowed to use the path in the FILES references. This may be used if multiple files with the same name (but different paths) are transported. The path should be relative, not absolute.

The extension for this container file is "**.ECUZIP"**.

The container includes exactly one container catalogue file (root folder of the archive). The name of the container catalogue file is **“EcuContainer.info”**. This container will be denoted as in this **ECU-Container** document.

The container needs to be consistent internally.

Note: File names are not case sensitive but references to files shall be case sensitive.

E.2. Common

As the ASAM CC specification describes a generic approach for exchange of engineering data and its configuration the following chapters describe the mapping to the MC use cases and the relevant data items.

In opposite to the ASAM CC specification as encoding for the catalogue XML file UTF 8 must be used.

A catalogue file for an ECU-Container consists of the elements ABLOCKS, AREFS, ABLOCK, AREF, FILES, FILE, CATEGORY and SHORT-NAME. Attributes, especially the attribute UPD need not be evaluated or processed from a tool.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

E.3. Category

For the MC use cases the following items CATALOG and ABLOCK must have a category sub tag. The possible content and their semantic are described in the following tables.

Table 25 Categories for <CATALOG>

CATEGORY	Catalog use case
ECU-FILE-CONTAINER	Identifier for this kind of container

Table 26 Categories for <ABLOCK>

CATEGORY	File types	Operating tool expects:
ANY	Unspecific	
A2L	ASAM MCD-2MC file, including AML	ECU software description
MAP	MAP, ELF file, …	ECU software description
BUS	FIBEX,CanDB, …	Bus description
HEX	specifies a common binary file that contains ECU code and data (may be a .s19, .hex, .bin, … file)	ECU software binary
DATA	specifies a common data exchange file (may be a .DCM, .CVX, .XML, .CDFX, … file)	ECU parameter (physical values)
DIAG	Files with diagnostic description (may be .odx, .pdx, … file)	ECU diagnostic description
FLASH	Files with flash information (may be a ,cfg, … file)	ECU flash description
SEED_KEY	Files with ECU protection information (may be a seed&key, … file)	DLL files used for Seed&Key
CHECKSUM	Files with checksum calculation (DLL)	DLL files for checksum
TRANSFORMER	Transformer DLL	Calculate transformed Parameters
CONFIGURATION	Configuration Files	Configuration files (e.g. for Transformer) that are linked to the “main” file via ABLOCK
LIBRARY	Generic DLL	DLL files for generic purpose
DOCU	specifies a common ECU documentation file (may be a .doc, .txt, .xls, … file)	Help files that can be shown to the calibration engineer
LINK	defines the dependency between files	File references

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

E.4. ABLOCK

The ABLOCK uses either FILES or AREFS.

The FILE tag contains references to the files in the ZIP container.

For the categories LINK an ABLOCK must only consist of AREFs. All other category values show the operating tool how the file or files has to be used.

Note: If files describe different variants for each of them a separate ABLOCK is necessary.

E.5. FILES

It is allowed to have multiple FILE references in one ABLOCK. In this case the operating tool has to use the files as a complete description of the item.

Note: For CATEGORY A2L this means that the first FILE is the master and all other are used as e.g. includes

For CATEGORY DATA this means that the different FILE must not describe the same parameter

For CATEGORY HEX this means that the different FILE must not describe the same address ranges

Note: If multiple FILE are used the tool shall use it in the sequence listed in the container (e.g. copy 1. FILE, then 2. FILE)

E.6. AREF

The AREF references in one ABLOCK define a link between the ABLOCKs which describes on or more file references.

Note: From the MC tool point of view each ABLOCK with the CATEGORY value ECU describes a consistent variant of data which can be used to interact with the device. To support also a more generic approach the CATEGORY value LINK is specified. In this case the operating tool must detect the semantic from the file categories of the referenced files.

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

E.7. Example for Container Catalog File

The following set of files is given

- ECU_Software_V01.a2l
- ECU_Software_V02.a2l
- ECU_Software_V03.a2l
- ECU_Software_V04.a2l
- ECU_Data_Part01.hex (belongs to ECU_Software_V01.a2l)
- ECU_Data_Part02.hex (belongs to ECU_Software_V01.a2l)
- ECU_Data_V03.hex (belongs to ECU_Software_V02.a2l)
- ECU_Data_V03.s19 (belongs to ECU_Software_V03.a2l)
- ECU_Data_V04.s19 (belongs to ECU_Software_V04.a2l)
- ECU_Data_Part_1.cdfx (belongs to ECU_Data_V03.hex)
- ECU_Data_Part_2.cdfx (belongs to ECU_Data_V03.hex)
- ECU_Flash_Description.odx (belongs to ECU, V01 and V02)
- ECU_SeedKey.dll (belongs to ECU, V01 and V02)
- FIM_TRANSFORMER.dll (belongs to ECU, V01 and V02)
- FIM_CONFIGURATION.dll (belongs to FIM_TRANSFORMER.dll)
- ECU_Docu.html (belongs to ECU, V01 and V02)
- ECU_Docu.pdf (belongs to ECU, V01 and V02)

```
<CATALOG>
<SHORT-NAME>My ASAP2 Container</SHORT-NAME>
<CATEGORY>ECU-FILE-CONTAINER</CATEGORY>
<ABLOCKS>
      <ABLOCK>
            <!-- the short name may be the same as the file name -->
            <!-- the short name shall follow the ident rules -->
            <SHORT-NAME>ECU_Software_V01</SHORT-NAME>
            <CATEGORY>A2L</CATEGORY>
            <FILES>
                  <FILE> ECU_Software_V01.a2l</FILE>
            </FILES>
      </ABLOCK>
      <ABLOCK>
            <!-- the short name may have a generic name -->
            <SHORT-NAME> ECU_A2L_Description</SHORT-NAME>
            <CATEGORY>A2L</CATEGORY>
            <FILES>
                  <FILE> ECU_Software_V02.a2l</FILE>
            </FILES>
      </ABLOCK>
      <ABLOCK>
            <SHORT-NAME>ECU_Software_V03</SHORT-NAME>
            <CATEGORY>A2L</CATEGORY>
            <!-- optional ADMIN-DATA removed -->
            <FILES>
                  <FILE> ECU_Software_V03.a2l </FILE>
            </FILES>
      </ABLOCK>
      <ABLOCK>
            <SHORT-NAME> ECU_Software_V04</SHORT-NAME>
            <CATEGORY>A2L</CATEGORY>
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
      <FILES>
            <FILE> ECU_Software_V04.a2l</FILE>
      </FILES>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME> ECU_Data_V01</SHORT-NAME>
      <CATEGORY>HEX</CATEGORY>
      <FILES>
            <!-- multiple files possible as there are no
      references to the single files -->
            <!-- both files are used together, e.g. code and
      data part -->
            <FILE> ECU_Data_Part01.hex</FILE>
            <FILE> ECU_Data_Part02.hex</FILE>
      </FILES>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>ECU_Data_V03</SHORT-NAME>
      <CATEGORY>HEX</CATEGORY>
      <FILES>
            <FILE> ECU_Data_V03.hex</FILE>
      </FILES>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>ECU_Data_V03</SHORT-NAME>
      <CATEGORY>HEX</CATEGORY>
      <FILES>
            <FILE>ECU_Data_V03.s19</FILE>
      </FILES>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>ECU_Data_V04</SHORT-NAME>
      <CATEGORY>HEX</CATEGORY>
      <FILES>
            <FILE>ECU_Data_V04.s19</FILE>
      </FILES>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>ECU_CDF</SHORT-NAME>
      <CATEGORY>DATA</CATEGORY>
            <!-- both files are used together, e.g. SW function
      1 and 2 -->
      <FILES>
            <FILE>ECU_Data_Part_1.cdfx</FILE>
            <FILE>ECU_Data_Part_2.cdfx</FILE>
      </FILES>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>ECU_Flash_Description</SHORT-NAME>
      <CATEGORY>DIAG</CATEGORY>
      <FILES>
            <FILE>ECU_Flash_Description.odx</FILE>
      </FILES>
</ABLOCK>
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
<ABLOCK>
      <SHORT-NAME>ECU_SEED_KEY</SHORT-NAME>
      <CATEGORY>SEED_KEY</CATEGORY>
      <FILES>
            <FILE> ECU_SeedKey.dll</FILE>
      </FILES>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>FIM_V01</SHORT-NAME>
      <CATEGORY>TRANSFORMER</CATEGORY>
      <FILES>
            <FILE>FIMV01.dll</FILE>
      </FILES>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>FIMV0101_CONFIGURATION</SHORT-NAME>
      <CATEGORY>CONFIGURATION</CATEGORY>
      <FILES>
            <FILE>FIMCFGV01ECU1.cfg</FILE>
      </FILES>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>ECU_Docu</SHORT-NAME>
      <CATEGORY>DOCU</CATEGORY>
      <FILES>
            <FILE>ECU_Docu.html</FILE>
            <FILE>ECU_Docu.pdf</FILE>
      </FILES>
</ABLOCK>
<!-- start of references -->
<ABLOCK>
      <SHORT-NAME>Reference_01</SHORT-NAME>
      <CATEGORY>LINK</CATEGORY>
      <AREFS>
            <AREF>ECU_Software_V01</AREF>
            <AREF>ECU_Data_V01</AREF>
      </AREFS>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>Reference_02</SHORT-NAME>
      <CATEGORY>LINK</CATEGORY>
      <AREFS>
            <AREF>ECU_Software_V02</AREF>
            <AREF>ECU_Data_V03</AREF>
      </AREFS>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>Reference_03</SHORT-NAME>
      <CATEGORY>LINK</CATEGORY>
      <AREFS>
            <AREF>ECU_Software_V03</AREF>
            <AREF>ECU_Data_V03</AREF>
      </AREFS>
</ABLOCK>
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
<ABLOCK>
      <SHORT-NAME>Reference_04</SHORT-NAME>
      <CATEGORY>LINK</CATEGORY>
      <AREFS>
            <AREF>ECU_Software_V04</AREF>
            <AREF>ECU_Data_V04</AREF>
      </AREFS>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>Reference_05</SHORT-NAME>
      <CATEGORY>LINK</CATEGORY>
      <AREFS>
            <AREF>ECU_Software_V05</AREF>
            <AREF>ECU_Data_V05</AREF>
      </AREFS>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>Reference_06</SHORT-NAME>
      <CATEGORY>LINK</CATEGORY>
      <AREFS>
            <AREF>ECU_Data_V03</AREF>
            <AREF>ECU_CDF</AREF>
      </AREFS>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>Reference_07</SHORT-NAME>
      <CATEGORY>LINK</CATEGORY>
      <AREFS>
            <AREF>ECU_Software_V01</AREF>
            <AREF>ECU_Software_V02</AREF>
            <AREF>ECU_Flash_Description</AREF>
      </AREFS>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>Reference_08</SHORT-NAME>
      <CATEGORY>LINK</CATEGORY>
      <AREFS>
            <AREF>ECU_Software_V01</AREF>
            <AREF>ECU_Software_V02</AREF>
            <AREF>ECU_SEED_KEY</AREF>
      </AREFS>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>Reference_09</SHORT-NAME>
      <CATEGORY>LINK</CATEGORY>
      <AREFS>
            <AREF>ECU_Software_V01</AREF>
            <AREF>FIM_V01</AREF>
      </AREFS>
</ABLOCK>
<ABLOCK>
      <SHORT-NAME>Reference_10</SHORT-NAME>
      <CATEGORY>LINK</CATEGORY>
      <AREFS>
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015


```
                <AREF>FIM_V01</AREF>
                <AREF>FIMV0101_CONFIGURATION</AREF>
          </AREFS>
     </ABLOCK>
     <ABLOCK>
          <SHORT-NAME>Reference_11</SHORT-NAME>
          <CATEGORY>LINK</CATEGORY>
          <AREFS>
                <AREF>ECU_Software_V03</AREF>
                <AREF>ECU_Software_V04</AREF>
                <AREF>ECU_Docu</AREF>
          </AREFS>
     </ABLOCK>
</ABLOCKS>
</CATALOG>
```

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Index of Keywords and Enum Values

A_INT64 ..... **31**
A_UINT64 ..... **31**
A2ML **34**, 171
A2ML_VERSION **35**
ABSOLUTE **106**
ADDR_EPK **36**, 168
ADDRESS_TYPE **37**, 65, 156, 229, 232, 234
ALIGNMENT_BYTE **38**, 166, 189
ALIGNMENT_FLOAT32_IEEE **39**, 166, 189
ALIGNMENT_FLOAT64_IEEE **40**, 166, 189
ALIGNMENT_INT64 ..... **41**, 166, 189
ALIGNMENT_LONG **42**, 166, 189
ALIGNMENT_WORD **43**, 166, 189
ALTERNATE_CURVES **122**
ALTERNATE_WITH_X **122**
ALTERNATE_WITH_Y **123**
ANNOTATION **44**, 50, 54, 65, 73, 131, 135, 145, 156
ANNOTATION_LABEL 44, **45**
ANNOTATION_ORIGIN 44, **46**
ANNOTATION_TEXT 44, **47**
ARRAY_SIZE **48**, 156
ASAP2_VERSION **49**
ASCII **74**
AXIS_DESCR **50**, 73, 230
AXIS_PTS **54**, 171
AXIS_PTS_REF 50, **58**
AXIS_PTS_X/_Y/_Z/_4/_5 ..... **60**, 189
AXIS_RESCALE_X **61**
AXIS_RESCALE_X/_Y/_Z/_4/_5 189
BIG_ENDIAN **31**
BIT_MASK **63**, 73, 156, 230, 232
BIT_OPERATION **64**, 156, 232
BLOB **65**, 171
BYTE **31**
BYTE_ORDER 50, 54, **67**, 73, 156, 166, 227, 230, 232
CALIBRATION **69**
CALIBRATION_ACCESS 54, 65, **69**, 73, 145
CALIBRATION_HANDLE **70**, 72
CALIBRATION_HANDLE_TEXT 70, **71**
CALIBRATION_METHOD **72**, 168
CALIBRATION_VARIABLES **162**
CHARACTERISTIC **73**, 171
CODE **162**
COEFFS **79**, 82
COEFFS_LINEAR **80**, 82
COLUMN_DIR **123**, 148
COM_AXIS **50**
COMPARISON_QUANTITY 73, **81**
COMPU_METHOD **82**, 171
COMPU_TAB **86**, 171
COMPU_TAB_REF 82, **88**
COMPU_VTAB **89**, 171
COMPU_VTAB_RANGE **91**, 171
CONSISTENT_EXCHANGE **93**, 234
CONVERSION **94**, 180
CPU_TYPE **95**, 168
CUBE_4 ..... **74**
CUBE_5 ..... **74**
CUBOID **74**
CURVE **74**
CURVE_AXIS **50**
CURVE_AXIS_REF 50, **96**
CUSTOMER **98**, 168
CUSTOMER_NO **99**, 168
DATA **162**
DATA_SIZE **100**, 166
DEF_CHARACTERISTIC **101**, 131
DEFAULT_VALUE 86, 89, 91, **102**
DEFAULT_VALUE_NUMERIC 86, **103**
DEPENDENT_CHARACTERISTIC 73, **104**
DEPOSIT 50, 54, **106**, 166, 227
DERIVED **237**
DIFFERENCE **106**
DIRECT **31**
DISCRETE 73, **107**, 156, 230, 232
DISPLAY_IDENTIFIER 54, 65, 73, **108**, 145, 156
DIST_OP_X/_Y/_Z/_4/_5 ..... **109**, 189
ECU **110**, 168
ECU_ADDRESS **111**, 156
ECU_ADDRESS_EXTENSION 54, 65, 73, **112**, 145, 156
ECU_CALIBRATION_OFFSET **113**, 168
EEPROM **162**
ENCODING 73, **114**, 230
EPK **115**, 168
EPROM **162**
ERROR_MASK **116**, 156, 232
EXCLUDE_FROM_FLASH **162**
EXTENDED_LIMITS 50, 54, 73, **117**, 180, 227, 230
EXTENDED_SI **237**
EXTERN **163**
FIX_AXIS **50**
FIX_AXIS_PAR 50, **118**

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

FIX_AXIS_PAR_DIST 50, **119**
FIX_AXIS_PAR_LIST..... 50, **120**
FIX_NO_AXIS_PTS_X/_Y/_Z/_4/_5.....
..... **121**, 189
FLASH **162**
FLOAT32_IEEE..... **31**
FLOAT64_IEEE..... **31**
FNC_VALUES..... **122**, 189
FORM **82**
FORMAT.....
... 50, 54, 73, **124**, 156, 180, 227, 230, 232
FORMULA..... 82, **125**
FORMULA_INV..... 125, **128**
FRAME **129**, 171
FRAME_MEASUREMENT 129, **130**
FUNCTION..... **131**, 171
FUNCTION_LIST 54, 73, **133**, 135, 156
FUNCTION_VERSION..... 131, **134**
GROUP..... **135**, 171
GUARD_RAILS..... 54, 73, **139**
HEADER..... **140**, 185
IDENTICAL **82**
IDENTIFICATION..... **141**, 189
IF_DATA .....
54, 65, 73, 129, 131, 135, 145, 156, 160, 162, 171
IF_DATA (Example) **142**
IN_MEASUREMENT..... 131, **143**
INDEX_DECR **31**
INDEX_INCR **31**
INPUT_QUANTITY **144**, 180
INSTANCE..... **145**, 171
INTERN..... **162**
LAYOUT.....145, **148**, 156, 214, 232
LEFT_SHIFT 64, **149**
LIMITS **150**, 180
LINEAR **82**
LITTLE_ENDIAN..... **31**
LOC_MEASUREMENT 131, **151**
LONG..... **31**
MAP **74**
MAP_LIST..... 73, **152**
MATRIX_DIM .....
..... 73, 145, **153**, 156, 214, 230, 232
MAX_GRAD..... 50, **154**
MAX_REFRESH .....
.....54, 65, 73, 145, **155**, 156
MEASUREMENT **156**, 171
MEMORY_LAYOUT..... **160**, 168
MEMORY_SEGMENT..... **162**, 168
MOD_COMMON **166**, 171
MOD_PAR **168**, 171
MODEL_LINK .....
.....54, 65, 73, 145, 156, **170**
MODULE **171**, 185
MON_DECREASE.....**173**
MON_INCREASE **173**
MONOTONOUS **173**
MONOTONY..... 50, 54, **173**, 180, 227
MSB_FIRST.....**31**, **67**
MSB_FIRST_MSW_LAST **31**, **67**
MSB_LAST **31**, **67**
MSB_LAST_MSW_FIRST **31**, **67**
NO_AXIS_PTS_X/_Y/_Z/_4/_5....**174**, 189
NO_CALIBRATION..... **69**
NO_OF_INTERFACES 168, **175**
NO_RESCALE_X 189
NO_RESCALE_X **176**
NOT_IN_ECU **162**
NOT_IN_MCD_SYSTEM **69**
NOT_MON.....**173**
NUMBER 73, **177**, 230
NUMERIC.....**248**
OFFLINE_CALIBRATION **69**
OFFLINE_DATA **162**
OFFSET_X/_Y/_Z/_4/_5 .....**178**, 189
ON_CHANGE **221**
ON_USER_REQUEST **221**
OUT_MEASUREMENT.....131, **179**
OVERWRITE.....145, **180**
PBYTE..... **31**
PHONE_NO.....168, **183**
PHYS_UNIT.....
...50, 54, 73, 156, 180, **184**, 227, 230, 232
PLONG..... **31**
PLONGLONG **31**
PRG_CODE.....**160**
PRG_DATA **160**
PRG_RESERVED **160**
PROJECT.....**185**
PROJECT_NO.....140, **186**
PWORD..... **31**
RAM.....**162**
RAT_FUNC..... **82**
READ_ONLY 50, 54, 73, **187**, 241
READ_WRITE 145, 156, **188**
RECORD_LAYOUT 171, **189**
REF_CHARACTERISTIC 131, 135, **195**
REF_GROUP.....**196**, 241
REF_MEASUREMENT 135, **197**
REF_MEMORY_SEGMENT .....
.....54, 73, 156, **198**
REF_UNIT 82, **199**, 237
REGISTER **162**
RES_AXIS **50**
RESERVED **162**, 189, **200**
RIGHT_SHIFT 64, **201**
RIP_ADDR_W/_X/_Y/_Z/_4/_5.....**202**

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

RIP_ADDR_W/_X/_Z/_4/_5 189
ROM..... **162**
ROOT..... 135, **204**
ROW_DIR **123**, 148
SBYTE **31**
SERAM **162**
SHIFT_OP_X 189
SHIFT_OP_X/_Y/_Z/_4/_5 ..... **205**
SI_EXPONENTS..... **207**, 237
SIGN_EXTEND..... 64, **206**
SLONG **31**
SRC_ADDR_X 189
SRC_ADDR_X/_Y/_Z/_4/_5 ..... **208**
STATIC_ADDRESS_OFFSETS .. 189, **209**
STATIC_RECORD_LAYOUT 189, **210**
STATUS_STRING_REF..... 82, **212**
STD_AXIS..... **51**
STEP_SIZE.....50, 54, 73, **213**, 227, 230
STRICT_DECREASE..... **173**
STRICT_INCREASE **173**
STRICT_MON..... **173**
STRUCTURE_COMPONENT **214**, 234
SUB_FUNCTION 131, **216**
SUB_GROUP..... 135, **217**
SUPPLIER 168, **218**
SWORD **31**
SYMBOL_LINK .....
.....54, 65, 73, 145, 156, **219**
SYSTEM_CONSTANT..... 168, **220**
TAB_INTP..... **83**, 86
TAB_NOINTP..... **83**, 86
TAB_VERB **83**, **89**
TRANSFORMER..... 171, **221**, **273**
TRANSFORMER_IN_OBJECTS. 221, **225**
TRANSFORMER_OUT_OBJECTS.....
.....221, **226**
TYPEDEF_AXIS 171, **227**
TYPEDEF_BLOB.....171, **229**
TYPEDEF_CHARACTERISTIC ...171, **230**
TYPEDEF_MEASUREMENT.....171, **232**
TYPEDEF_STRUCTURE 171, **234**
UBYTE..... **31**
ULONG..... **31**
UNIT 171, **237**
UNIT_CONVERSION 237, **239**
USER.....168, **240**
USER_RIGHTS 171, **241**
UTF16.....**114**
UTF32.....**114**
UTF8.....**114**
UWORD..... **31**
VAL_BLK **74**
VALUE..... **74**
VAR_ADDRESS **243**, 244
VAR_CHARACTERISTIC **244**, 251
VAR_CRITERION.....**245**, 251
VAR_FORBIDDEN_COMB.....**246**, 251
VAR_MEASUREMENT.....245, **247**
VAR_NAMING **248**, 251
VAR_SELECTION_CHARACTERISTIC ...
.....245, **249**
VAR_SEPARATOR **250**, 251
VARIABLES.....**162**
VARIANT_CODING.....171, **251**
VERSION 140, 168, **253**
VIRTUAL 156, **254**
VIRTUAL_CHARACTERISTIC.....73, **255**
WORD **31**

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Figure Directory

Figure 1 Structure of the MCD standards of ASAM AE 14
Figure 2 AXIS_RESCALE_X 61
Figure 3 Comparison quantity 75
Figure 4 DEPENDENT_CHARACTERISTIC 104
Figure 5 data deposition 106
Figure 6 Memory layout (mirrored segments) 161
Figure 7 Linear interpolation for a characteristic curve 203
Figure 8 VIRTUAL_CHARACTERISTIC 255
Figure 9 Schematic data flow of description data 258
Figure 10 Map with referenced normalization curves 270
Figure 11 Workflow between BLOB's and Transformer 274

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015

Table Directory

Table 1	formula operator compatibility	12
Table 2	formula operator notiation	12
Table 3	byte-order mark coding	18
Table 4	Hierarchic division of the keywords	19
Table 5	Predefined data types	29
Table 6	Map ASAM data types to predefined data types	32
Table 7	BIT_OPERATION	64
Table 8	Byte order (2 Byte values)- memory data deposition	67
Table 9	Byte order (4 Byte values) - memory data deposition	68
Table 10	Byte order (8 Byte values) - memory data deposition	68
Table 11	GUARD_RAILS	139
Table 12	MEMORY_SEGMENT	163
Table 13	Example of NLLM - variants coding	252
Table 14	Example of PUMKF - variants coding	252
Table 15	BNF Terminology	259
Table 16	Grammar in the extended Backus-Naur format	259
Table 17	IEEE-Floating-Point-Format (32-Bit)	267
Table 18	IEEE-Floating-Point-Format (64-Bit)	267
Table 19	Codes for scaling units (CSE)	271
Table 20	Members of TransformerStatus	276
Table 21	Parameter structure TransformerDescription	276
Table 22	Parameter structure DllDescription	277
Table 23	Parameter structure TransformationData	277
Table 24	Parameter structure TransformerResult	278
Table 25	Categories for <CATALOG>	284
Table 26	Categories for <ABLOCK>	284

Licensed to PRIVATE
Downloaded by 3100103956@zju.edu.cn on 18.03.2015



Association for Standardisation of
Automation and Measuring Systems

E-mail: support@asam.net
Internet: www.asam.net

© by ASAM e.V., 2015